Engineering Handbooks

Handbook 3 of 3

Cloud Delivery & DevOps

From a laptop to production, the way teams ship today.

For engineers who have shipped software before and are coming back to it. It assumes you know what a server, a build and a deployment are. It shows how each is done now, why it changed, and where to see it working in a real, public codebase.

1 · AI Engineering & Architecture
The new layer: models, retrieval, agents, evaluation, safety.

2 · Modern Software Engineering
Your skills, brought up to date: the service and its repository.

3 · Cloud Delivery & DevOps
From a laptop to production: pipelines, identity, the cloud.

Edition 1 · Reference implementation: `triage-assistant` at `handbook-1 (unknown)`

How to read this handbook

Every chapter is anchored to one real system: **triage-assistant**, an on-call assistant built as the template a team starts its AI services from. Alerts live in Postgres, owned by teams. People sign in with the organization's identity provider and see only their teams' alerts. A model answers their questions from those alerts and the team's runbooks, and cites what it used. The service was load-tested, broken on purpose, and deployed to AWS through a real pipeline. Its code is public at github.com/Parzon/triage-assistant.

Numbers in this handbook come from that system unless the text says otherwise, and each is given with the setup it was measured on. Code links point to the exact version this edition was checked against, so the line numbers beside an excerpt are the ones you will see on GitHub.

Three ways through this handbook, depending on what you need first:

IF YOU WANT TO	READ
See the whole delivery path once	Chapters 1, 3, 5 and 13: from laptop to production, pipelines, keyless identity, releases and rollback
Deploy a service to AWS	Chapters 6 to 9, then 12 and 14: one VM, infrastructure as code, networking, ECS, the real build, teardown
Compare platforms	Chapters 9 to 11: ECS, Kubernetes and EKS, Azure and Google Cloud

Each chapter has the same parts, so you can read at the depth you need:

IN ONE SENTENCE
What the topic is, in words you can use in a meeting.

AT A GLANCE
The points to keep if you read nothing else.

THEN AND NOW
How it was usually done, how it is done today, and why it changed.

FIGURES
The mechanism, drawn. The text explains the figure.

IN THE REPO
Where the idea lives in the reference code, with the lines quoted beside the explanation.

HOW TO SAY IT
A few sentences for a design review.

TRAPS
What goes wrong in practice, and the fix.

GOING DEEPER
Detail for when you need it. Collapsed on the web, printed in full.

QUESTIONS
To check your understanding. Answers follow.

Contents

PART I · WHAT CHANGED

1 From laptop to production

PART II · SOURCE AND BUILD

2 Git and the pull request

3 CI/CD and artifacts

4 The software supply chain

PART III · IDENTITY

5 Identity without keys

PART IV · GETTING CODE ONTO SERVERS

6 The deployment ladder, and one VM done properly

7 Infrastructure as code

PART V · RUNNING ON AWS

8 Networking on AWS

9 Containers on ECS Fargate

10 Kubernetes and EKS

11 The same ideas on Azure and Google Cloud

PART VI · THE REFERENCE BUILD

12 The reference build, end to end

PART VII · OPERATING IT

13 Environments, releases and rollback

14 Cost, teardown and lessons learned

APPENDICES

A Glossary

B Decision index

C Repository index

D Trap index

PART I · WHAT CHANGED

1 From laptop to production

IN ONE SENTENCE

A change now travels one automated path: a pipeline reviews and tests it, builds it once into an immutable image, and deploys it by declaring which version should run, so a rollback is the same operation as a release.

AT A GLANCE

- Every change goes through a pull request, automated checks and a review before it reaches the main branch.
- A release is a version tag. It produces one image per component, built once and never rebuilt for another environment.
- Environments differ only in configuration. The bytes tested in staging are the bytes production runs.
- Deploying means "run version X". Rolling back means "run the previous version" through the same pipeline.
- Infrastructure is code in the same repository, changed through pull requests like everything else.
- A Windows laptop works when the repository, the toolchain, the agent and the container engine live inside a WSL2 distribution's own file system; the Windows drive is for the editor and the browser.

THEN AND NOW

THEN	NOW	WHY IT CHANGED
Copy files to a server over SSH or FTP, or run a deploy script by hand	→ **A pipeline builds, tests and deploys; people approve, they do not type commands on servers**	Manual steps are skipped under pressure, and nobody can see what was done.
Build on each server from a branch	→ **Build once in CI, push to a registry, promote the same image everywhere**	Two hosts building the same commit can end up with different bytes.
A configuration file per environment, often holding secrets, in the repo or on the server	→ **Configuration from the environment; secrets from a secret store**	The same image must run in every environment, and secrets must not travel with it.
Hand-configured servers that drift apart	→ **Infrastructure as code; servers and containers replaced, never patched in place**	Rebuilding from code is repeatable; patching by hand is not.

THEN	NOW	WHY IT CHANGED
A big release night every few weeks	→ **Small releases, any time, with zero-downtime deploys and an automatic rollback**	Small changes are easier to review, test and undo.
A separate operations team runs what developers hand over	→ **The team that builds the service also runs it, with the pipeline and monitoring as shared tools**	The people who know the code fix it fastest, and feel the cost of fragile releases.

The path a change takes

The figure below is the path the reference service actually uses, end to end. A developer's only actions are opening a pull request, merging it once it is reviewed and green, and pushing a version tag. Everything after the tag is automated, gated and reversible. Each stage is a chapter of this handbook.

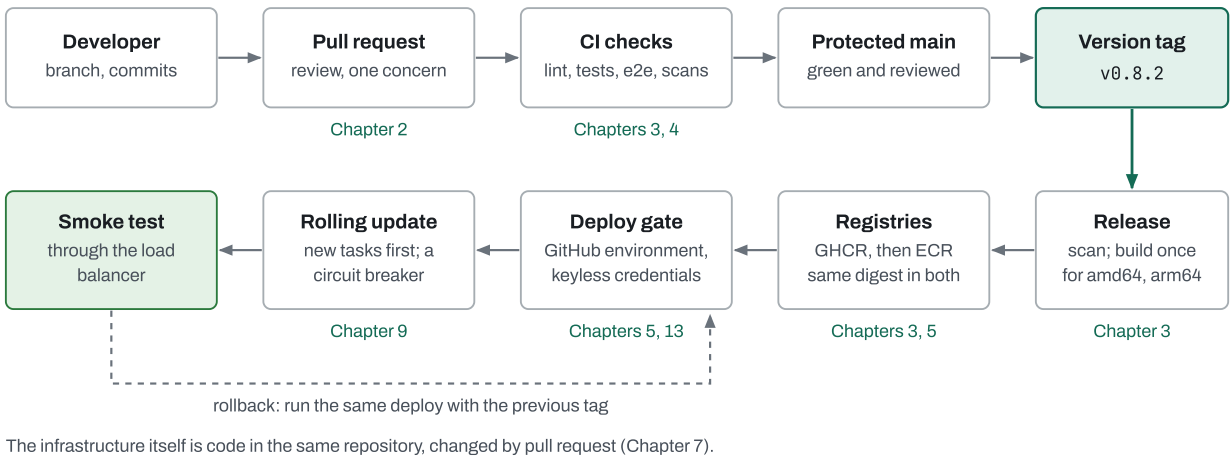


Figure 1.1 The delivery path of the reference service. After the version tag, no person touches a server. A rollback is not a special procedure: it is the deploy, run with an older version.

Three properties make this path trustworthy, and the rest of the handbook explains how each is achieved.

- **Everything is reviewed.** Code, infrastructure, pipeline definitions and even accepted security risks change only through pull requests.
- **Everything is reproducible.** The release builds from locked dependencies and pinned base images. The infrastructure is rebuilt from code, and a drift check reports any change made by hand.
- **Everything is reversible.** Releases roll forward and back through the same deploy, and database changes are written so the previous release keeps working.

What the reference build proved

A path is only as good as its evidence. These properties were each tested on the reference build, on AWS, with traffic flowing.

PROPERTY	HOW IT IS ACHIEVED	EVIDENCE
No long-lived keys anywhere	OIDC roles for CI, task and instance roles for workloads, sign-in sessions for people	Chapter 5
Reproducible infrastructure	CDK in the repository; a diff reviewed before every change; drift detection	A manual change was detected and replaced through a pull request
Immutable, traceable artifacts	Immutable image tags; the same digest in both registries; deploys by exact version	Pushing a different image over an existing tag was refused
Zero-downtime deploys	Rolling updates that start the new version first and wait for it to be healthy	352, 967 and 1,441 probe requests during three deploys, none failed
Safe failure	A deployment circuit breaker; startup checks that give up; a load balancer that keeps the old version serving	A deliberately bad release was rolled back automatically
Safe rollback	Additive database migrations; a code-only rollback when the database is ahead	Rollbacks across a migration on a VM and on ECS
Scale and resilience	Two tasks across two availability zones; autoscaling from two to four	A killed task caused no failed requests; a 7-minute load test of 33,601 requests had none fail

One artifact, promoted through environments

The single most important rule on the path is **build once, promote the same artifact**. A release builds each component exactly once. Staging, production and any other environment then run those same bytes, identified by a version tag and, underneath it, a content digest that cannot change. Only the configuration differs: addresses, limits and secrets arrive from the environment at runtime.

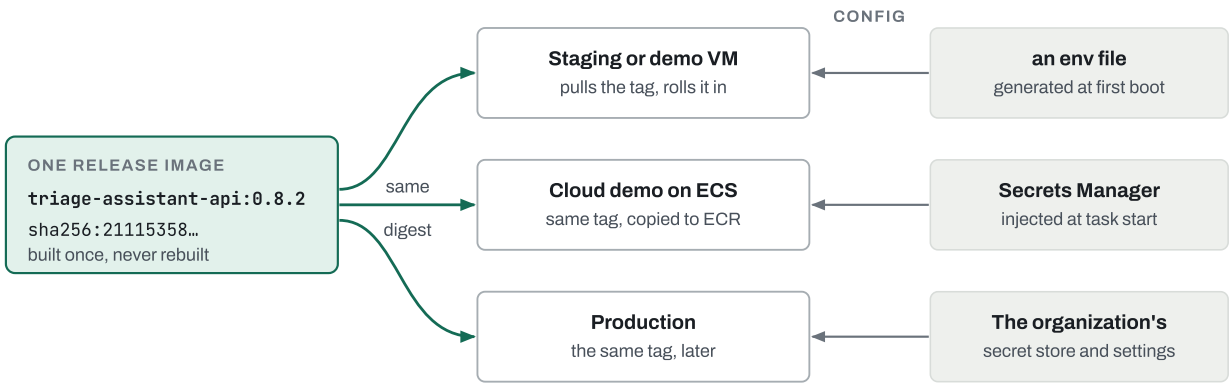


Figure 1.2 The image is the constant; the configuration is the variable. Because nothing environment-specific is baked into the image, a problem found in staging is a problem in the exact bytes production would run.

Two consequences follow. A version must name exact bytes, so nothing is ever deployed as `latest`: two pulls of `latest` can be different code, and a rollback has no name to return to. And a host never builds from source: building on a server needs compilers and internet access there, and two hosts building the same commit can still differ, because a package was republished or a base image patched in between.

IN THE REPO `.github/workflows/release.yml` the release, described at the top of its own workflow

Pipelines are files in `.github/workflows/`, reviewed like code. The release workflow opens with a comment that states its whole contract: what a tag publishes, under which names, for which processors, and what happens before the release notes are written.

- One tag produces three image names, the exact version, the minor line and the commit, and moves `latest`, which nothing here deploys: the comment says so.
- It tests what it published, not what it built: it pulls the release on both processor types and runs the stack from it.
- The same builds run, without pushing, on any pull request that changes their inputs, so a broken release build is caught before the tag.

```yaml
name: Release

# A version tag on main publishes the production images (api, web and the
# TLS edge) to GitHub's container registry, for linux/amd64 and linux/arm64
# (Apple Silicon, AWS Graviton) under one tag:
#   git tag -a v1.4.0 -m "v1.4.0" && git push origin v1.4.0
#   → ghcr.io/<owner>/triage-assistant-api:1.4.0, :1.4, :sha-<commit>, :latest
#      ghcr.io/<owner>/triage-assistant-web:  the same tags
#      ghcr.io/<owner>/triage-assistant-edge: the same tags
# :latest (docker/metadata-action's default) moves with every release, before
# the smoke test below: deploy by version, never by latest.
# Then it pulls what it published, on both architectures, and runs the
# stack from those images before writing the release notes. A host deploys
# it with `make deploy tag=1.4.0` (docs/runbooks/demo-vm.md).
#
# Pull requests that change the images' inputs run the same builds, on both
```

ADR-0011 ACCEPTED

Releases are version tags that publish images; one host deploys by rolling

PROBLEM	Building on each host made every host a build machine, and a plain container restart refused requests for 6.7 seconds.
DECISION	A release is a `vX.Y.Z` tag on main that publishes images. Hosts pull and never build. A host deploys by starting the new version beside the old one.
SEE IT IN	release.yml builds and publishes; scripts/deploy.sh rolls a release onto one host.
RESULT	Zero failed requests during the swap, and a clean host serving 41 seconds after checkout.

The environments

Each environment has one job. The table is the reference service's set, and it is a reasonable default for any team.

ENVIRONMENT	ITS JOB	DATA
Laptop, development	Write and debug code: hot reload, a debugger, a fake model provider, a demo identity provider	Seeded, disposable
Laptop, production shape	Check it behaves like production before pushing: the production images, proxy and server settings	Seeded
CI	The same checks on a clean machine for every pull request, including browser tests against the production shape	None kept

ENVIRONMENT	ITS JOB	DATA
Staging or demo	Shared and always on: demos, pilots, real provider keys, failure drills	Demo data, never production data
Production	Users, signed in through the organization's identity provider	Real

Production data never flows down to laptops. When a bug genuinely needs production-shaped data, the answer is an anonymized snapshot in staging, or synthetic data with the same shape, not a copy of production on someone's machine.

The same service at three sizes

A path like this does not require a large platform. The same service, with the same images, runs at three sizes, and each step up adds pieces only when a named need appears.

	A NEW PROJECT, WEEK ONE	ONE VIRTUAL MACHINE	A MANAGED CLOUD PLATFORM
Compute	api and web in Compose	gunicorn workers, rolling deploys	ECS on Fargate behind a load balancer
Data	Postgres	plus a connection pooler; backups off the host	RDS or Aurora; Multi-AZ in production
Shared state	none	Valkey for rate-limit counters	ElastiCache for Valkey
Edge	localhost	Caddy for HTTPS, then nginx	A load balancer with a managed certificate
Watching	JSON logs and a metrics endpoint	Prometheus, Grafana, Alertmanager, Jaeger	CloudWatch, or managed Prometheus and Grafana
Delivery	CI with tests and scans	plus releases to a registry and a deploy script	plus a cloud registry and infrastructure as code

The reference build ran all three. The cloud column ran on AWS without a domain name, a real identity provider or a paid model, which the service's production checks allow only when each exception is named. Chapter 12 tells that story.

> HOW TO SAY IT
>
> *"We build each release once and promote the same image through every environment; only configuration changes."*
>
> *"Deploying and rolling back are the same pipeline with a different version. Nobody runs commands on production servers."*

When the laptop is Windows

Everything in this handbook assumes a Linux shell, a container engine and a coding agent that can run the project's checks. A corporate Windows laptop can provide all three, but the choice of *where the work lives* decides whether the stack runs at full speed or breaks in ways that look like bugs in your code. The short version: the repository, the toolchain and the agent live inside a WSL2 distribution's own file system, the container engine runs there too, and the project ships a dev container so the setup is the same on every machine.

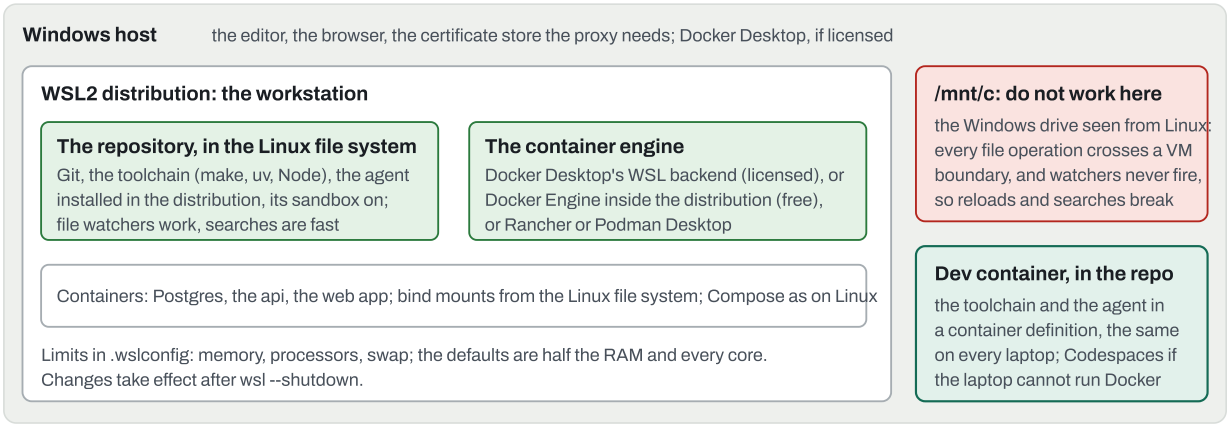


Figure 1.3 A Windows laptop arranged for the reference build. The distribution's own file system is the workstation; the Windows drive is for the editor and the browser. Microsoft's own comparison lists cross-file-system access as the one thing WSL2 does slower than WSL1, and both Microsoft and Docker say to keep the files on the Linux side.

Before the first install, one ticket. Almost everything below installs per user without administrator rights: the editor, Git, the container desktop, the agent. Five things do not, and they are better asked for together than discovered one at a time: enabling the Linux subsystem and the virtual-machine platform (a reboot), the device-management policy for it (it can silently disable custom networking and ignore the limits you set), either a container-desktop seat or approval for the free engine, an allowlist entry for the agent's executable and the shells it spawns in the endpoint-security tool, and the proxy's root certificate as a file with the proxy's address. A locked-down laptop with those five settled is a normal workstation; without them it is a week of puzzles.

Where the files live. Keep every repository in the distribution's home directory, never under `/mnt/c`. Two mechanisms, not one: file operations across the boundary go through a file-server protocol with a round trip per metadata call, and file-change events raised on the Windows side never reach Linux, so a development server's reload and a test watcher silently stop working. The coding agent's own

troubleshooting page lists the same symptom (fewer search matches than expected) with the same fix. Open the Linux side from Windows with `explorer.exe .` or the `\\wsl$` share when you need the file manager.

Which container engine. Docker Desktop installs per user without administrator rights and uses the distribution as its backend, with the limits set in `.wslconfig` rather than in its own settings; it needs a paid subscription for commercial use in a company with more than 250 employees or more than ten million dollars in revenue, at roughly nine to twenty-four dollars per user per month depending on the tier, so the question goes to procurement before the first install. Docker Engine installed inside the distribution is the same open-source engine without the desktop product, free, and runs with `systemd` enabled in `wsl.conf`; you manage its updates, certificates and DNS, and the two must not coexist in one distribution. Rancher Desktop and Podman Desktop give a Docker-compatible socket without the license, and need WSL or Hyper-V enabled once by an administrator. Microsoft now ships its own container runtime inside WSL, which needs no engine at all, but it has no Compose support yet, so it cannot run this stack today.

The dev container. The portable form of all this is a container definition in the repository: a base image with the toolchain, pinned versions of the package managers, the agent installed through its published dev-container feature, and the host's Docker socket passed in so Compose works from inside. The editor reopens the folder in that container, and a new laptop goes from clone to a running stack without a hand-built environment; the agent's own reference container adds an outbound firewall with an allowlist, which is the setting its documentation requires for unattended runs. The same definition runs in a hosted codespace when a laptop cannot run Docker at all: at the list price of a four-core machine, six hours a day for a month is roughly fifty dollars per developer, and the source then lives on the vendor's cloud, which is a policy question before it is a cost question.

The agent on Windows. Install it inside the distribution with the Linux installer, not from PowerShell; a native Windows install works too, with PowerShell or Git Bash as its shell, but the command sandbox runs only on Linux, macOS and WSL2, so unattended runs on native Windows have no isolation. In WSL2 the sandbox needs two small packages and excludes Docker commands by configuration, because Docker is incompatible with it; an excluded command runs with full access, and the agent can edit the Compose file before running it, so keep Compose commands behind a permission prompt. Two things an administrator must know: policy deployed through the Windows registry is not read inside the distribution unless a key says so, so a laptop can be locked down on one side and open on the other; and Windows-side endpoint sensors do not see processes inside the WSL virtual machine unless the vendor's WSL plug-in is installed, which Microsoft's own product has.

Proxies and inspected TLS. A corporate proxy that re-signs TLS needs its root certificate in five places, not one: the Windows store (the installers validate against it), the distribution's certificate bundle, every container image that fetches packages during a build, and the tools that carry their own bundles rather than the system's, which includes the Python package manager (a system-certificates switch) and npm (a certificate-file setting). The agent trusts the operating-system store by default and takes `HTTPS_PROXY` and `NO_PROXY` from its settings file's environment block, which matters because its background processes inherit the first shell that started them, not yours; it speaks plain HTTP proxies only, so a SOCKS or NTLM proxy needs a gateway in front. A VPN that changes DNS usually breaks the distribution's generated resolver file first; turning generation off and writing the resolver by hand is the documented fix. If a repository must stay on the Windows drive, the development servers' file watchers can fall back to polling through their environment variables, at a cost in battery and latency.

Git across the boundary. Commit a `.gitattributes` with `* text=auto eol=lf` so line endings are a repository decision rather than each developer's `core.autocrlf`, and set that option to `false` on the Windows side; a shell script with carriage returns fails inside a container with a message about a file that plainly exists. On a Windows checkout, enable long paths in Git and in Windows (the 260-character limit is reached by any `node_modules`), turn on Developer Mode for symbolic links, and expect the executable bit to be lost on NTFS: set it in the index with `git update-index --chmod=+x`. Inside the distribution's file system none of this applies, which is one more reason to work there.

THE WINDOWS GOTCHAS THAT COST THE MOST TIME

SYMPTOM	CAUSE	FIX
The dev server never reloads; tests run on stale files	The repository is under /mnt/c; file events do not cross the boundary	Move it to the distribution's home directory.
Everything is slow: installs, searches, Git status	Cross-file-system access, or Windows binaries shadowing Linux ones on PATH	Linux file system; install Node and Python inside the distribution; trim the Windows PATH from the Linux one if needed.
A script fails in the container with "no such file", and the file exists	Carriage returns from a Windows checkout	The `.gitattributes` rule; renormalize once; `core.autocrlf` off.
Builds die mid-way; the machine freezes	The distribution took half the RAM, or a runaway tool took all of it	Limits in `.wslconfig`; the agent's tool memory cap; restart the distribution after changing them.
Docker Desktop refuses to start, or "cannot connect to the daemon"	An engine was also installed inside the distribution; or WSL is below the version Desktop needs	One engine per distribution; update WSL with the web download when the store is blocked.
Package downloads fail with certificate errors in a build	The proxy's root certificate is in Windows but not in the distribution or the image	Install it in all three places; pass it to the build.
DNS stops after connecting to the VPN	The generated resolver file points at a server the VPN hides	Disable generation in `wsl.conf`; write the resolver yourself; or mirrored networking mode.

SYMPTOM	CAUSE	FIX
The agent's sandbox is unavailable	Native Windows, or WSL1, or a distribution that restricts user namespaces	WSL2 with the two packages; a profile for the sandbox on distributions that need it; otherwise a container for unattended runs.
A published port is unreachable from the browser	Networking mode and port forwarding interact; mirrored mode has its own exceptions	Keep the default NAT mode unless a tool needs otherwise; check the ignored-ports setting in mirrored mode.
A colleague's laptop behaves differently from yours	Hand-built environments	Ship the dev container definition in the repository; the toolchain, the agent and its allowlist come with it.

Traps

SYMPTOM	CAUSE	FIX
A rollback has nothing to go back to	The service was deployed as `latest`, which names whatever was pushed last	Deploy exact version tags, or digests, from a registry with immutable tags.
Two servers on the same version behave differently	Each built from source; a dependency or base image changed between builds	Build once in CI; hosts only pull.
Every release replaces a component that did not change	Image layers record file times, and every CI checkout gets new ones, so no two builds are byte-identical	Compare builds by a hash of their inputs, and replace a component only when its inputs changed.
An image tag now points at different code	Anyone who can push a package can re-push a tag	Make the release workflow the only pusher, use immutable tags, verify signed build provenance, or deploy by digest.

SYMPTOM	CAUSE	FIX
Images from a public repository can be pulled by anyone	Packages built by a public repository's workflow are public by default on GitHub's registry	Expect it: never put a secret in an image.

Questions

1. Why should staging and production run the same image rather than each building from the same commit?
2. What is the difference between a release and a deploy in this path?
3. Why is `latest` never deployed?
4. What makes a rollback safe when the release being undone changed the database?

Answers

1. Because a build is not reproducible bit for bit: packages get republished and base images patched between builds. Running the same image, identified by its digest, means what was tested is exactly what runs.
2. A release is a version tag that builds, scans and publishes images once. A deploy tells an environment to run a particular released version. One release can be deployed many times, to many environments, and an older one can be deployed again as a rollback.
3. It is a movable label: two pulls can return different code, and there is no name to roll back to. A deploy must name exact bytes.
4. Writing migrations so they only add (expand) before anything is removed (contract), so the previous release still runs on the newer schema. A rollback then deploys the older code and leaves the schema alone. Chapter 13 covers it.

PART II · SOURCE AND BUILD

2 Git and the pull request

IN ONE SENTENCE

Git stores immutable snapshots and moves labels between them, and the pull request is the single gate through which every change, reviewed and tested, reaches the always-deployable main branch.

AT A GLANCE

- Git's model is small: commits never change, and branches, tags and HEAD are labels pointing at commits.
- A file lives in three places: your working tree, the staging area (the next commit), and the last commit.
- Trunk-based development is the default for services: one main branch, always deployable, fed by short-lived branches.
- Branch protection turns the pull request into a gate: required checks, required reviews, and a linear history.
- An issue number threads through the branch, the commits, the pull request and the release, so every change can be traced to its reason.

THEN AND NOW

THEN	NOW	WHY IT CHANGED
Centralized version control: one server, locks, commits straight to trunk	→ **Git: every clone is a full copy, and changes arrive through pull requests**	Cheap branches and offline work, with review built into the flow.
Long-lived feature and release branches (GitFlow)	→ **Trunk-based: branches live for hours or days, main is always deployable**	Long branches drift and make merges painful; services ship continuously.
Code review by meeting or email, sometimes after the merge	→ **Review in the pull request, before the merge, beside CI results**	A reviewer sees the diff, the tests and the evidence together.
Every small commit kept on trunk, with merge commits everywhere	→ **Squash merges: one commit per change on main, and a linear history**	One change is one revertible commit; the detail stays in the pull request.

THEN	NOW	WHY IT CHANGED
Tickets and commits linked by hand, or not at all	→ **An issue number in the branch name and `Closes #12` in the pull request link them automatically**	Traceability nobody has to remember to maintain.
Review rules held in people's heads	→ **Branch protection, code owners and a pull request template**	Rules that are enforced are followed by humans and AI coding tools alike.

The model: snapshots and labels

Most confusion with Git disappears once the model is clear. A **commit** is a snapshot of the whole project plus a pointer to its parent commit. Its identifier is a hash of the snapshot, the parent, the author, the time and the message, so a commit can never change: change anything, even the message, and you have a different commit. A **branch** is a label pointing at one commit, which moves forward when you commit. A **tag** is a label that stays put: committing does not move it, and by convention nobody does, though `git tag -f` and a force-push can unless a tag ruleset forbids it. **HEAD** is where you are.

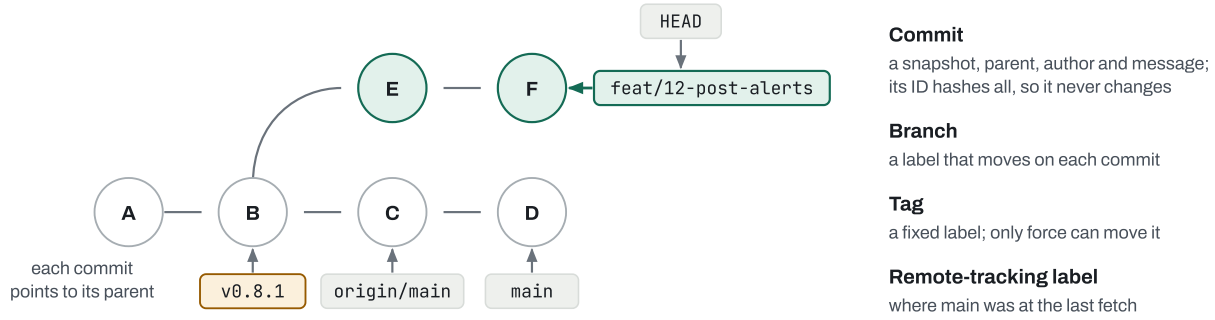


Figure 2.1 Everything in Git is commits that never change and labels that point at them. Pushing asks the remote to move its label; it refuses unless the move is a fast-forward, which is why a force-push is dangerous.

A remote is simply another copy of the same database. `fetch` downloads its new commits and moves your `origin/main` label. `pull` is a fetch followed by a merge. `push` uploads your commits and asks the remote to move its branch label, which it refuses unless your history contains its history.

One practical consequence matters for operations: **a running service reads the disk, not Git**. On restart it loads whatever the working tree holds, including uncommitted edits or half-resolved conflict markers. That is why a production host is never a developer's checkout: a deploy puts a specific, built release on it (Chapter 3).

Three places a file lives

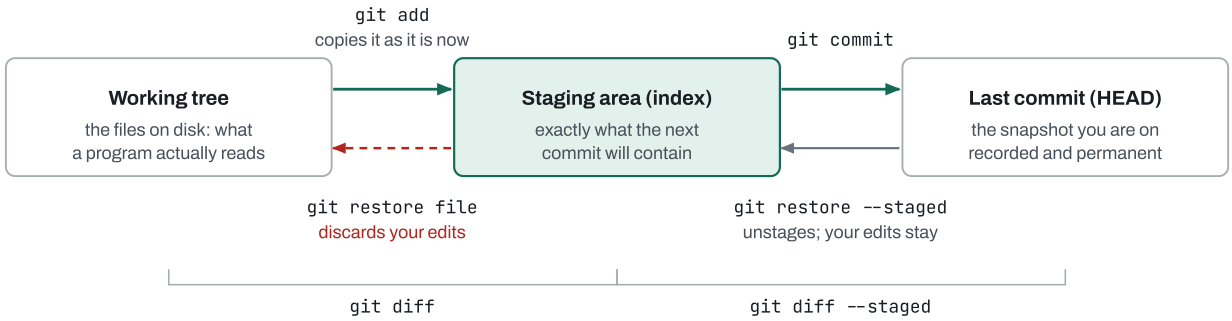


Figure 2.2 Staging copies a file as it is at that moment. Edit it again and the two differ, which is why `git status --short` can show a file as both staged and modified.

In `git status --short` the left column compares the staging area with the last commit, and the right column compares the working tree with the staging area. `MM` means "staged, then edited again": add it again. Untracked files appear in neither diff, which is how a new file gets forgotten.

Merge, rebase and squash

Three ways to bring a branch's work into main produce three different histories. Teams pick one per repository; the reference service squash-merges and requires a linear history.

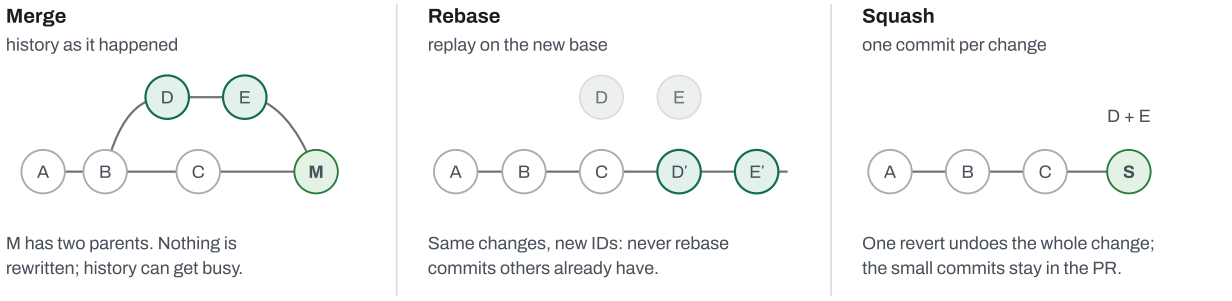


Figure 2.3 Merge keeps history as it happened, rebase rewrites it as if the work started later, and squash records each change as one commit. The pull request title becomes the squashed commit's message, so it should read well on its own.

A conflict looks the same in all three. Git writes both versions into the file between `<<<<<<<`, `=======` and `>>>>>>>` markers; you edit the file to its final form, stage it and continue, or abort and start again. If a commit you rewrote disappears, `git reflog` still knows where every label pointed recently.

Trunk-based development

For a service that deploys continuously, the default is **trunk-based development**: one permanent branch, `main`, always deployable, and short branches that live for hours or days, each merged through a pull request and then deleted. GitHub flow is the same idea. **GitFlow**, with its `develop`, `release/*` and `hotfix/*` branches, exists for software shipped in several supported versions at once, such as installed products. Recognize it; do not pick it for a service.

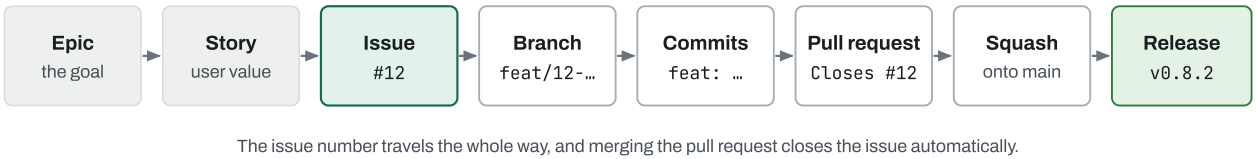


Figure 2.4 From a goal to a shipped release. Nobody copies commit IDs into tickets: the platform links them. With Jira, the ticket key in the branch name and commit message does the same.

The pull request as a gate

The pull request is where a change is explained, verified and reviewed. Its value depends on the rules around it, and on GitHub those rules are **branch protection**. On the reference repository, `main` requires:

- six green checks: `lint`, `test-api`, `build`, `security`, `web-build` and `e2e`, on a branch that is up to date with main;
- one approving review, including one from the code owners of every path touched, with approvals dismissed when new commits arrive;
- a linear history; with squash as the only merge method the repository allows, every change lands as one commit.

Code owners say whose approval a path needs. The pull request template asks the author for the evidence a reviewer needs, so verification is written down rather than assumed.

IN THE REPO `.github/pull_request_template.md` and `.github/CODEOWNERS`

Every new pull request opens with this template already filled in as its description. The checklist asks for exactly the evidence that would have caught past failures.

- "How I verified it" wants commands and numbers, not "tested locally".
- The checklist asks about failure paths, backward-compatible migrations and bounded metric labels: the three things most often missed.
- The code owners file treats accepting a vulnerability or exempting a secret as decisions that need an owner's review, like an architecture decision.

```
Closes #

## What and why

<!— One paragraph: the change, and the reason for it. —>

## How I verified it

<!— Commands and numbers, not "tested locally". E.g. `make check`,
     `make e2e`, `make drills d="db-freeze"`, a before/after latency. —>

## Checklist

- [ ] `make check` passes locally (lint, types, tests; CI adds the scans, image checks and e2e)
- [ ] Tests cover the failure paths, not only the happy path (dependency down / hung, invalid input,
forbidden)
- [ ] Browser- or nginx-visible change: `make prod-up && make e2e`
- [ ] Database path, pools or timeouts touched: the database drills (`make drills d="db-freeze db-hang
pgbouncer-freeze"`)
- [ ] Migration: backward compatible (expand now, contract in a later release), `lock_timeout`,
concurrent indexes
- [ ] New metric labels are bounded; new alerts have a promtool test
- [ ] New dependency: one line on why, and why not the standard library or an existing one
- [ ] Docs: the handbook chapter, an ADR for a decision, a runbook for a procedure, the gotcha list for
a trap
- [ ] No secrets, personal data or real customer data in code, tests, fixtures or logs
```

```
# Decisions are reviewed by whoever owns the architecture
/docs/adr/             @Parzon

# Accepting a vulnerability or exempting a secret is a decision too
/.trivyignore.yaml     @Parzon
/.gitleaks.toml        @Parzon
/.gitleaksignore       @Parzon
```

Habits that keep history useful

- **Branch names** as `type/issue-short-description`, such as `fix/31-pool-leak`. Create the branch before the first edit.
- **Commit messages** in the Conventional Commits style (`feat:`, `fix:`, `docs:`), with an imperative subject and the reason in the body.
- **Small pull requests** with one concern each. This matters more with AI coding tools, which produce large diffs quickly; a large diff gets skimmed, not reviewed.
- **Never commit secrets**, generated output or environments (`.venv`, `node_modules`, build directories). A secret that reaches a commit is rotated, not just deleted: on a public repository it was copied the moment it was pushed.

> **HOW TO SAY IT**
>
> *"We're trunk-based: short branches, squash-merged through pull requests that need green checks and a code owner's review. Main is always deployable."*
>
> *"Every change traces back to an issue through the branch name and the pull request, so we can answer why any line exists."*

When an agent wrote the commits

Nothing in the pull-request gate changes when a coding agent authored the change; what changes is how much weight each part carries. The person who opens the pull request owns it: the kernel and the Apache projects both settled on that rule, with an attribution trailer on the commit (`Assisted-by:` or an equivalent) so the history says what was generated. A commit-message hook enforces the trailer. Small pull requests matter more: one vendor's review service reports an average of 7.5 findings on pull requests over a thousand lines and half a finding under fifty, which is as much about what reviewers can read as about what agents write. The files that define the gates, the pipeline, the lint configuration, the hooks and the lockfiles, get a code-owners entry, so a change to them needs a named reviewer and an agent cannot loosen a rule inside the same pull request that needs it loosened. Required status checks stay the merge gate; a review bot comments and never blocks, runs in comment-only mode for a few weeks first, reads its rules from a file the repository owns, and is capped on nits. Review effort follows the risk tier of the change: strict for authentication, personal data, money and migrations, sampled elsewhere. The four layers that this sits on top of are drawn in chapter 13 of the software handbook.

Traps

SYMPTOM	CAUSE	FIX
An undo of one experiment also lost other uncommitted work in the same file	`git checkout <file>` (or `git restore`) discards every unstaged change in that file (staged changes are kept: the file is restored from the staging area)	Commit first, or undo the experiment with a reverse patch.
Teammates' branches no longer line up with yours after a rebase	Rebasing commits others already have gives them new IDs	Rebase only your own unpushed work; use `--force-with-lease` on your own branch, never on a shared one.
A service restarted mid-merge runs conflict markers as code	A running service reads the working tree, not Git	Production is never a developer checkout; deploy built releases.
A solo owner cannot merge their own pull request	GitHub never lets an author approve their own pull request	Until a second reviewer exists, merge with the admin bypass, deliberately and noted in the description.
Branch protection is missing on a private repository	It needs a paid plan for private repositories	Budget for it, or keep the repository public.

SYMPTOM	CAUSE	FIX
Shell scripts fail with odd errors after a checkout on Windows	Line endings converted to CRLF	A `.gitattributes` file that forces LF.
Your branch vanishes and files disappear while you work	Another person or automated agent switched branches in the same checkout	One `git worktree` per parallel session; confirm the current branch before every commit; stage explicit paths, never everything.
"No checks" right after opening a pull request	The platform had not attached the runs yet	Look at the workflow runs for the branch rather than trusting an empty answer.
A commit went in even though lint had just failed	Commands chained with `;`, which runs the next step regardless	Chain with `&&`, so a failing check stops the chain.
Nobody can tell which commits were generated	No attribution in the history	An `Assisted-by:` trailer, enforced by a commit-message hook; the committer still owns the change.
The review bot approved; a real bug merged	The bot was the gate and its precision is unknown	Comment-only mode with repo-owned rules; required checks and a human approval remain the gate.

- GOING DEEPER **Recovering from the usual mistakes**

A commit on the wrong branch. Create the right branch where you are (it keeps the commit), then move the wrong branch back one commit. Uncommitted work travels with you when you switch branches, so switching before committing is often enough.

A rebase gone wrong. `git reflog` lists every position HEAD has had. Reset the branch to the entry before the rebase.

Files deleted from disk by a branch switch. If they were committed on any branch, `git show branch:path` prints them back. Untracked files are never touched by a switch.

A secret pushed to a public repository. Rotate it first. Rewriting history afterwards removes it from the repository, but not from forks, caches and anyone who already

cloned.

Questions

1. Why can a commit never change, while a branch changes all the time?
2. When is rebasing safe, and when is it not?
3. What does a squash merge give a team, and what does it cost?
4. Which branch protection settings turn a pull request into a real gate?
5. Why should a production server never run from a Git checkout?

Answers

1. A commit's ID is a hash of its snapshot, its parent, its author and time, and its message, so any change makes a different commit. A branch is only a label pointing at a commit, and it moves forward with each new commit.
2. It is safe on your own commits that nobody else has. It is unsafe on commits others already have, because rebasing gives them new IDs and their copies no longer match.
3. One commit per change on main, so one revert undoes a whole feature and the history reads as a list of changes. The small commits are no longer on main, but they remain visible in the pull request.
4. Required status checks on an up-to-date branch, required reviews including code owners, stale approvals dismissed on new commits, and a linear history. Without them, a pull request is only a suggestion.
5. A service reads its files from disk, including uncommitted edits and conflict markers, and a checkout can drift from any recorded version. A deploy should place a specific built release on the host.

PART II · SOURCE AND BUILD

3 CI/CD and artifacts

IN ONE SENTENCE

Continuous integration checks every change before it can merge; continuous delivery turns a tagged commit into immutable, versioned images and deploys exactly those, so what was tested is what runs.

AT A GLANCE

- **CI** is the merge gate: lint, types, unit and integration tests, an image check, scans and browser tests, on every pull request.
- **CD** builds an artifact, pushes it to a registry and deploys it. The artifact is a container image named by version and identified by a digest.
- Four rules: build once and promote; artifacts are immutable; environments differ only in configuration; rollback is deploying the previous artifact.
- A tag can move; a digest cannot. Deploy by exact version from a registry with immutable tags, or by digest.
- One tag can serve several processor types through a multi-architecture index, and each architecture is built natively.

THEN AND NOW

THEN	NOW	WHY IT CHANGED
A build server configured through its web interface, with plugins	→ **Pipelines as YAML files in the repository, run on fresh hosted machines**	The pipeline is reviewed, versioned and reproducible like the code.
Nightly builds that tell you tomorrow what broke today	→ **Checks on every pull request, before the merge**	A failure is caught on the change that caused it.
Build outputs copied to a shared drive or server	→ **Images in a registry, named by version, verified by content digest**	Every environment fetches the same bytes, and can prove it.
One processor type for every server	→ **One version tag pointing at images for amd64 and arm64**	Arm servers, such as AWS Graviton, cost about a fifth less, and laptops run on Arm too.

THEN	NOW	WHY IT CHANGED
Trust that the build server built what you think	→ **Build provenance and a software bill of materials attached to each image**	You can look up which commit and workflow produced an image, and what is inside it; signing the record makes it proof.

Continuous integration: the merge gate

On every pull request, the reference service runs six jobs on fresh machines: five in parallel, and the image build once lint has passed. All six are required by branch protection (Chapter 2), so a pull request cannot merge until they pass. Each job is the same command a developer can run locally, which keeps "it passed on my machine" and "it passed in CI" the same statement.

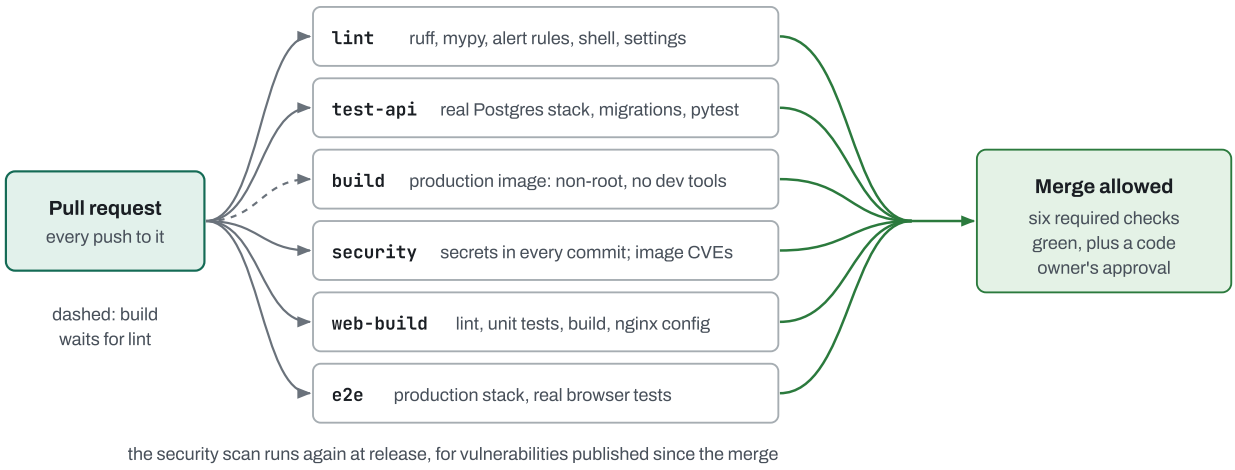


Figure 3.1 The merge gate of the reference service. A newer push to the same branch cancels the run still in progress, so the team pays only for checking the latest version of a change.

Two kinds of test sit in this gate, and they catch different bugs. A **unit test** checks one function in isolation, in milliseconds. An **integration test** runs real parts together, such as the api against a real Postgres, connection pooler and cache, and catches wiring bugs a unit test cannot see. The `test-api` job builds a throwaway stack for the integration tests and deletes it afterwards. The `e2e` job goes one step further and drives a real browser through the production-shaped stack, which once caught a bug no unit test could reproduce. Handbook 2 covers testing in depth.

IN THE REPO `.github/workflows/ci.yml`

The top of the CI workflow sets three rules that apply to every job.

- `concurrency` with `cancel-in-progress`: a newer push cancels the run of an older one on the same branch.
- `permissions: contents: read`: the workflow's built-in token can only read. A compromised step cannot push code or packages.
- Every third-party action is pinned to a full commit SHA, not a version tag, because a tag can be moved to malicious code. Chapter 4 explains why.

The `test-api` job runs the exact command a developer runs, so a local run and CI cannot disagree about what "tested" means.

```yaml
name: CI

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

# A newer push to the same branch cancels the run still in progress.
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

# Least privilege for the GITHUB_TOKEN: these jobs only read the repo.
permissions:
  contents: read

# Actions are pinned to full commit SHAs (tags can be moved to malicious
# code, as in the 2025 tj-actions/changed-files compromise). Dependabot
# keeps the SHAs and the version comments current.
```

```yaml
test-api:
  runs-on: ubuntu-24.04
  steps:
    - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
    # The exact command developers run: a throwaway compose stack with real
    # Postgres, PgBouncer and Valkey, migrations + `alembic check`, then
    # pytest with the coverage gate from pyproject.toml.
    - run: make .env
    - run: make test-api
```

Where CI runs, and what it costs

GitHub Actions runs each job on a fresh, GitHub-hosted virtual machine. Standard runners are free for public repositories, such as the reference one; a private repository gets a monthly quota of minutes and is billed by the minute beyond it, and larger runners are always billed. Three habits keep the bill and the wait down: cache dependency downloads keyed on the lockfile, run expensive workflows only when their inputs change (path filters), and cancel superseded runs. A **self-hosted runner** executes whatever a workflow tells it to. Never attach one to a public repository that accepts pull requests from outside: a stranger's pull request would run code on your machine.

Continuous delivery: a tag becomes a release

A release starts when someone pushes a version tag, such as `v0.8.2`, to a commit on main. Versions follow **semantic versioning**: `MAJOR.MINOR.PATCH`, where the major number changes for a breaking change, the minor for a compatible feature, and the patch for a fix. Teams choose between deploying every merge to main (continuous deployment) and deploying on a tag (continuous delivery). The reference service deploys on a tag.

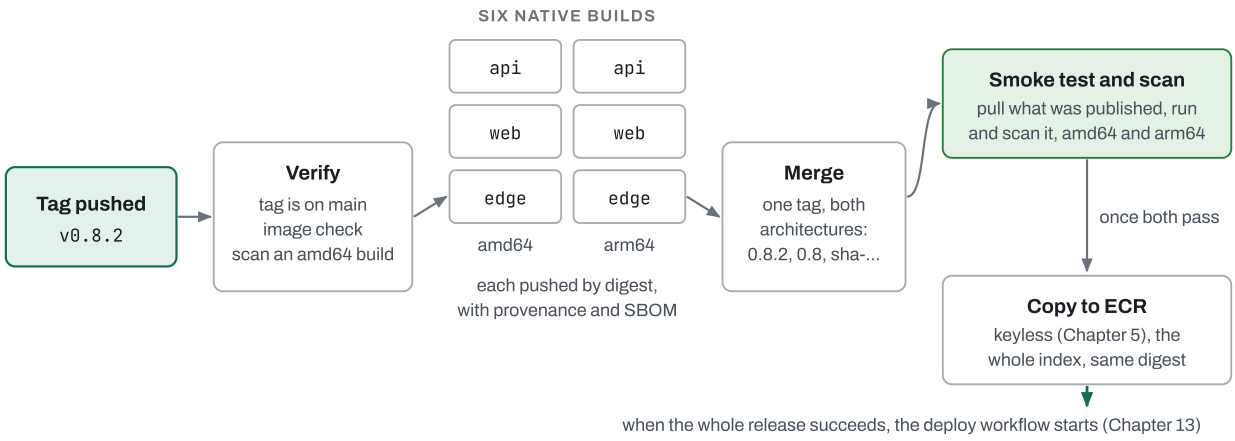


Figure 3.2 The release workflow. It builds each image once per processor type, joins them under the release tags, and then tests what it published, not what it built. The copy to ECR waits for the smoke test and the scan of what was published, because ECR tags are immutable: an image that failed either must never land there. The deploy starts only if the whole release succeeded.

Each step guards against a specific failure. Checking that the tag points at a commit on main stops anyone releasing code that never passed review. Scanning again at release catches vulnerabilities published after the merge. Testing the published images, rather than the build outputs, proves the registry holds what the team thinks it holds. Two gaps remain. The release-time scan and image check run on images the verify job builds for them on its amd64 machine, from the same commit: neither the arm64 images nor the exact published bytes are scanned. Scanning the pushed index by digest, once per architecture, would close that gap. And the copy to ECR does not wait for the smoke test, so a release that fails it still lands in ECR under a tag that can never be re-pushed. Only the deploy waits. Making the copy depend on the smoke test would close the gap.

IN THE REPO `.github/workflows/release.yml` the release's guards and its build outputs

Two parts of the release workflow carry most of its guarantees.

- `git merge-base --is-ancestor` fails unless the tagged commit is already on main.
- The image check and the vulnerability scan run again at release time, on an amd64 build of the tagged commit made for them. Then every published image is scanned again, for each platform: the arm64 images are built on their own machines, and were never scanned before. The copy to ECR waits for that scan and for the smoke test.
- A tag push publishes each build by digest; a pull request only builds, to a cache.
- `provenance` and `sbom` store, next to the image in the registry, which commit and workflow built it and what it contains.

```
57  # Tags can be pushed from any commit; only code that went through a
58  # reviewed, green PR into main gets released.
59  - name: The tag points at a commit on main
60    run: git merge-base --is-ancestor "$GITHUB_SHA" origin/main
61  - name: Production api image - non-root, no dev tools, every module imports
62    run: make image-check
63  # Again at release, not only at merge: vulnerabilities published since
64  # the PR merged block the release too (ADR-0022).
65  - name: No fixable HIGH or CRITICAL vulnerability in what ships (Trivy)
66    run: make scan
```

```
124   # A tag push publishes by digest; a PR only builds.
125   outputs: ${{ github.event_name == 'push' && format('type=image,name={0},push-by-digest=true,name-
126   canonical=true,push=true', env.IMAGE) || 'type=cacheonly' }}
127   # What went into the image, stored next to it in the registry:
128   # the build's provenance and a software bill of materials.
129   provenance: ${{ github.event_name == 'push' && 'mode=max' || 'false' }}
      sbom: ${{ github.event_name == 'push' }}
```

Tags, digests and multi-architecture images

An image is the immutable build output, identified by a **digest**: a hash of its content, such as `sha256:21115358…`. A **tag** is a movable label, like a Git branch. The release workflow moves `0.8` on every 0.8.x release on purpose; a re-push can move any tag unless the registry forbids it. So deploy an exact version from a registry with immutable tags, or deploy by digest, and never deploy `latest` or a moving tag such as `0.8`.

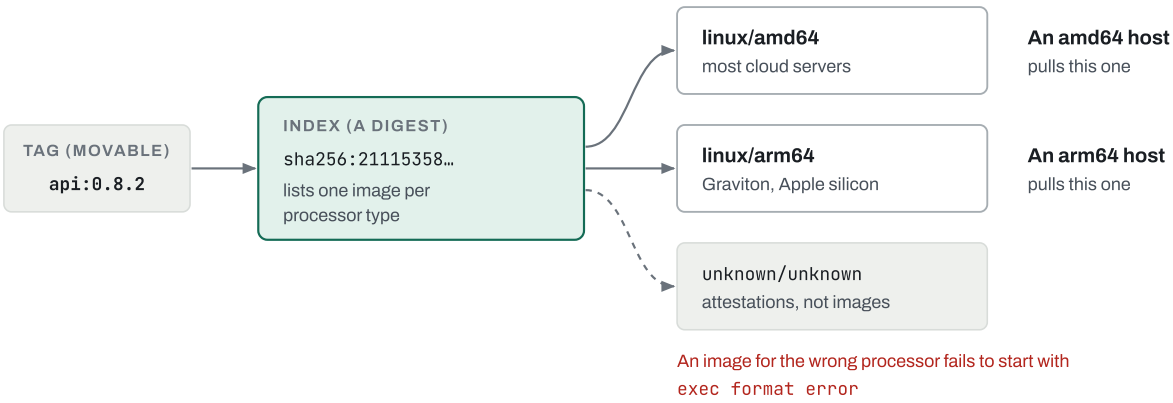


Figure 3.3 One version tag, several images. The client picks the image for its own processor. A plain pull followed by a push copies only that one image, which is why copying a release to another registry uses a tool that copies the whole index.

The reference release builds each architecture on a native runner (an Arm machine for arm64) rather than emulating one on the other. Emulated builds are slow and occasionally behave differently. Each build pushes its image by digest, and a final job joins the digests under the version tags.

When the release was copied from GitHub's registry to Amazon ECR, the index digest in ECR was identical to the one on GitHub: the same bytes in a second place. Pushing a different image over an existing tag in ECR was then refused, because the repositories have immutable tags. A tag in that registry always names the same bytes.

> **HOW TO SAY IT**
>
> *"CI is our merge gate: the same commands developers run locally, on a clean machine, and the pull request can't merge until they pass."*
>
> *"A release is a tag. It builds once for both processor types, tests what it published, and we deploy that exact version everywhere."*

Traps

SYMPTOM	CAUSE	FIX
A copied release runs on one processor type and fails on the other	Pull then push copies only the image for the machine doing the copy	Copy the whole index with `docker buildx imagetools create` or `skopeo copy --all`.
`exec format error` **when a container starts**	The image was built for another architecture, often arm64 from a laptop	Build in CI for both, or pass `--platform linux/amd64` when building locally.

SYMPTOM	CAUSE	FIX
The registry's scan never ran on a copied image	Scan-on-push fires for ordinary pushes; a copy can take another path	Scan in the pipeline, before publishing, and do not rely on the registry alone.
An old vulnerability blocks an unrelated pull request	A new advisory was published for the base image; main already has the fix	Read the failing step before changing anything; update the branch from main.
A release has to be cut again just to retry one failed job	Treating a failed run as a failed release	Re-run only the failed jobs of the same run.
A timing-sensitive test fails once on main and passes on retry	The test depends on where a time window boundary falls	Treat it as a real flake: make time controllable in the test, or assert on behavior that does not depend on the clock.
A self-hosted runner ran an outsider's code	The runner was attached to a public repository that accepts external pull requests	Use hosted runners for public repositories; isolate self-hosted ones.

Questions

1. What is the difference between a unit test and an integration test, and why does CI run both?
2. Why does the release workflow test the images it published rather than the ones it built?
3. What is the difference between a tag and a digest, and which should a deploy use?
4. Why might a team build arm64 images at all?
5. What does build provenance answer that a version number does not?

Answers

1. A unit test checks one function in isolation and is fast. An integration test runs real components together and catches wiring bugs, such as a wrong connection setting or a missing migration. Each catches failures the other cannot.
2. Because what matters is what hosts will pull. Testing the published images, on each architecture, proves the registry holds working images under the release tags.
3. A tag is a movable label; a digest is a hash of the content and never changes. Deploy by digest, or by an exact version tag in a registry that makes tags immutable. Never by `latest` or a tag that is moved on purpose, such as a minor line.
4. Arm servers such as AWS Graviton cost less for the same work, and many developer laptops are Arm. One tag with both images lets every host pull the right one.

5. Which commit and which workflow run produced this exact image. A version number is only a label someone chose; provenance is a record the build wrote and stored with the image. It proves the answer only when it is signed and the signature is checked; the reference release's is not signed.

PART II · SOURCE AND BUILD

4 The software supply chain

IN ONE SENTENCE

Most of what a service runs was written by someone else, so teams now pin every dependency exactly, scan what they ship on every change, take updates through reviewed pull requests, and record each accepted risk with a reason, an owner and an expiry.

AT A GLANCE

- The supply chain is everything you ship or run but did not write: packages, base images, CI actions, build tools, and the scanners themselves.
- Pin by content, not by name: lockfiles with hashes, base images by digest, CI actions by commit SHA. Names can be moved; content hashes cannot.
- Updates arrive as pull requests from a bot and pass the same checks as any change.
- Scan for known vulnerabilities and leaked secrets on every pull request, again at release, and weekly.
- Block what has a fix. When the fix cannot be taken yet, accept the risk explicitly, with an expiry that forces the question again. Findings with no fix at all cannot block anything, so list them in a report-only scan and review them.

THEN AND NOW

THEN	NOW	WHY IT CHANGED
Install the newest version of each library at build time	→ **Lockfiles with hashes, installed in frozen mode**	A build should install exactly what was reviewed, every time.
A base image named by tag, such as `python:3.13-slim`	→ **The tag plus its digest, moved together by a reviewed update**	A tag can point at different content tomorrow; a digest cannot.
CI plugins and actions used by version tag	→ **Actions pinned to a full commit SHA**	Attackers have moved popular actions' tags to code that stole CI credentials.
A yearly security review or penetration test	→ **Automated scans on every pull request, at release and weekly**	New vulnerabilities are published every day, in images nobody changed.

THEN	NOW	WHY IT CHANGED
Exceptions agreed in an email	→ A file of accepted risks, each with a reason, an approver and an expiry, reviewed by code owners	An exception nobody revisits becomes permanent.
Leaked secrets found in an audit months later	→ Every commit scanned for secrets; pushes containing them blocked	Once pushed to a public repository, a secret is copied within minutes.

What you ship but did not write

A small service's own code is a thin layer on top of a much larger stack. Each layer has its own way of changing under you, and its own control.

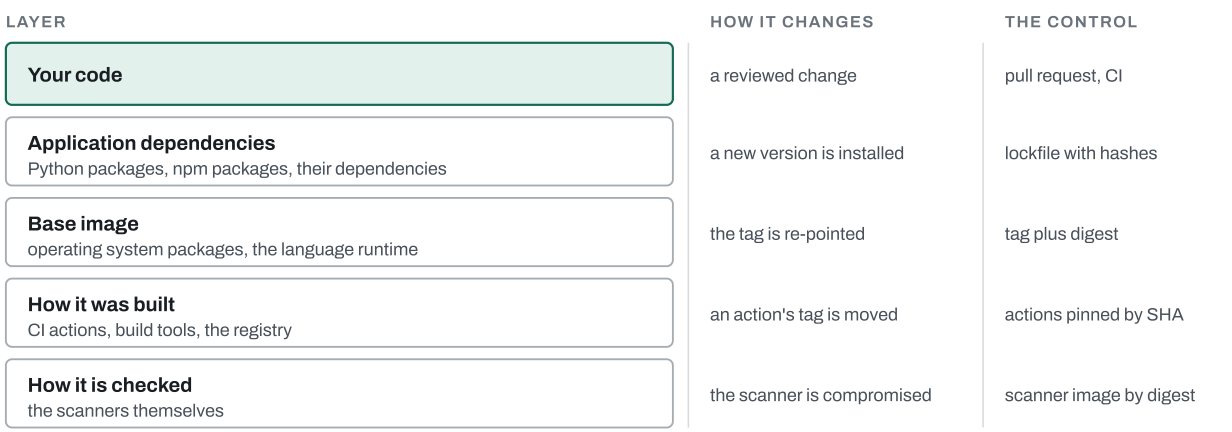


Figure 4.1 The layers of a container image, from the code you wrote down to the tools that check it. Every control on the right does the same thing: it names content by its hash rather than by a name someone else can move.

The bottom layer is easy to forget. A scanner runs with access to your CI environment, so a compromised scanner is a compromised pipeline. This has happened: attackers published a Trivy release, binaries and container image alike, and moved almost every tag of its GitHub Action, all to code that stole CI credentials (CVE-2026-33634). Teams that ran a scanner image pinned to a digest from before the attack, and did not take the new release, were not affected. A digest pin stops a tag from being re-pointed; a reviewed bump is the chance to catch a bad new release. The reference service runs its scanners that way, from the same commands locally and in CI, with no access to the Docker socket.

Pinning, and taking updates

Pinning alone would freeze a service with every vulnerability it ever had. The other half is a bot, Dependabot here, that opens pull requests to move the pins: grouped monthly for packages and actions,

weekly for base images, and immediately for a known vulnerability. Each is an ordinary pull request, so CI and review decide whether it merges. A major version gets read like any other change.

IN THE REPO `apps/api/Dockerfile` and `.github/dependabot.yml`

The base images are named by tag for people and by digest for machines. Dependabot moves both together in one reviewable pull request.

- Even the build tool, `uv`, comes from an image pinned by digest.
- The Dependabot file explains its own schedule: grouped monthly updates, but base images weekly, because operating-system fixes arrive only through a rebuilt base.
- Actions stay pinned to SHAs; the bot moves the SHA and its version comment together.

```
FROM ghcr.io/astral-
sh/uv:0.12.21@sha256:a7aed3216253ee804de3e2d8afa5073baa1a177335345d43845cd4165e43b711 AS uv

# ---- deps: virtualenv with runtime dependencies only ----------------------
FROM python:3.13-slim@sha256:7c61056e61ac89e852de05f3dc6fa51a6dd2181797bceed46aa725dd7cb2cd3b AS deps
```

```
# Dependency update PRs, monthly and grouped (one PR per ecosystem, not one
# per package). Monthly, not weekly: a vulnerable dependency still gets its
# own security PR at once, and everything else can wait for the batch. The
# Dockerfiles' base images are the exception, weekly: security updates do
# not cover OS packages, and a rebuilt base is how a .trivyignore.yaml
# entry gets removed before it expires (ADR-0026).
# Each is an ordinary PR: CI must pass before merging, and a
# major version gets read like any other change. GitHub Actions stay pinned
# to commit SHAs; Dependabot moves the SHA and its version comment together.
```

Scanning, and what to do with a finding

Two kinds of scan run in the reference pipeline. **Vulnerability scanning** (Trivy) compares the packages inside each production image, and the web app's runtime dependencies, with databases of known vulnerabilities. **Secret scanning** (gitleaks) reads every commit in the history for things that look like keys and passwords. Both run on every pull request; the vulnerability scan runs again at release and weekly, because new advisories are published for images nobody changed. Each of those runs builds the images itself on an amd64 machine, so it checks the same source, not the exact published bytes, and not the arm64 images.

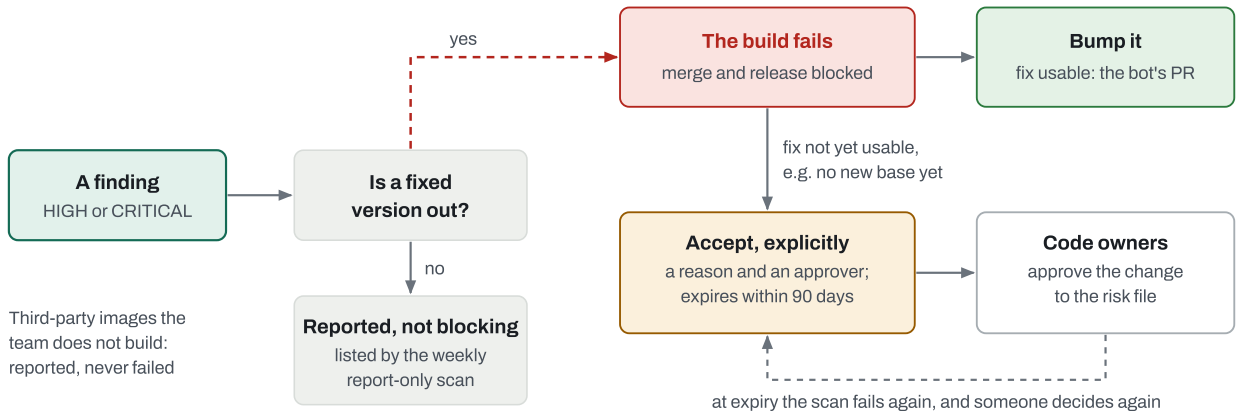


Figure 4.2 The policy for a vulnerability. Blocking only on findings that can be fixed keeps the check meaningful: a check that is red for reasons nobody can act on teaches people to ignore red. The blocking scan runs with `--ignore-unfixed`, so a finding with no fix anywhere never shows there; the weekly run adds a report-only scan without that flag, which lists them (44 in the api image when it was added), so they are seen and revisited.

The note at the bottom left of the figure is a lesson learned. The weekly scan of the third-party images the stack merely runs, such as Keycloak, Grafana and pgvector, first found 336 findings in 12 of 14 images, nearly all waiting on releases from those projects. A job that fails every Monday for things the team cannot change is an email nobody reads, so that scan now reports instead of failing, and is read when those images are upgraded. The service's own images still fail on any fixable finding.

The first scan also changed the images themselves. Two vulnerabilities lived inside `pip`, which the production image never used, so pip was removed and the image check now refuses an image that contains it. Another lived in nginx's optional modules, so the image moved to nginx's slim variant. Removing what you do not need is often a better fix than patching it.

IN THE REPO `.trivyignore.yaml` and `scripts/check_trivyignore.py`

The accepted-risks file states its own rules at the top, and a small script enforces them, because the scanner honors a reason and an expiry but requires neither.

- An entry must say why the risk is safe for now and who accepted it.
- It must expire within 90 days, after which the scanner reports the finding again.
- Without the script, a risk accepted once would stay accepted forever, for no stated reason.

```
# Accepted vulnerabilities (make scan, make scan-compose; ADR-0022). A
# finding goes here only when it cannot be fixed now: no fixed version is
# used by the image yet, or the vulnerable code is shown to be unreachable.
# Each entry needs:
#   statement:  why it is safe for now, and who accepted it
#   expired_at: at most 90 days out; Trivy reports it again after that
# scripts/check_trivyignore.py refuses an entry without both, and the code
# owners must approve every change to this file (.github/CODEOWNERS).
```

```
#!/usr/bin/env python3
"""Every accepted vulnerability in .trivyignore.yaml has a reason and an
expiry date no more than 90 days out.

Trivy honours `statement` and `expired_at` but requires neither: without
this check a risk accepted once stays accepted forever, for no stated
reason. Run by `make scan` and `make scan-compose`.
"""
```

ADR-0022 ACCEPTED

Scan images and commits; pin base images and scanners by digest

PROBLEM	Nothing said whether what shipped had a known vulnerability, or whether a commit carried a secret. The first scan found 20 fixable HIGH vulnerabilities, none in the team's own code.
DECISION	Scan on every pull request, before release and weekly; block HIGH and CRITICAL findings that have a fix; when the fix cannot be taken yet, accept the risk with a reason and an expiry; pin base images and scanners by digest.
LATER	ADR-0026 made the third-party image scan report instead of fail, and moved base-image updates to weekly.
SEE IT IN	scan.yml, .trivyignore.yaml, and the `security` job in ci.yml.

A secret found in a commit is handled in a fixed order: rotate it first, because once pushed it may already be copied, and only then remove it. A fake credential in a test is allowed only by its exact fingerprint, with the value shown to be fake. GitHub's push protection adds a second layer by refusing the push itself.

> **HOW TO SAY IT**
>
> *"Everything we depend on is pinned by content hash, and updates arrive as reviewed pull requests from a bot."*
>
> *"We block fixable high and critical findings on every pull request and at release. When a fix exists but we can't take it yet, the risk is accepted in writing, with an owner and an expiry. Findings with no fix anywhere don't fail the build."*

Traps

SYMPTOM	CAUSE	FIX
A trusted CI action starts exfiltrating secrets	It was referenced by tag, and the tag was moved to malicious code	Pin every third-party action to a full commit SHA.
The scanner itself is the attack	It ran from a third-party action or a mutable download	Run scanners from images pinned by digest, with no Docker socket.
Builds differ depending on the day they ran	`apt-get upgrade` or unpinned installs inside the Dockerfile	Pin the base by digest and take fixes as a reviewed digest bump.
Everyone ignores the red security check	It fails on findings nobody can fix	Block only findings with a fix; accept a fix you cannot take yet, with an expiry; report third-party images.
A pull request goes red for a vulnerability it did not introduce	A new advisory for the base image	Take the bot's digest bump, or accept the risk with an expiry.
The secret-scanning action fails for an organization	The gitleaks GitHub Action needs a license for organizations	Run its command-line tool instead.
A leaked key was deleted from the repository, but it is still abused	Deleting it does not unpublish it	Rotate first, then remove.
An agent added a package that does not exist, or a look-alike	Models invent package names between one in twenty and one in five times	Installs only from a lockfile the pipeline checks; a registry allowlist where one exists.
A pinned scanner action ran attacker code	Tags were moved after a maintainer account was compromised; a tag is not a pin	Pin every action to a full commit SHA and let the update bot move the SHA.

GOING DEEPER Provenance, bills of materials, and checking a new dependency

Provenance is a record of how an artifact was built: which source commit, which workflow, which builder. A **software bill of materials (SBOM)** lists the packages inside it. The reference release stores both next to each image in the registry, so a question such as "which of our images contain this library version?" can be answered from the registry, without rebuilding anything. The reference release's records are BuildKit attestations, and they are not signed: anyone who can push the package could push different ones, so they describe a build without proving it. To make provenance verifiable, sign it, for example with `actions/attest-build-provenance` (checked with `gh attestation verify`) or Sigstore's `cosign`, and verify the signature before deploying. Frameworks such as SLSA grade how strongly a pipeline can prove these claims.

Before adding a dependency at all, the reference team checks five things: its license (Redis and Docker Desktop both changed theirs, and Terraform moved to the Business Source License), that it is maintained and has a reasonable security record, that it publishes builds for both processor types, its size, and that it can be pinned and covered by the scans and the update bot. The pull request then says, in one line, why it is needed.

Questions

1. Why pin a base image by digest when the tag already names a version?
2. Why does the policy block only findings that have a fixed version?
3. Why scan again at release and weekly if every pull request is scanned?
4. A real API key is found in a commit. What happens first, and why?

Answers

1. A tag can be re-pointed to different content, including a rebuilt image with other packages. A digest is a hash of the content, so it always means the same bytes. Updates then arrive as a reviewed change to the digest.
2. A finding with no fix leaves nothing to do but wait. Blocking on it keeps the check permanently red, and people learn to ignore it. With `--ignore-unfixed` the blocking scan does not even report them, so the reference service's weekly run adds a report-only scan without that flag, which keeps them in sight. The accepted-risks file, with a reason and an expiry that forces a fresh decision, is for the other case: a fix exists upstream but cannot be taken yet.
3. Vulnerabilities are published every day for packages that have not changed. An image that was clean when its pull request merged can have a known critical vulnerability a week later.
4. Rotate the key, so the leaked value stops working. Removing it from history comes second: on a public repository it may already have been copied.

PART III · IDENTITY

5 Identity without keys

IN ONE SENTENCE

People, pipelines and running services each prove who they are to a trusted identity provider and receive credentials that expire on their own, so there is no permanent key anywhere to leak.

AT A GLANCE

- Three kinds of caller need cloud access: **people**, **CI pipelines** and **workloads**. Each now gets temporary credentials, by a different route.
- People sign in once a day through single sign-on with MFA. Their sessions end by themselves.
- Pipelines use **OIDC federation**: the CI platform signs a token that says which job is running, and the cloud exchanges it for credentials that last an hour.
- Workloads use a role attached to the compute. The platform hands out and rotates the credentials.
- The **trust policy** is the security boundary. It is an exact match on claims, so write it from the token you actually receive, not from an example.

THEN AND NOW

THEN	NOW	WHY IT CHANGED
An IAM user per developer, with access keys in `~/.aws/credentials`	→ **Single sign-on behind the company identity provider, with role sessions that expire**	A key on a laptop leaks with the laptop. A session dies on its own.
Access keys stored as CI secrets	→ **OIDC federation: the job's signed token is exchanged for one-hour credentials**	Stored CI keys are the classic leak, and someone has to rotate them.
Keys in a config file or baked into a server image	→ **A role attached to the VM or container; credentials served by the platform and rotated**	A server that holds no secret cannot leak one.
SSH keys and port 22 for operators	→ **Session Manager and Run Command, authorized by IAM and recorded**	No open port, and every session or command sent is attributed to a person; session logging keeps what was typed.

THEN	NOW	WHY IT CHANGED
A shared admin account for deploys	→ **Pipelines deploy; people are read-only in production and request elevated access when needed**	Every production change goes through a reviewed, recorded path.

Three callers, three routes

The old model gave every caller a long-lived secret: a password, an access key, an SSH key. Each one had to be stored somewhere, rotated by someone, and revoked when a person left. The modern model replaces all of them with the same pattern. A caller proves who it is to a system that already knows it, and receives credentials with a short, fixed lifetime.

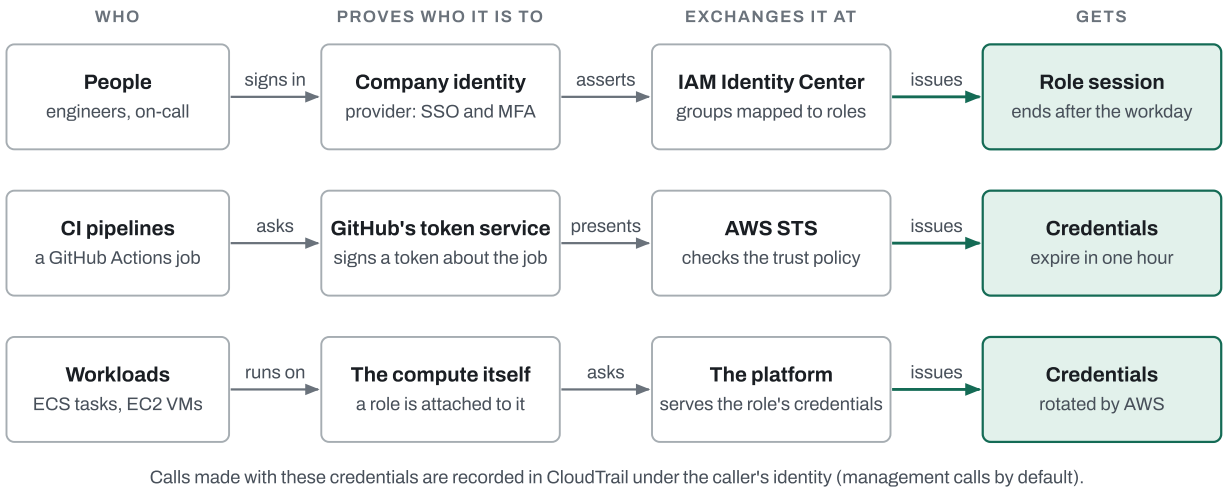


Figure 5.1 Three kinds of caller, three routes to the same result: credentials that expire on their own. No route stores a permanent key.

The rest of this chapter takes each lane in turn. Pipelines get the most space, because that is where the change is newest and where the configuration is easiest to get subtly wrong.

People: single sign-on, and no standing access to production

An engineer signs in once a day through the company's identity provider (Okta, Microsoft Entra ID and others), with MFA. AWS IAM Identity Center maps the groups that provider asserts to roles in each account: development, staging, production. The command-line tools then fetch short-lived role credentials as needed. When the sign-in session ends, usually after a working day, everything stops working until the person signs in again. A stolen laptop's credentials die on their own.

Production adds one more rule: people do not deploy by hand. The pipeline deploys through its own role. People are read-only by default and get elevated access only on request, for a fixed time, and every use of it is recorded. This is often called break-glass or just-in-time access.

The reference build used a single throwaway account, so it signed in with the simpler `aws login`. The idea is the same: a browser sign-in that yields temporary credentials, and no access keys on disk. Root user access keys were never created.

Pipelines: OIDC federation

A CI job is the caller with the most power and the least supervision: it runs automatically, on shared infrastructure, and can push images or change infrastructure. It also cannot type a password. The old answer was an access key stored in the CI system's secrets. The modern answer is **OpenID Connect (OIDC) federation**: the CI platform vouches for the job, and the cloud decides whether to trust that.

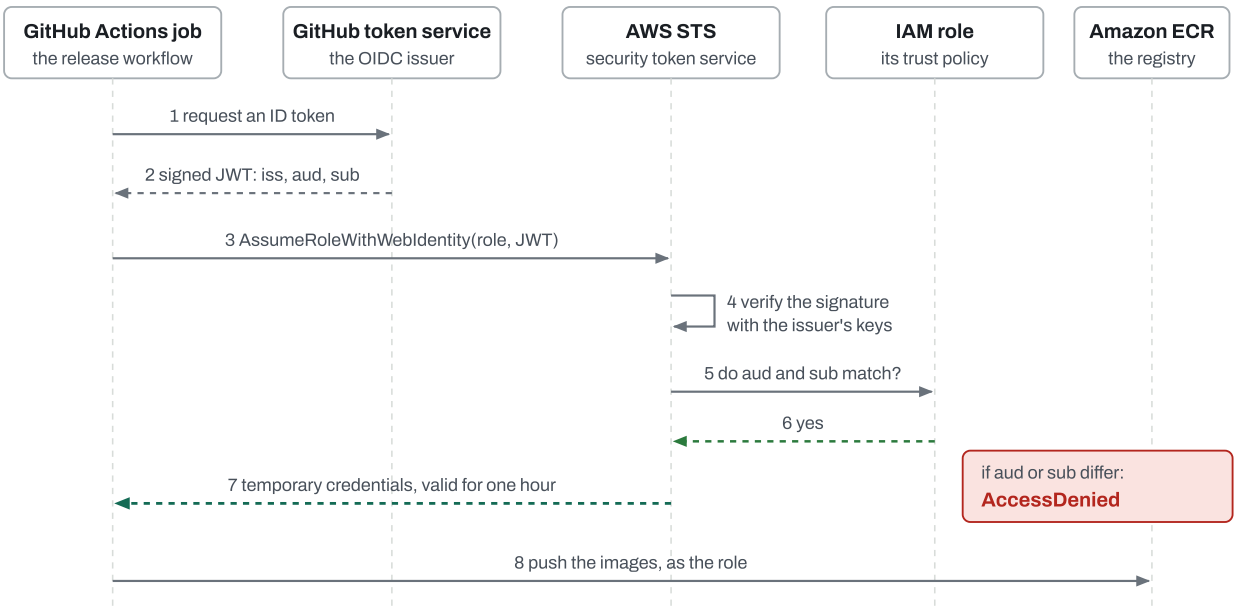


Figure 5.2 OIDC federation, step by step. GitHub vouches for the job, AWS checks that claim against the role's trust policy, and the job gets credentials that last an hour. Nothing secret is stored in the repository or in CI.

Four pieces make it work:

1. **The job asks for a token.** GitHub signs a JSON Web Token (JWT) for each job that has `permissions: id-token: write`. Without that line, no token is issued.
2. **The token describes the job.** Its `iss` (issuer) is GitHub's token service. Its `aud` (audience) is whatever the job asks for: the AWS login action, `aws-actions/configure-aws-credentials`, asks for `sts.amazonaws.com`, while a token fetched without an audience carries the repository owner's URL and fails the trust policy. Its `sub` (subject) names the repository and either the git ref that triggered the job or the GitHub environment the job runs in.
3. **AWS trusts the issuer once per account.** An IAM OIDC provider entry tells AWS to accept tokens signed by GitHub's published keys.
4. **A role decides which jobs it will serve.** Its trust policy allows `sts:AssumeRoleWithWebIdentity` only when `aud` and `sub` match. A different repository, branch, tag or environment is refused.

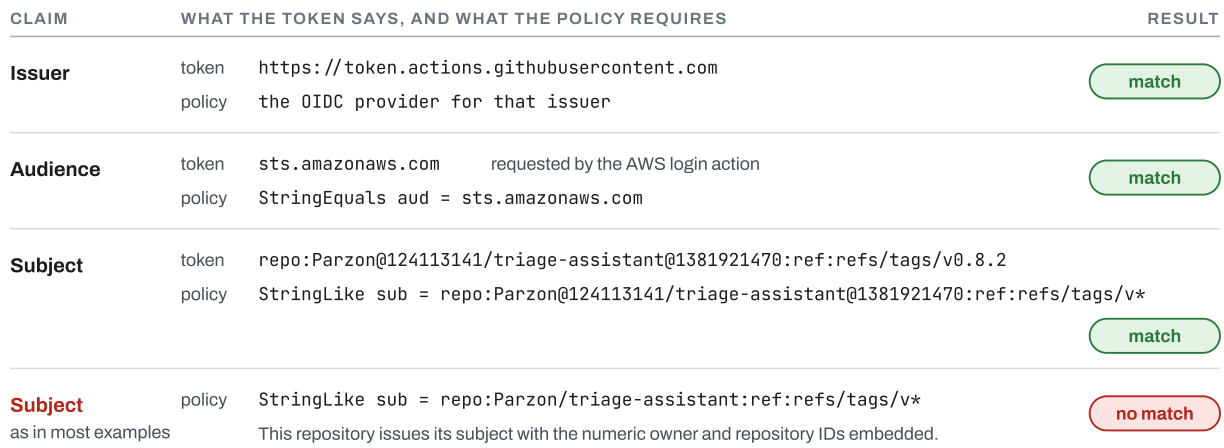


Figure 5.3 The trust policy is a string match against the token's claims. The last row is the policy most examples would lead you to write. It never matches this repository's tokens, and nothing in the error says which condition failed.

IN THE REPO `infra/aws/cicd.py` the OIDC provider and the image-push role

This CDK stack (Python that becomes CloudFormation) creates the account's one OIDC provider for GitHub and the role the release workflow uses to copy images into Amazon ECR.

- `SUB_PREFIX` is the subject read from the repository's own OIDC setting, not typed from an example. The comment says how to read it.
- The audience must be exactly `sts.amazonaws.com`.
- The subject must be a version tag of this repository. A pull request, a branch push or a fork produces a different subject and gets nothing.
- A version tag is only as trustworthy as the right to create one. A tag runs the release workflow of the commit it points at, so a tag on another branch could drop the workflow's "tag is on main" check. A tag ruleset lets only admins create, move or delete `v*` tags, and the comment says that this trust depends on it.
- Sessions last at most one hour.
- Push and pull are granted on the three image repositories only. `ecr:GetAuthorizationToken` cannot be scoped to a repository, so it is the one permission on `*`, and the code says why.

```
# This repo issues GitHub's "immutable subject" claim, which embeds the numeric owner and repo IDs
# (survives renames, blocks a re-created repo of the same name). Read it with:
#   gh api repos/Parzon/triage-assistant/actions/oidc/customization/sub
# A copy of this template has other ids: put its own prefix here (the same command, on its repo),
# or no role can be assumed. scripts/new-project.sh renames the repo, not these numbers.
SUB_PREFIX = "repo:Parzon@124113141/triage-assistant@1381921470"
ISSUER = "token.actions.githubusercontent.com"
```

```
# AWS trusts GitHub's signed tokens (one provider per account).
provider = iam.OpenIdConnectProvider(
    self, "GitHub", url=f"https://{ISSUER}", client_ids=["sts.amazonaws.com"]
)
role = iam.Role(
    self,
    "EcrPush",
    role_name="triage-gha-ecr-push",
    max_session_duration=Duration.hours(1),
    assumed_by=iam.WebIdentityPrincipal(
        provider.open_id_connect_provider_arn,
        conditions={
            "StringEquals": {f"{ISSUER}:aud": "sts.amazonaws.com"},
            # Only a release tag of this repo: not a PR, not a branch, not a fork. A tag
            # pushed on any branch runs that branch's release.yml, which could drop its
            # "tag is on main" check: the repository's tag ruleset, which lets only admins
            # create v* tags, is what makes this a release (using-this-template.md).
            "StringLike": {f"{ISSUER}:sub": f"{SUB_PREFIX}:ref:refs/tags/v*"},
        },
    ),
)
for repo in repos.values():
    repo.grant_pull_push(role)  # scoped to these repos, not ecr:*
# GetAuthorizationToken cannot be scoped to a repository.
role.add_to_policy(
    iam.PolicyStatement(actions=["ecr:GetAuthorizationToken"], resources=["*"])
)
```

One role per job, gated where the risk is

The reference build has two pipelines that touch AWS, and each has its own role. Copying images is low risk and happens on every release, so its role trusts any version tag. Deploying changes production, so its role trusts only jobs that run inside a **GitHub environment** named `aws-demo`.

That choice moves the approval rules out of the trust policy and into GitHub. When a job names an environment, its token's subject ends in `:environment:aws-demo` instead of the git ref. The environment's own protection rules then decide which jobs may run in it: here, only those from protected branches, and only after a required reviewer approves. That reviewer is what turns continuous deployment into deploy-on-approval, and adding or removing one never touches AWS.

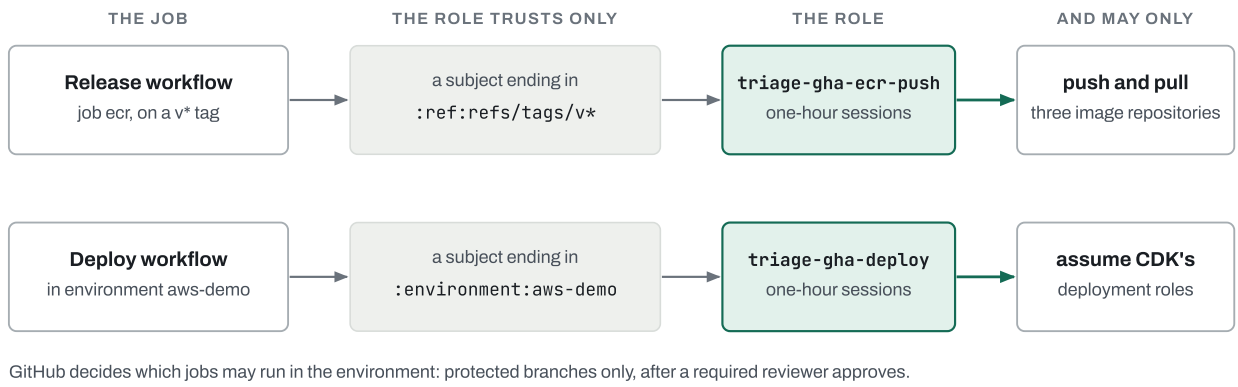


Figure 5.4 Two jobs, two roles. The deploy role trusts an environment rather than a ref, so GitHub's environment rules become the gate in front of production.

IN THE REPO `.github/workflows/deploy.yml` and the deploy role in `infra/aws/cicd.py`

The deploy job names the environment and asks for a token. Then one action exchanges the token for credentials. The role ARN is a repository secret. It grants nothing on its own, but it names the AWS account, and GitHub masks only secrets in public logs: a variable is printed with every step's inputs. A job's `if` cannot read secrets, so a first job reads it and answers yes or no. While it is unset, nothing deploys, so a copy of the template works without an AWS account.

- `environment: name: aws-demo` is what changes the token's subject.
- `id-token: write` is granted to this job only, not to the whole workflow.
- `mask-aws-account-id` hides the account id where the action and CDK print it.
- The deploy role may assume only the roles that `cdk bootstrap` created. CloudFormation does the actual work with its own execution role.
- Here the subject is matched with `StringEquals`, because there is no wildcard to allow.

```yaml
jobs:
  # A job's `if` cannot read secrets: this one reads the secret and answers yes or no.
  enabled:
    if: >-
      github.event_name == 'workflow_dispatch' ||
      (github.event.workflow_run.conclusion == 'success' && github.event.workflow_run.event == 'push')
    runs-on: ubuntu-24.04
    outputs:
      deploy: ${{ steps.check.outputs.deploy }}
    steps:
      - id: check
        env:
          CONFIGURED: ${{ secrets.AWS_DEPLOY_ROLE_ARN != '' }}
        run: echo "deploy=$CONFIGURED" >> "$GITHUB_OUTPUT"

  deploy:
    needs: enabled
    if: needs.enabled.outputs.deploy == 'true'
    runs-on: ubuntu-24.04
    environment:
      name: aws-demo
      url: ${{ steps.smoke.outputs.url }}
    permissions:
      contents: read
      id-token: write
```

```yaml
- uses: aws-actions/configure-aws-credentials@e1253824e5c10ff9df46874f81ed3ec929e19cfd # v6.3.0
  with:
    role-to-assume: ${{ secrets.AWS_DEPLOY_ROLE_ARN }}
    aws-region: ap-south-1
    # The account id also appears in what follows (CDK's output, resource ARNs).
    mask-aws-account-id: true
```

```python
deploy = iam.Role(
    self,
    "Deploy",
    role_name="triage-gha-deploy",
    max_session_duration=Duration.hours(1),
    assumed_by=iam.WebIdentityPrincipal(
        provider.open_id_connect_provider_arn,
        conditions={
            "StringEquals": {
                f"{ISSUER}:aud": "sts.amazonaws.com",
                # A job that names an environment gets this subject instead of its ref.
                f"{ISSUER}:sub": f"{SUB_PREFIX}:environment:aws-demo",
            },
        },
    ),
)
deploy.add_to_policy(
    iam.PolicyStatement(
        actions=["sts:AssumeRole", "sts:TagSession"],
        resources=[
            f"arn:aws:iam::{self.account}:role/cdk-hnb659fds-*-{self.account}-{self.region}"
        ],
    )
)
```

Workloads: roles attached to the compute

A running service should never hold a key either. On AWS, a role is attached to the compute itself: a **task role** for an ECS task, an **instance profile** for an EC2 VM. The platform serves that role's credentials to the code at runtime and rotates them. Your code, using the standard AWS SDK, picks them up without any configuration.

ECS has two roles per task, and mixing them up is a common mistake. The **task execution role** is used by ECS itself, before your code runs, to pull the image and inject secrets. The **task role** is what your code may call once it is running, such as S3 or a queue. A service that needs no AWS API at runtime needs no task role at all.

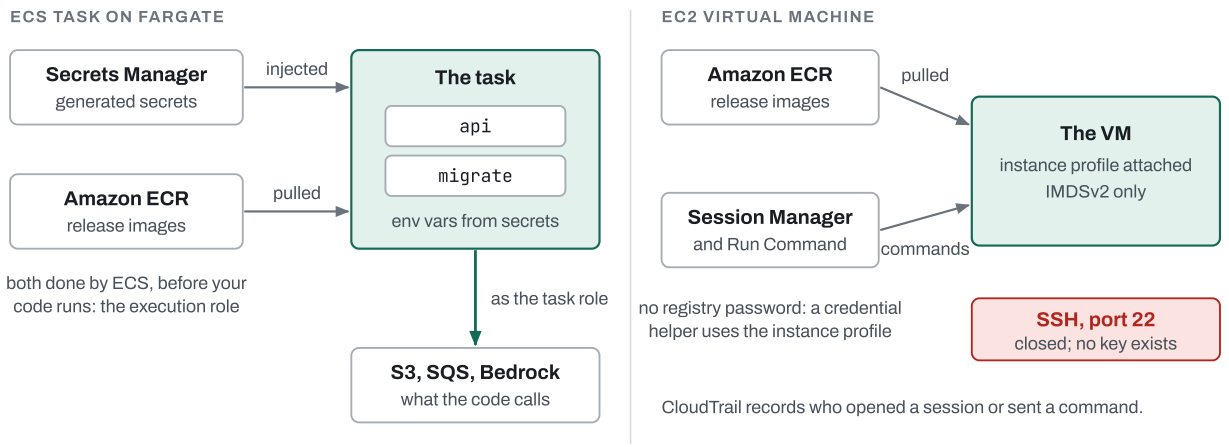


Figure 5.5 Workload identity on the two AWS compute types used in the reference build. Neither holds a key: the role is attached to the compute, and the platform supplies the credentials. CloudTrail records the session and command API calls, not what is typed inside a session; the VM stack's session logging keeps that, and Run Command output, in CloudWatch Logs (Chapter 6).

Application secrets that are not AWS credentials, such as a database password or a model provider's API key, follow the same principle. They are generated by Secrets Manager, read by ECS when the task starts, and handed to the container as environment variables. They never appear in the image, the task definition, the CloudFormation template or the repository.

IN THE REPO `infra/aws/service.py` and `infra/aws/vm.py`

Two helpers in the ECS stack: one asks Secrets Manager to generate a secret, the other tells ECS to inject a field of it into a container at start. The VM stack's docstring lists what makes a single VM acceptable to a platform team.

- Secrets are generated by AWS. Nobody types or sees them.
- The comment on `exclude_punctuation` records a real constraint: the api builds a database URL from the password.
- In this throwaway account a secret is deleted with the stack; the `production:` comment says to keep it (`RETAIN`), since the database it opens may outlive the stack.
- The VM has no SSH key and no port 22. Images come from ECR through the instance role, so no registry password exists anywhere.
- CloudTrail records who opened a session or sent a command, to which instance, and when, as the docstring says. It does not record the commands typed inside a session, and it hides a Run Command's parameters. Session logging keeps those: the VM stack streams every session, and the operator script's Run Command output, to a log group (Chapter 6).

```python
def generated_secret(scope: Construct, cid: str, username: str | None = None) -> sm.Secret:
    # No punctuation: the api builds a URL from these, and @ : / % would need encoding.
    return sm.Secret(
        scope,
        cid,
        generate_secret_string=sm.SecretStringGenerator(
            secret_string_template=json.dumps({"username": username} if username else {}),
            generate_string_key="password",
            exclude_punctuation=True,
            password_length=32,
        ),
        # Throwaway account: deleted with the stack. production: RETAIN (a destroyed secret
        # cannot be read back, and the database it opens may outlive the stack).
        removal_policy=RemovalPolicy.DESTROY,
    )


def secret(s: sm.ISecret, field: str) -> ecs.Secret:
    # Injected by ECS at start from Secrets Manager: never in the task definition or the template.
    return ecs.Secret.from_secrets_manager(s, field)
```

```
"""Stage 3: the one-VM deployment, done the way a platform team would accept it.

Compared with a hand-built VM:
- no SSH key and no port 22: operators use SSM Session Manager / Run Command, which the
  instance role allows; CloudTrail records who started a session or sent a command, and
  when, and the log group /triage/vm-operations keeps what ran: every session's input and
  output, and Run Command output (ops.sh vm-run);
- images come from ECR, authenticated by the instance role through the ECR credential helper,
  so no registry password exists anywhere;
- IMDSv2 only, encrypted disk, a security group that admits HTTP and nothing else;
- the host is built by cloud-init from code (adapted from infra/vm/cloud-init.yaml in the repo),
  so a replacement VM is one `cdk deploy` away.
```

A case from the reference build: the test that proved nothing

When the image-push role was first set up, the plan was to prove the trust policy worked by breaking it on purpose. The policy was edited to name the wrong repository, a release tag was pushed, and the job failed with `Not authorized to perform sts:AssumeRoleWithWebIdentity`, as expected. Then the correct repository name was restored and only the failed jobs were re-run. They failed again.

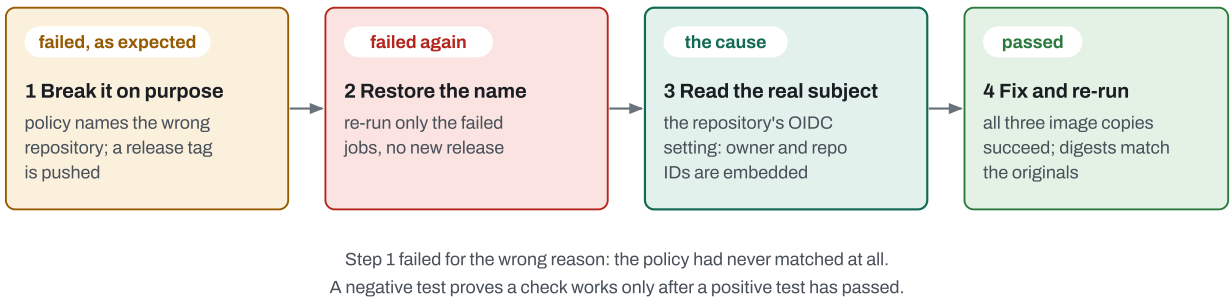


Figure 5.6 The deliberate break looked like proof that the policy worked. It failed only because the policy had been wrong from the start.

The policy had been written in the subject format most documentation shows, `repo:owner/name:…`. This repository's tokens use a form that embeds the numeric owner and repository IDs, which survives renames and stops a deleted-and-recreated repository with the same name from inheriting the trust. Reading the repository's OIDC subject setting showed the real value. With the prefix fixed, all three image copies succeeded, and the digests in ECR matched the ones published to GitHub's registry: the same bytes in a second place.

Two habits come out of this. Check the real claims first, before blaming anything else, because the error never says which condition failed. And re-run failed jobs rather than cutting a new release: it is faster, and it tests exactly the same thing.

> **HOW TO SAY IT**
>
> *"Our pipelines don't hold cloud keys. Each job gets a token from GitHub, AWS trades it for one-hour credentials, and the role only trusts a release tag or the production environment."*
>
> *"Workloads use the role attached to their compute, and people use SSO. If a laptop or a CI log leaks, there is no long-lived credential in it."*

Defaults, and when to choose otherwise

CALLER	DEFAULT TODAY	AVOID	CHOOSE OTHERWISE WHEN
People	SSO with MFA through the company identity provider; role sessions per account	IAM users with access keys	A break-glass account for when SSO itself is down: kept offline, alarmed on use

CALLER	DEFAULT TODAY	AVOID	CHOOSE OTHERWISE WHEN
CI pipelines	OIDC federation, one role per job purpose, trust scoped by tag or environment	Stored access keys; one powerful role for every job	A CI system that cannot issue OIDC tokens: then short-lived keys with automated rotation
Workloads	Task role, instance profile, or on Kubernetes IRSA or EKS Pod Identity	Keys in environment variables or in images	Code outside AWS that must call it: IAM Roles Anywhere, or OIDC from that platform
Operators on hosts	Session Manager and Run Command	SSH keys and an open port 22	Never by default; SSH only as a documented, temporary exception

The same idea on Azure and Google Cloud

The names change; the mechanism does not. Each cloud trusts the CI platform's OIDC tokens and attaches identities to compute.

CONCERN	AWS	AZURE	GOOGLE CLOUD
People's sign-in	IAM Identity Center: `aws sso login`	Entra ID: `az login`	`gcloud auth login`
CI without keys	IAM OIDC provider and a role	Entra ID federated credential (`azure/login`)	Workload Identity Federation (`google-github-actions/auth`)
A workload's identity	IAM role: task role, instance profile	Managed identity	Service account
Secrets	Secrets Manager, SSM Parameter Store	Key Vault	Secret Manager

Traps

SYMPTOM	CAUSE	FIX
The job fails with `Not authorized to perform sts:AssumeRoleWithWebIdentity`, and the policy looks right	The subject in the policy is not the one the token carries. The error never says which condition failed.	Read the real subject from the repository's OIDC setting, or print the token's claims in a test job, and match on that.
No token is issued at all	The job lacks `permissions: id-token: write`	Grant it on the job that needs it, not the whole workflow.
A deploy job that worked on tags is refused after you add an environment	A job that names an environment gets `…:environment:NAME` as its subject, not its ref	Trust the environment in the deploy role. It is the better gate anyway.
Anyone who can push can publish images to the cloud registry	The role trusts every `v*` tag, and a tag runs the workflow of the commit it points at, checks removed included	A tag ruleset: only admins create, move or delete release tags.
A deliberate break of the policy fails, so the policy seems proven	The policy may never have matched	Run the passing case first, then the failing one.
You cannot scope the registry login permission to one repository	`ecr:GetAuthorizationToken` is account-wide by design	Grant it on `*`, scope push and pull to the repositories, and say why in a comment.
The CDK deploy role can do anything in the account	The bootstrap's CloudFormation execution policy is AdministratorAccess by default	Narrow the execution policy before production use.
A workflow input runs as shell code	An input interpolated with `${{ }}` directly inside `run:`	Pass inputs through `env:` and quote them, as `deploy.yml` does.

- GOING DEEPER **What a trust policy can and cannot see**

GitHub's token carries many claims: the repository and its owner, the ref, the commit, the workflow file, the environment, the event that triggered the run. A trust policy can always test `aud` and `sub`, and AWS STS now also accepts several of GitHub's own claims

as condition keys, among them `repository_id`, `repository_owner_id`, `ref`, `environment` and `job_workflow_ref`. A role can then trust one workflow file directly. AWS advises the immutable IDs over names, because a renamed or deleted repository's name can be claimed by someone else. The subject still matters most, because every cloud's trust policy can match it, and GitHub lets a repository or organization customize which claims are combined into it. Including the workflow file in the subject, for example, is the portable way to let a role trust one specific workflow rather than any job in the repository.

The same customization setting is what produced the immutable form seen in the case above. Reading it, rather than assuming the default, is the reliable way to know what the policy must match.

`StringLike` allows the wildcards `*` and `?`; `StringEquals` does not. Use the exact form wherever there is nothing variable to allow, as the deploy role does, so a typo cannot widen the trust.

One OIDC provider entry per issuer is allowed in each account. Several roles can trust the same provider, each with its own conditions, which is how one account serves several repositories and environments.

Questions

1. What does a CI job need before it can obtain cloud credentials through OIDC, and what does it receive?

2. Why does the deploy role trust a GitHub environment rather than a tag or branch?

3. You break a trust policy on purpose and the job fails. What must you check before concluding that the policy works?

4. What is the difference between an ECS task execution role and a task role?

5. Why is removing SSH access a security improvement rather than only an inconvenience?

6. Which permission in the image-push role cannot be limited to the three repositories, and how is that handled?

Answers

1. The job needs `permissions: id-token: write`, so the CI platform will sign a token for it, and the cloud account needs an OIDC provider entry that trusts that platform. It presents the token to AWS STS and receives temporary credentials for a role, valid here for at most one hour.
2. A job that names an environment gets a subject ending in `:environment:NAME`. Trusting that subject means GitHub's environment protection rules, such as allowed branches and required reviewers, decide which jobs can reach production. Approvals can then change without editing AWS.

3. That the correct policy actually passes. A failure only proves something if a success was seen first. Otherwise the policy may never have matched, as in the case in this chapter.
4. The execution role is used by ECS before your code runs: to pull the image and to inject secrets. The task role is the identity your code uses to call AWS services while it runs. A task that calls no AWS API at runtime needs no task role.
5. There is no key to leak, no open port to attack, and every session or command is authorized by IAM, and CloudTrail records who opened or sent it, to which instance and when. The commands themselves are kept by session logging, which streams each session and Run Command's output to CloudWatch Logs, as the reference VM does.
6. `ecr:GetAuthorizationToken`, which is account-wide by design. It is granted on `*` with a comment saying why, while push and pull are granted only on the three repositories.

PART IV · GETTING CODE ONTO SERVERS

6 The deployment ladder, and one VM done properly

IN ONE SENTENCE

Deployment has climbed from copying files by hand to declaring the desired state and letting a platform converge on it, and a single, well-built virtual machine is still a sound first rung if it keeps the habits that make the next rung easy.

AT A GLANCE

- Each rung of the ladder hands one more job to automation: copying, then running, then replacing, then scheduling, then owning the machines.
- One VM is a good start for a pilot: one host, one script, logs in one place. What matters is that it runs released images, never a checkout.
- A deploy on one host can drop no requests: start the new version beside the old one, wait until it is healthy, then drain the old one.
- A proper VM has no SSH key and no open port 22, pulls images with its own role, and is rebuilt from code rather than patched by hand.
- Keep the rule "CI builds one image per version; deploying means run this version", and moving to a managed platform later is small.

THEN AND NOW

THEN	NOW	WHY IT CHANGED
`git pull` and a restart on the server	→ **The server pulls a released image by version and rolls it in**	A checkout can carry uncommitted edits; an image is exactly what was tested.
Servers set up by hand and patched in place	→ **Servers built from code at first boot, and replaced rather than repaired**	A server you can rebuild in minutes is one you never have to nurse.
SSH keys for every operator, port 22 open	→ **Session Manager and Run Command, authorized and recorded**	No key to leak, no port to attack, every session attributed to a person.
A maintenance window for each release	→ **Rolling deploys that keep serving, even on one host**	Users expect the service to stay up during a release.

THEN	NOW	WHY IT CHANGED
Logs read on the box	→ **Logs shipped off the box by the container runtime or an agent**	A replaced or broken host takes its local logs with it.

The ladder

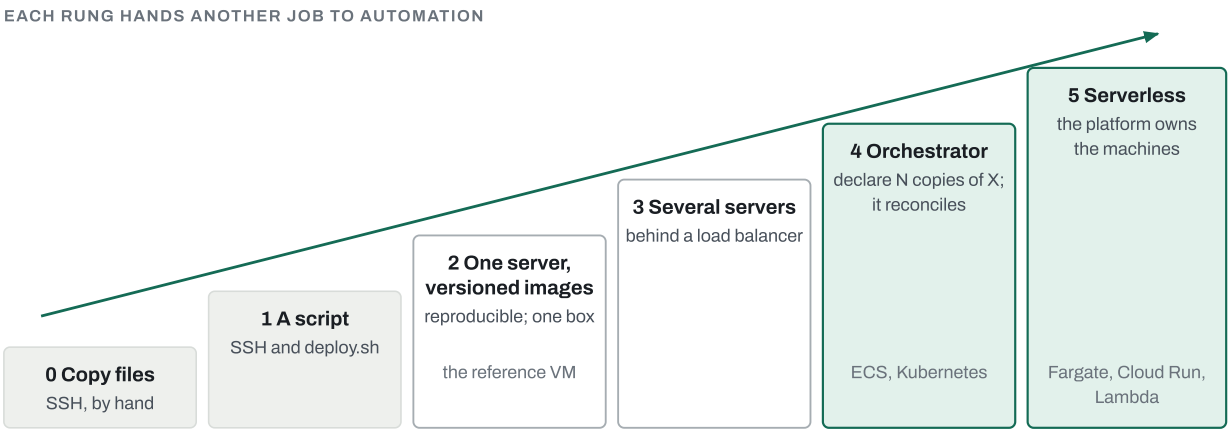


Figure 6.1 The deployment ladder. Higher rungs remove manual work and add capacity and redundancy, at the price of more concepts to learn. SSH survives at the top only as break-glass access.

Rungs 0 and 1 are where many teams started: log in, copy, restart. Rung 2 is the first that is reproducible, because the server runs an image built by CI. Rung 3 adds capacity and redundancy but leaves N machines to update without taking the fleet down. Rung 4 is the big change in thinking: instead of running commands, you **declare** what should be running, "two copies of version 0.8.2", and an orchestrator such as Kubernetes or ECS works continuously to make reality match, replacing a few copies at a time. Rung 5 removes the machines from view entirely.

"Serverless" does not mean one very reliable server. It means many small, disposable units that the platform replaces when they are unhealthy. Fargate and Cloud Run run your container images; Lambda runs a zip file or an image in lightweight virtual machines. A Python virtual environment isolates packages but not the operating system's libraries or the interpreter build, which is why a platform that wants an image gains nothing from a virtual environment alone.

One VM, without ceremony

A single VM is the right first home for a pilot with few users, where a short night-time maintenance window is acceptable and the team is new to operations. Where ECS asks a new team to learn roles, task definitions, load balancers and networking at once, a VM is one box, one script and logs in one place. The reference service ran on a VM before it ran on ECS, with the same images.

- **The image goes to a registry, not to Git.** The VM pulls it by version, authenticated by its own role on AWS, and never builds anything.

- **The deploy definition matches the version.** The compose file and deploy script must match the images, so they come from the same release: shipped by CI with each deploy, or checked out at the release tag. Never a working copy someone edited.
- **The deploy script never stops what works:** it pulls the version, runs migrations as a one-off container, starts the new api beside the old one and waits for it to be healthy, drains the old one, and only then records the version, so later commands run it too. If the new api never becomes healthy, the script removes it and stops; the old one was never stopped.
- **Rollback is the same script with the previous version.**

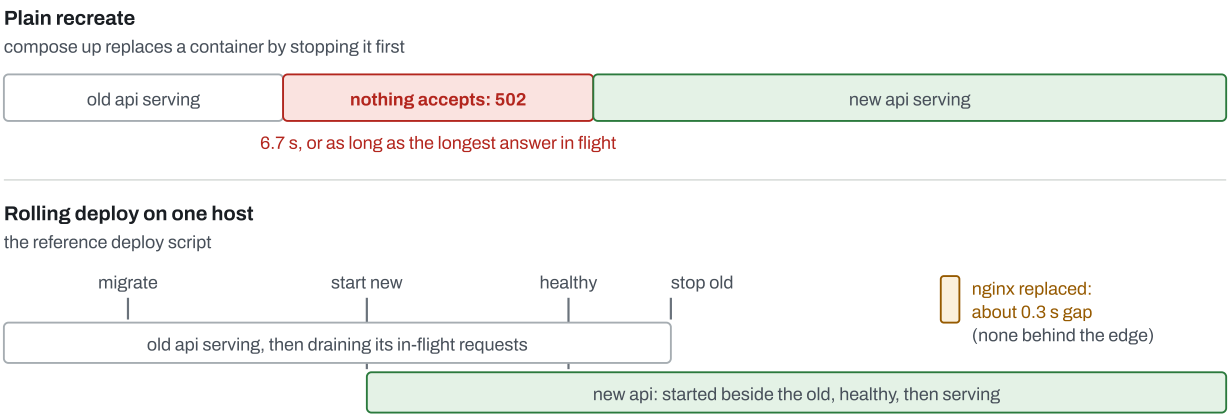


Figure 6.2 Why "recreate" is downtime. The rolling deploy failed no request during the api swap, with every in-flight stream finished. The one remaining gap is replacing the single proxy that owns the port: about 0.3 s when nginx owns it, as on the AWS VM. Behind the reference TLS edge, nginx swaps with no failed request, and the gap moves to replacing the edge itself, about 2 s, which the script does only when the edge changed. Only a load balancer in front of two hosts removes it.

IN THE REPO `scripts/deploy.sh` the rolling deploy for one host

The deploy script opens with its own design: why the obvious command is not used, and the five steps that replace it. It is about 190 lines of Bash, checked by a linter in CI, and its behavior is measured by a failure drill.

- Migrations run while the old version still serves, which is why they must be backward compatible.
- The new api is started beside the old one, and nginx must resolve it before the old one stops.
- Rollback is the same script with the previous version, and the schema is never reversed.
- Step 5's comment describes nginx owning the port. Further down, the script notes that the edge holds requests while nginx restarts (no request failed), and that it replaces the edge, about 2 s of refused connections, only when the edge's build inputs changed.

```
#!/usr/bin/env bash
# Deploy a release to this host without refusing requests.
#
#   make deploy tag=1.4.0          images from $IMAGE_PREFIX (ghcr.io/..., see .env)
#   PULL=0 make deploy tag=local   images already on this host (local testing)
#
# `docker compose up -d` replaces a container by stopping it first: the old
# api drains its in-flight requests while nothing accepts new ones (measured:
# 502s for 6.7s, and for as long as the longest answer in flight). Instead:
#   1. migrate (the old api is still serving: migrations must be backward
#      compatible - add first, remove in a later release)
#   2. start the new api NEXT TO the old one and wait until it is healthy
#   3. wait for nginx to resolve it (resolver valid=10s in default.conf)
#   4. stop the old api: it finishes its in-flight requests (streams
#      included, up to graceful_timeout) while nginx sends new ones to the new
#   5. replace nginx: behind the TLS edge no request fails (the edge retries);
#      where nginx publishes the port itself, ~0.3 s of refused connections;
#      then the edge, only if its image changed (~2 s of refused connections)
# Rollback = deploy the previous tag. The database stays as it is (the
# code rolls back, the schema does not), so the previous tag must be able to
# run on it: the expand/contract rule.
```

The VM done properly

The reference build ran the same stack on an EC2 virtual machine defined in CDK. What makes it acceptable to a platform team, compared with a hand-built server:

- **No SSH key and no port 22.** Operators use Session Manager and Run Command, which the instance's role allows. CloudTrail records who opened a session or sent a command; the stack's session logging keeps what was typed, and the Run Command output, in a log group.
- **No registry password.** A credential helper gets ECR credentials from the instance's role.

- **Built from code.** cloud-init installs Docker, creates the deploy user, clones the repository for its compose files and scripts, and generates the `.env` with random passwords at first boot. Because cloud-init runs only once, a changed recipe replaces the VM rather than editing it.
- **Small surface.** The metadata service accepts only its token-based version (IMDSv2), the disk is encrypted, and the security group admits HTTP and nothing else.

IN THE REPO `infra/aws/vm.py` the VM's role, what operators do, and its single open port

The VM's identity, its operations log and its network surface, in CDK.

- The role trusts only the EC2 service, and carries one managed policy: the one Session Manager needs. It may pull from the three image repositories, nothing more.
- What operators do is kept, not only that they connected: Session Manager's preferences, a document the stack owns, stream every session's input and output to the log group, and the operator script sends Run Command output there too. The role may write to that group and no other.
- The security group's description says it plainly: HTTP in, no SSH.

```
role = iam.Role(
    self,
    "Role",
    assumed_by=iam.ServicePrincipal("ec2.amazonaws.com"),
    managed_policies=[
        iam.ManagedPolicy.from_aws_managed_policy_name("AmazonSSMManagedInstanceCore")
    ],
)
for repo in repos.values():
    repo.grant_pull(role)
```

```
# What operators do on the host, not only that they connected. Session Manager reads
# its preferences from one document per account and region, SSM-SessionManagerRunShell:
# this stack creates it (it must not exist already) and deletes it with the stack.
operations = logs.LogGroup(
    self,
    "Operations",
    log_group_name="/triage/vm-operations",
    retention=logs.RetentionDays.ONE_MONTH,  # production: what audits require
    removal_policy=RemovalPolicy.DESTROY,  # production: RETAIN
)
```

```
sg = ec2.SecurityGroup(self, "Sg", vpc=vpc, description="HTTP in; no SSH")
sg.add_ingress_rule(ec2.Peer.any_ipv4(), ec2.Port.tcp(80), "the app")
```

Results on the VM, with requests flowing throughout: the first start of an older release from ECR brought all six services up healthy; the upgrade to the next release migrated, started the new api beside the old one and drained it; and the rollback, across that migration, said *"the database is ahead of 0.8.0: rolling back the code only, no migration"*. 139 requests arrived during the rollback, and none failed.

Making it boring for a team new to operations

CONCERN	THE HABIT
Data	A database snapshot before any deploy with a migration; migrations that leave the previous version working; backups copied off the host and a restore rehearsed on a clean one.
Access	No SSH open to the internet; a CI credential that can only run the deploy script.
Secrets	From a secret store or generated at first boot, into a file only the app can read; never in Git or the image.
HTTPS	An edge proxy on the VM with automatic certificates, or a cloud load balancer with a managed certificate.
Staying up	Restart policies, automatic security updates, container log rotation (the usual way a disk fills), disk and CPU alarms, an uptime check from outside that emails a person.
Recovery	A setup script or image, so losing the VM costs an hour, and a one-page runbook: deploy, roll back, logs, restart, whom to call.

The reference runbook for a demo VM even includes the request to send IT: Ubuntu Server LTS, 4 vCPU, 8 GB of memory and 40 GB of disk, with the reasoning. The full stack with monitoring idles at about 880 MiB but its memory limits add up to about 5 GiB; images take 2.9 GB, logs are capped per container, and two weeks of daily database dumps fit in about 1 GB.

Agents on hosts, and immutable infrastructure

A monitoring or log-shipping **agent** is a small background program that works for a remote service; nothing to do with AI agents. It is needed because the container runtime keeps only each container's standard output, and host metrics such as CPU and disk are a separate job. Often the runtime's logging driver is enough on its own, sending container output straight to CloudWatch.

How an agent reaches a host depends on the rung. On plain VMs it is baked into a **golden image** built by a tool such as Packer, or installed by the first-boot script. On ECS or Cloud Run it is a setting of the service, with no host to touch. On Kubernetes it is a DaemonSet: one copy per node. In every case the host is rebuilt from a versioned definition, never patched by hand. That is what **immutable infrastructure** means.

When to climb the next rung

Move off the single VM when there are paying customers, when releases must not drop even the brief gap of replacing the only proxy, or when the VM itself becomes a burden to patch and watch. If the habits of this chapter are kept (CI builds one image per version, and a deploy means "run this version"), the move to ECS in Chapter 9 changes where the images run, not how they are built.

> **HOW TO SAY IT**
>
> *"For the pilot we run one VM: released images, a rolling deploy that drops no requests, no SSH, rebuilt from code. Moving to ECS later changes where it runs, not how it's built."*

Traps

SYMPTOM	CAUSE	FIX
Every deploy causes a burst of 502 errors	Recreating the only container stops it before the new one exists	Start the new version beside the old, wait for health, then drain the old.
A script that worked yesterday cannot find the api container	A rolling deploy renames containers (`api-2`, `api-3`)	Never hardcode a container name; ask Compose for the service's container.
A routine `compose up` quietly rolled the service back	It ran without the deployed version set	The deploy records the version in the env file, so every later command uses it.
The VM boots without Docker or the deploy user, and no error appears	One YAML line in the first-boot file failed to parse, and cloud-init ignored the whole file	Validate it with `cloud-init schema` before deploying, in CI.
A first-boot loop ran as two broken commands	In a folded YAML block, a more-indented line keeps its newline	Parse the YAML and run the resulting command once in a test.
The cloud refuses the instance type	Account limits: a free-plan account may launch only free-tier-eligible types	List the eligible types first and pick one that meets the stack's minimum.
The backup was lost with the server	It was stored on the same disk	Copy dumps off the host, and rehearse a restore on a clean one.
A certificate expired without warning	Let's Encrypt no longer sends expiry emails	Monitor certificate expiry yourself.

Questions

1. What changes in your thinking between rung 3 and rung 4 of the ladder?
2. Why does recreating a single container cause downtime, and how does a rolling deploy avoid it on one host?

3. Why should a VM never run `git pull` and build?

4. What makes the reference VM acceptable to a platform team?

5. When is it time to move from one VM to a managed container platform?

Answers

1. You stop running commands against machines and start declaring the state you want, such as two copies of a version. The orchestrator reconciles reality to that declaration continuously, including replacing failed copies.
2. Recreating stops the old container first, so nothing accepts new requests while it drains. A rolling deploy starts the new version beside the old one, waits until it is healthy and reachable, and only then drains the old one.
3. A checkout can carry edits and drift, and builds on different hosts can differ. Pulling a released image means the host runs exactly what CI tested.
4. No SSH key or open port 22, access through Session Manager, with every session recorded in CloudTrail, images pulled with the instance's own role, a single open port, IMDSv2 and an encrypted disk, and a host rebuilt from code at first boot.
5. When there are paying customers, when releases must not drop any requests at all, or when the VM becomes a burden. If CI already builds versioned images and deploys mean "run this version", the move is small.

PART IV · GETTING CODE ONTO SERVERS

7 Infrastructure as code

IN ONE SENTENCE

Infrastructure as code describes networks, servers, databases and permissions in reviewed files, and a tool works out and applies the difference between that description and what exists, so environments can be rebuilt, reviewed and audited like software.

AT A GLANCE

- You declare the end state; the tool compares it with its record of what exists and makes only the calls that close the gap.
- Always look at the difference before applying it: `cdk diff` or `terraform plan`, ideally in the pull request.
- CDK writes AWS infrastructure in a real programming language and hands it to CloudFormation. Terraform uses its own language and reaches any cloud through providers.
- A change made by hand in the console is **drift**: detect it, and fix it through the code. A deploy that touches that resource will silently undo it; one that does not leaves it in place, unseen.
- Infrastructure changes go through pull requests, and CI applies them with keyless credentials.

THEN AND NOW

THEN	NOW	WHY IT CHANGED
Click through the cloud console, then write down what you did	→ **Declare it in code; the tool creates it**	A console session leaves no reviewable record and cannot be repeated exactly.
Hand-written CloudFormation JSON or YAML	→ **CDK in Python or TypeScript, or Terraform's HCL**	Loops, functions and types replace thousands of lines of copied template.
Configuration management tools changing running servers	→ **Immutable images, with a first-boot script for the last mile**	Replacing a server is more predictable than mutating it.
Nobody sure what a change will do	→ **A diff or plan reviewed in the pull request before anything is applied**	The reviewer sees which resources will be created, changed or destroyed.
Environments that slowly drift apart	→ **Drift detection against the recorded state**	A manual change is found instead of being discovered during an outage.

How the tools work

Every infrastructure-as-code tool has the same three parts: a description of the desired end state, a record of what it has already created, and an engine that turns the difference into API calls. The tools differ in the language you write and in where the record lives.

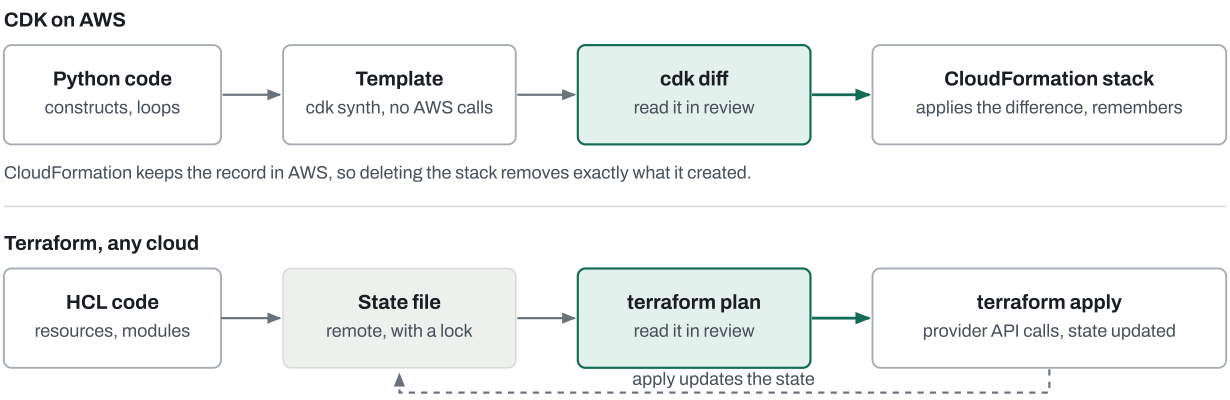


Figure 7.1 Two engines, one idea. Code is turned into a desired state, compared with the record of what exists, and only the difference is applied. The step in the accent color is the one a reviewer must read.

CDK (AWS Cloud Development Kit) lets you write infrastructure in Python, TypeScript and other languages. It synthesizes a CloudFormation template, and **CloudFormation** creates the resources and remembers them as a stack. Because AWS keeps that record, deleting the stack removes exactly what it created, and drift detection compares reality with it.

Terraform uses its own declarative language, HCL, and reaches any cloud or service through **providers**. It keeps its record in a **state file**. That file must live remotely (an S3 bucket, an Azure storage blob, a Google Cloud bucket) with a lock, so two people applying at once cannot corrupt it, and it must be protected: it can contain secrets. Terraform moved to the Business Source License; OpenTofu is its open-source fork. **Pulumi** offers real programming languages across clouds.

CHOOSE	WHEN
CDK	The work is AWS-only, and the team prefers writing a real programming language. The reference service uses it.
Terraform or OpenTofu	The organization runs several clouds, or a platform team has already standardized on it. This is the common case in larger organizations.
Pulumi	Several clouds, and a strong preference for a general-purpose language over HCL.

The reference infrastructure, as five stacks

The reference service's AWS infrastructure is about a thousand lines of Python in `infra/aws/`, split into five stacks by lifetime and cost. Things that are free and permanent (the registry, the CI roles, the network) live apart from the things that cost money by the hour (the VM, the ECS application), so the expensive parts can be destroyed and re-created at will.

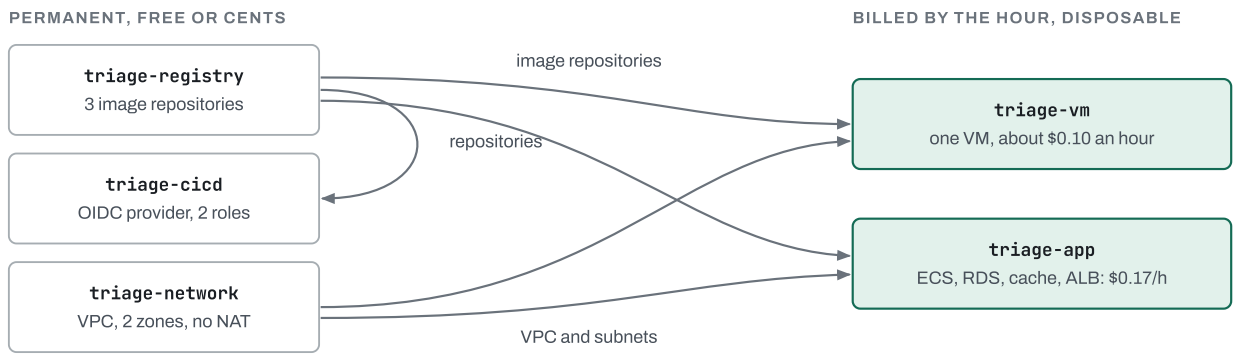


Figure 7.2 Stacks split by lifetime. Each arrow is a cross-stack reference, which CloudFormation implements as an export from the producer and an import in the consumer: the producer must be deployed first, and cannot delete what a consumer still imports.

IN THE REPO `infra/aws/app.py` and a dependency fixed in `service.py`

The CDK entry point wires the five stacks together by passing Python objects. CDK turns each object passed between stacks into a CloudFormation export and import.

- The account comes from the environment, not from the code, so no account ID is committed to the public repository.
- The second excerpt is a fix for a real teardown failure: CloudFormation deleted a database and its password secret at the same moment, and the database's delete step then failed. An explicit dependency orders the deletes.
- The comment records the recovery procedure for anyone who meets the same failure.

```
app = cdk.App()
env = cdk.Environment(account=os.environ["CDK_DEFAULT_ACCOUNT"], region="ap-south-1")
registry = RegistryStack(app, "triage-registry", env=env)  # Stage 1
CicdStack(app, "triage-cicd", repos=registry.repos, env=env)  # Stage 2
network = NetworkStack(app, "triage-network", env=env)  # shared by 3-5, free
VmStack(app, "triage-vm", vpc=network.vpc, repos=registry.repos, env=env)  # Stage 3
AppStack(
    app, "triage-app", vpc=network.vpc, db_subnets=network.db_subnets, repos=registry.repos, env=env
)  # Stages 4-5
app.synth()
```

```
# Delete the database before its master secret. The template refers to the secret only
# inside a {{resolve:secretsmanager:...}} reference, and CloudFormation deleted both at
# once: the DB's delete handler then could not resolve the secret → DELETE_FAILED
# (recovery: delete-stack --retain-resources for what is already gone).
db.node.add_dependency(owner.node.default_child)  # the secret only, not its attachment
```

The first attempt at that fix depended on the secret's whole construct, which includes an attachment that itself depends on the database: a cycle. It was caught when the template was synthesized, before anything was deployed, which is one reason to run `cdk synth` and a template linter in CI.

Drift: when reality and the code disagree

To test scaling, the reference ECS service was scaled from one task to two with a single command-line call: a change made outside the code. CloudFormation's drift detection reported it precisely. Left alone, the next deploy, which updates the service with a new task definition every time, would have set the count back to one without anyone noticing.

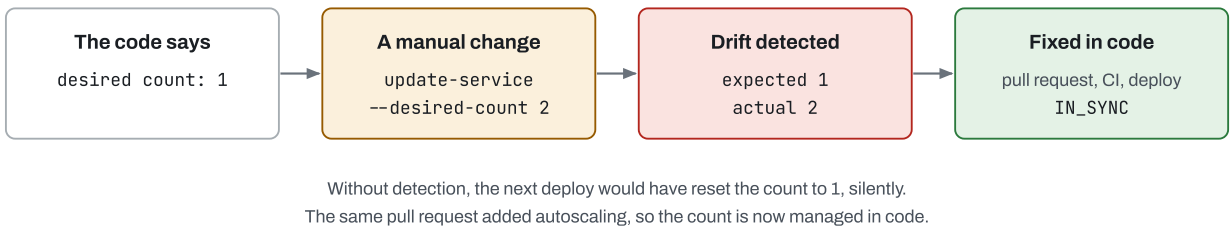


Figure 7.3 Drift on the reference stack. The fix went the proper way: a pull request, green CI, a merge, and the deploy workflow applying it. Drift detection then reported the stack in sync.

That reset depends on the deploy touching the drifted resource. A normal CloudFormation update compares the new template with the previous one, not with reality, so drift on a resource the update leaves alone stays in place, unseen until drift detection runs. Run detection regularly, and reconcile what it finds in code, or with a drift-aware change set (`--deployment-mode REVERT_DRIFT`), which compares with the actual state and brings drifted resources back. Terraform refreshes the real state before it plans, so its next plan shows the drift and applying it reverts the change.

The last mile: first-boot scripts

A virtual machine still needs something to run on its first boot. **cloud-init** (passed as `user_data` on AWS) runs once, at first boot: install the container runtime, log in to the registry with the machine's role, pull a specific version, run it with a restart policy, fetch secrets at runtime. It does not run again on later deploys. A new version either replaces instances (a new launch template version, rolled out by an Auto Scaling group's instance refresh) or runs a command on the existing ones through Run Command. The mature form bakes the runtime and agents into a golden image and keeps the first-boot script to "pull and run".

The same registry in Terraform, on three clouds

The language and the workflow carry over between clouds; the resources do not. An AWS container service has no Azure twin with the same fields, so it is rewritten. The image registry from this chapter's first stack, in Terraform on each cloud:

```
resource "aws_ecr_repository" "api" {
  name                 = "triage-assistant-api"
  image_tag_mutability = "IMMUTABLE"
  image_scanning_configuration { scan_on_push = true }
}

resource "azurerm_container_registry" "images" {
  name                = "triageassistant"          # globally unique, letters and digits only
  resource_group_name = azurerm_resource_group.app.name
  location            = "centralindia"
  sku                 = "Basic"
}

resource "google_artifact_registry_repository" "images" {
  repository_id = "triage-assistant"
  location      = "asia-south1"
  format        = "DOCKER"
}
```

> HOW TO SAY IT
>
> *"All infrastructure is code in the repo. Changes go through pull requests with the diff in review, CI applies them with keyless credentials, and drift detection catches anything changed by hand."*

Traps

SYMPTOM	CAUSE	FIX
`No export named … found` when deploying one stack	It references a resource in another stack, and that stack was not updated first	Deploy the producing stack first; cross-stack references are ordered dependencies.
A change made in the console disappeared	The next deploy reset it to what the code says	Run drift detection, and make every change in code.
A stack delete ends in `DELETE_FAILED`	Resources were deleted in parallel although one needed the other	Add an explicit dependency; finish a stuck delete by retaining the resources that are already gone.
A dependency fix creates a cycle	Depending on a whole construct pulls in parts that depend back	Depend on the specific resource; synthesize and lint the template in CI.

SYMPTOM	CAUSE	FIX
The account ID ended up in a public repository	Generated output (`cdk.out`, the context cache) was committed	Ignore generated output from the first commit; read the account from the environment.
The tool cannot work out the account from a sign-in profile	Some sign-in methods are not understood by every tool version	Export the account explicitly in the environment before running it.
Secrets readable in the Terraform state	State stores resource attributes in plain text	Remote, encrypted state with restricted access and a lock.
A small edit to the first-boot script replaced the server	cloud-init runs only at first boot, so a changed script means a new instance	Expect it; keep data off the instance and the boot script small.

Questions

1. What are the three parts of every infrastructure-as-code tool?
2. Why must a Terraform state file be remote and locked?
3. What is drift, and why is it dangerous even when the manual change was correct?
4. When would you choose Terraform over CDK?

Answers

1. A description of the desired state, a record of what already exists (a CloudFormation stack or a Terraform state file), and an engine that turns the difference into API calls.
2. So the whole team shares one record, and two applies cannot run at once and corrupt it. It must also be access-controlled and encrypted, because it can contain secrets.
3. Drift is a difference between the real infrastructure and the code. If the next deploy updates that resource, it restores what the code says, so a correct manual change is silently undone. If it does not, the change stays unseen, and nobody reviewing the code knows it exists. Either way the code no longer describes what is running.
4. When the organization runs several clouds, or a platform team has already standardized on Terraform. CDK fits AWS-only work by a team that prefers a general-purpose language.

PART V · RUNNING ON AWS

8 Networking on AWS

IN ONE SENTENCE

An AWS network is built from layers you compose (a private network, subnets that a route table makes public or private, gateways for traffic in and out, and firewalls that name each other), and the safest designs leave only the load balancer reachable from the internet.

AT A GLANCE

- A **VPC** is your own isolated network in one region. A **subnet** is a slice of it in one availability zone.
- A subnet is public or private only because of its **route table**: public means it has a route to an internet gateway.
- A **NAT gateway** lets private subnets reach out without being reachable. It costs money per hour and per gigabyte.
- **Security groups** are stateful allow-lists on each resource, and can name other security groups instead of IP addresses: the load balancer may reach the api, and only the api may reach the database.
- Put the database in a subnet with no route to the internet at all, so a single security group mistake cannot expose it.

THEN AND NOW

THEN	NOW	WHY IT CHANGED
Physical firewalls and VLANs configured by a network team	→ **VPCs, subnets and security groups defined in code**	The network is reviewed and rebuilt with the application.
Firewall rules written as IP address ranges	→ **Security groups that reference other security groups**	Addresses change as tasks start and stop; group membership does not.
A DMZ host in front of an internal network	→ **A public subnet holding only the load balancer**	The same idea, with the platform managing the edge.
One data center	→ **At least two availability zones**	A zone can fail or run out of capacity; the service should not notice.
A bastion host with SSH for administration	→ **Session Manager, with no inbound port at all**	Nothing to attack, and every session recorded.

A home router, split into AWS resources

A home router does three jobs in one box, and AWS turns each into a separate resource you configure on its own. The mapping is a good way in.

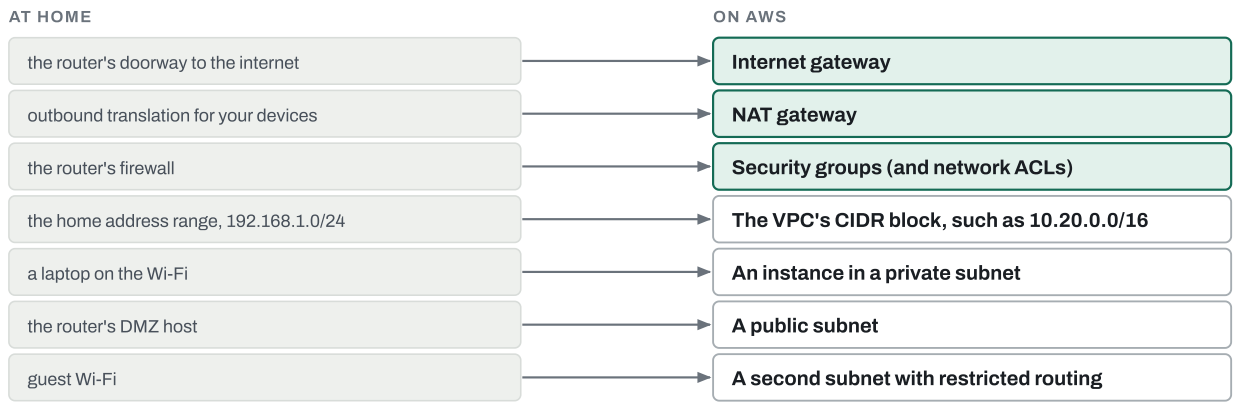


Figure 8.1 The home router's jobs, and their AWS equivalents. On AWS each job is a separate resource, which is why a cloud network has more parts to configure but also more control.

The parts

- **VPC.** Your own isolated network, spanning one region, with an address range written in CIDR notation.
- **Subnets.** Slices of that range, each pinned to one availability zone. Routing is decided per subnet by its route table, which is the real reason subnets exist. The usual minimum is a public and a private subnet in each of two zones, because load balancers and database subnet groups both require two.
- **Internet gateway.** The doorway. A subnet with a route to it is public.
- **NAT gateway.** Lets private subnets make outbound connections, to a model provider's API or a package mirror, without accepting inbound ones. It is billed per hour and per gigabyte processed.
- **Security groups.** Stateful, per-resource, default-deny allow-lists. Stateful means a reply to an allowed request is allowed automatically. A rule can name another security group as its source.
- **Network ACLs.** A coarser, stateless filter per subnet. Most teams leave them at their defaults unless compliance requires more.

The reference network

The reference stack's network is deliberately cheap, and the code says so. It has two zones with public subnets for the load balancer, the VM and the ECS tasks, and isolated subnets for the database, with no route to the internet in either direction. There is no NAT gateway, because nothing in a private subnet needs to reach out. The trade-off is named in the code: tasks get public IP addresses so they can pull images, and they rely on security groups, not subnets, to stay unreachable.

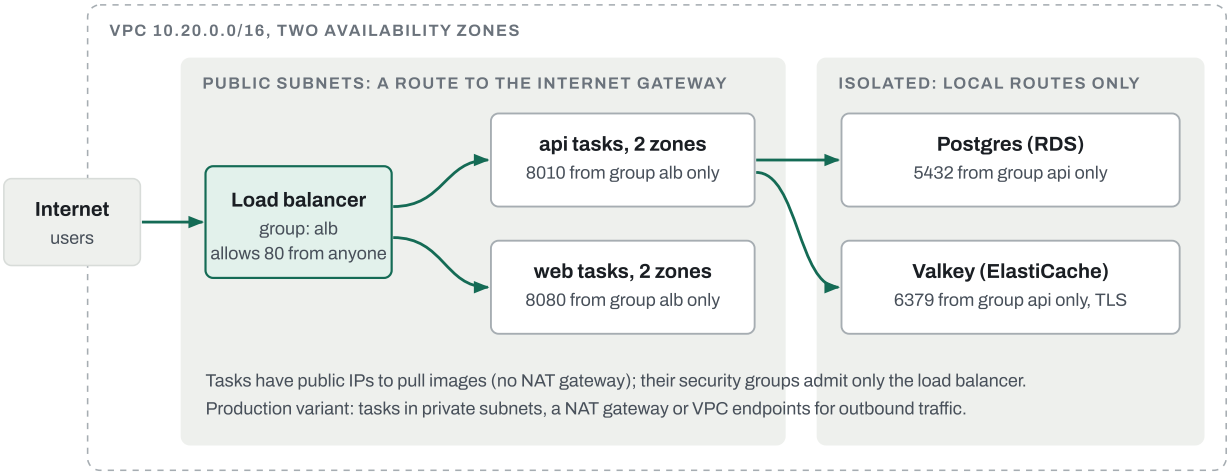


Figure 8.2 The reference network. Security groups name each other in a chain: internet to load balancer, load balancer to tasks, api to data. A task added by autoscaling joins its group and is covered without any rule changing.

IN THE REPO `infra/aws/network.py`

The whole network is about forty lines. Its docstring states the design and its trade-off, and the second excerpt is a fix that only running it could have revealed.

- Two kinds of subnet: public, and isolated (no route out at all), each a /24 in two zones.
- `nat_gateways=0` saves about six cents an hour plus data charges, at the cost of public addresses on the tasks.
- The zones are named in code rather than looked up, so the template can be produced without cloud credentials.
- RDS refused to create the database in the first two zones for lack of capacity. Capacity is per zone and changes over time; a third isolated subnet solved it without touching anything already running.

```
"""Stages 3-5: one VPC shared by the VM and the ECS stack. A VPC itself costs nothing.

Two availability zones (an ALB and an RDS subnet group both require two). Public subnets hold
the load balancer, the VM and the Fargate tasks; isolated subnets (no route to the internet in
either direction) hold the database. No NAT gateway: nothing in a private subnet needs to reach
out, and one would cost ~$0.06/hour plus data. The trade-off: tasks get public IPs (each
$0.005/hour) and rely on security groups, not subnets, to stay unreachable.
"""
```

```python
# A constant, not read from the VPC: the app stack trusts proxies in this range
# (FORWARDED_ALLOW_IPS), and a token would make it import from this stack.
VPC_CIDR = "10.20.0.0/16"


class NetworkStack(Stack):
    def __init__(self, scope: Construct, cid: str, **kw) -> None:
        super().__init__(scope, cid, **kw)
        self.vpc = ec2.Vpc(
            self,
            "Vpc",
            ip_addresses=ec2.IpAddresses.cidr(VPC_CIDR),
            # Named, not looked up: synth works without credentials.
            availability_zones=["ap-south-1a", "ap-south-1b"],
            nat_gateways=0,
            subnet_configuration=[
                ec2.SubnetConfiguration(
                    name="public", subnet_type=ec2.SubnetType.PUBLIC, cidr_mask=24
                ),
                ec2.SubnetConfiguration(
                    name="data", subnet_type=ec2.SubnetType.PRIVATE_ISOLATED, cidr_mask=24
                ),
            ],
        )
        # Added after RDS refused 1a and 1b: "no subnets exist in Availability Zones with
        # sufficient capacity ... for db.t4g.micro; choose from ap-south-1c". Capacity is per AZ
        # and changes; an extra isolated subnet (main route table: local routes only) gives the
        # database a third zone without touching the subnets the VM already uses.
        self.data_1c = ec2.Subnet(
            self,
            "Data1c",
            vpc_id=self.vpc.vpc_id,
            availability_zone="ap-south-1c",
            cidr_block="10.20.10.0/24",
        )
        self.db_subnets = [*self.vpc.isolated_subnets, self.data_1c]
```

Address ranges in CIDR notation

CIDR notation writes a range as an address and the number of fixed bits. `10.20.0.0/16` fixes the first 16 of 32 bits and leaves 16 free: 65,536 addresses. A smaller number after the slash is a bigger range. AWS reserves five addresses in every subnet, so a `/24` holds 251 usable ones. A subnet's range caps how many resources it can hold (one address each, not one per connection), which matters for tasks that each get their own network interface.

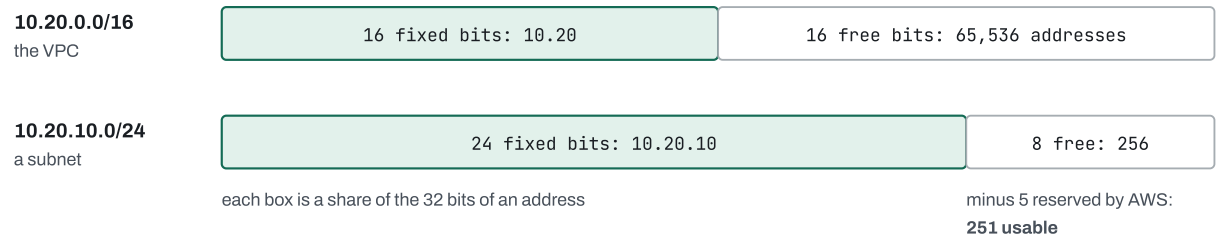


Figure 8.3 CIDR notation, drawn to scale. Every bit moved from "free" to "fixed" halves the range. Plan ranges that do not overlap with other networks you may later connect to, such as an office or another VPC.

Toward a production network

The production version of this design moves the tasks into private subnets with no public addresses. Outbound traffic then needs a NAT gateway, and because it is billed per gigabyte, heavy traffic such as image pulls should go through **VPC endpoints**: private connections to ECR and S3 that skip the NAT gateway. If a security team restricts outbound traffic, the model provider's API host must be on the allowed list, or every assistant request fails. The public DNS record points at the load balancer as an alias, never at an IP address, because the load balancer's addresses change.

> HOW TO SAY IT
>
> *"Only the load balancer is public. Security groups chain from it to the tasks and from the api to the data, by group rather than by IP, and the database sits in subnets with no route to the internet at all."*

Traps

SYMPTOM	CAUSE	FIX
The database will not create in a zone the catalog says it supports	Capacity is per zone and temporary	Add a subnet in another zone; plan for at least three possible zones for data.
Removing a security group rule did not cut the application off	Security groups are stateful; existing connections, such as a database pool, stay open	Know that revoking access does not end live sessions; stop the resource for a real outage test.
The bill shows large data processing charges	Image pulls and S3 traffic through a NAT gateway are billed per gigabyte	VPC endpoints for ECR and S3.

SYMPTOM	CAUSE	FIX
Every model call fails after a network hardening change	Outbound traffic to the provider's API is no longer allowed	Allow the provider's hostname explicitly; configure the corporate proxy where there is one.
Public IP addresses appear on the bill	AWS charges for every public IPv4 address, about half a cent an hour each	Private subnets in production; count public addresses in a cheap design.
The site breaks when the load balancer's IP changes	DNS pointed at an IP address	Use an alias record that points at the load balancer's name.

Questions

1. What makes a subnet public?
2. Why reference security groups rather than IP ranges in rules?
3. How many usable addresses does a /24 subnet have on AWS, and why?
4. Why does the reference stack put its tasks in public subnets, and what keeps them safe?

Answers

1. Its route table has a route to an internet gateway. Nothing about the subnet itself is public or private; routing decides.
2. Tasks and instances come and go with new addresses. A rule that names a group covers every current and future member, so autoscaling needs no rule changes, and the intent ("only the api may reach the database") is explicit.
3. 251: a /24 has 256 addresses, and AWS reserves five in every subnet.
4. To pull images without paying for a NAT gateway. Their security groups admit traffic only from the load balancer's group, so a public address does not make them reachable. Production would move them to private subnets.

PART V · RUNNING ON AWS

9 Containers on ECS Fargate

IN ONE SENTENCE

ECS on Fargate runs containers with no servers to manage: you describe each task once, a service keeps the right number running behind a load balancer, and every release is a rolling replacement that rolls itself back if the new version cannot start.

AT A GLANCE

- A **task definition** is a versioned description of containers: image, CPU and memory, environment, secrets, logs, health check. A **service** keeps N copies running.
- A release registers a new task definition revision and updates the service: new tasks start beside the old ones, and the old ones stop once the new are healthy.
- The deployment circuit breaker rolls a release back by itself when its tasks keep failing to start.
- ECS has **no readiness-only** health signal: the container's own check and the load balancer's check both end in a replaced task. Point both at a shallow endpoint.
- Anything that must be counted once across copies, such as rate limits or sessions, needs a shared store. A sidecar in each task counts on its own.

THEN AND NOW

THEN	NOW	WHY IT CHANGED
Compose on a server you patch	→ **Task definitions and services on Fargate, no hosts to manage**	The platform owns the machines, the patching and the placement.
A restart policy brings a crashed container back	→ **The service scheduler replaces failed tasks, in any availability zone**	Recovery from a lost host is automatic, not just from a crashed process.
A proxy container routes paths to services	→ **Load balancer listener rules, including path rewriting**	Routing becomes managed configuration, highly available by default.
A deploy script and a manual rollback	→ **Rolling updates with a circuit breaker that rolls back on its own**	A bad release stops itself before users notice.
Scaling by resizing the server	→ **Autoscaling the number of tasks on requests per target**	Capacity follows traffic, within limits you set.

The pieces

PIECE	WHAT IT IS
ECR	The image registry.
Task definition	The versioned container spec, with numbered revisions: images, CPU and memory, environment, secrets from Secrets Manager, log destination, health checks, ports, and the order containers start in.
Service	Keeps N copies of a task running, replaces failed ones, and performs rolling deploys.
Application Load Balancer	The public entry point. Terminates TLS with a managed certificate, routes by path, and sends traffic only to healthy tasks.
Execution role and task role	The first lets ECS pull images and read secrets; the second is what your code may call (Chapter 5).
CloudWatch Logs	Every container's standard output and error, shipped by the platform.

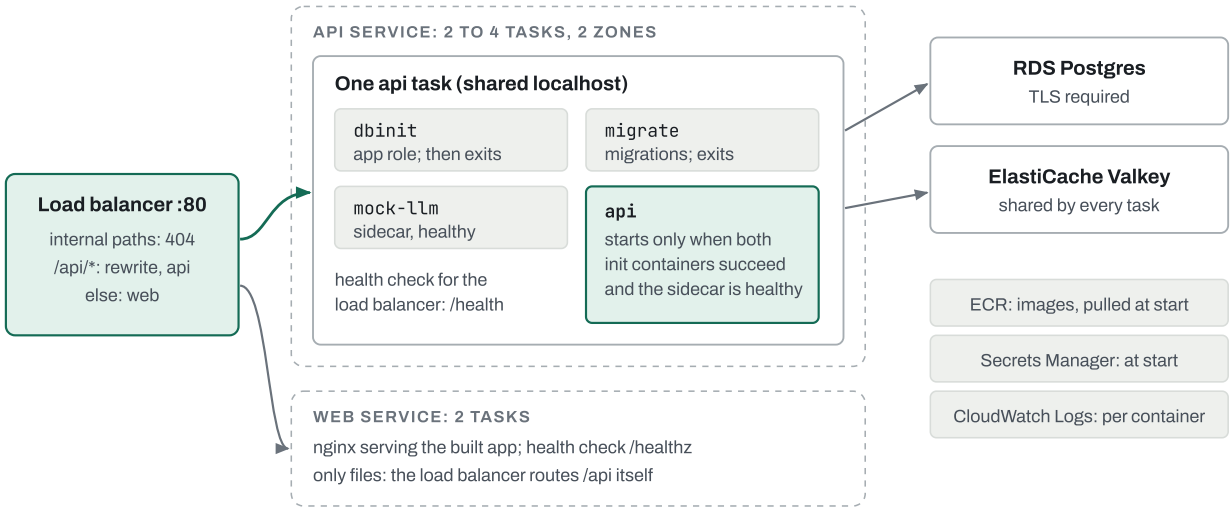


Figure 9.1 The reference service on ECS Fargate. The containers in one task share a network namespace and talk over localhost. Services that scale separately are separate services, found through the load balancer.

Compose, translated to ECS

COMPOSE	ECS
a `services:` entry	a task definition and a service
`build:`	CI builds; the task definition names an image in ECR
`ports:`	the container port, and a load balancer target group
`environment:` and `.env`	`environment` for plain values; `secrets` from Secrets Manager or Parameter Store
`depends_on:`	inside one task, `dependsOn` with START, COMPLETE, SUCCESS or HEALTHY; between services, nothing: the app retries
`restart:`	the service scheduler replaces failed tasks
named volumes	mostly none: state lives in RDS and S3, or EFS when a shared file system is truly needed
the database and cache containers	RDS and ElastiCache
the Compose network and its service names	the VPC and security groups; the load balancer, Service Connect or Cloud Map for names

The Compose file stays as the local development environment, and the task definition is its production counterpart. Five Compose containers often become two or three ECS services plus managed services.

Rehearse the task locally first

Most cloud failures are configuration, and a Fargate task can be rehearsed on one machine before any money is spent: containers in a task share one network namespace, which `docker run --network container:<first>` reproduces, and a plain Postgres container with only an owner user stands in for RDS, which runs no setup scripts. The rehearsal for the reference build found four problems:

- **The production guard worked.** With production settings the api refused to start: plain-HTTP cookies and a fake model. Without a domain or a model provider, both were honest exceptions, named one by one in a waiver setting rather than turned off.
- **The web image assumed Compose.** Its nginx found the api through Docker's internal DNS, which Fargate does not have. On AWS the load balancer routes `/api/*` straight to the api and strips the prefix, and nginx only serves files.
- **A rollback trap.** Rolling back across a migration made the migration tool fail on a revision it did not know. The migration step was taught to recognize that case and do a code-only rollback (Chapter 13).
- **Least privilege.** A setup container creates the application's own database user, allowed to read and write rows but not to change the schema, and does it idempotently, because RDS will not run setup

scripts.

How a task starts: init containers

Each api task first runs two short-lived **init containers**: one creates the least-privileged database user, the next runs migrations as the schema owner. Only when both have exited successfully, and the model sidecar is healthy, does the api container start. Every new task migrates before it serves, which is simple, but tasks can start together (the first deploy starts two), and two migration runs at once collided: one failed. So the runs take turns on a database lock, and a deploy also starts one new task at a time.

One detail of how ECS evaluates those dependencies cost an afternoon. When the database was down, a new task's first init container failed straight away; the second, which depended on it, then never ran; and the api, which depended only on the second, waited forever. The task stayed PENDING for 13 minutes, billed, until stopped by hand.

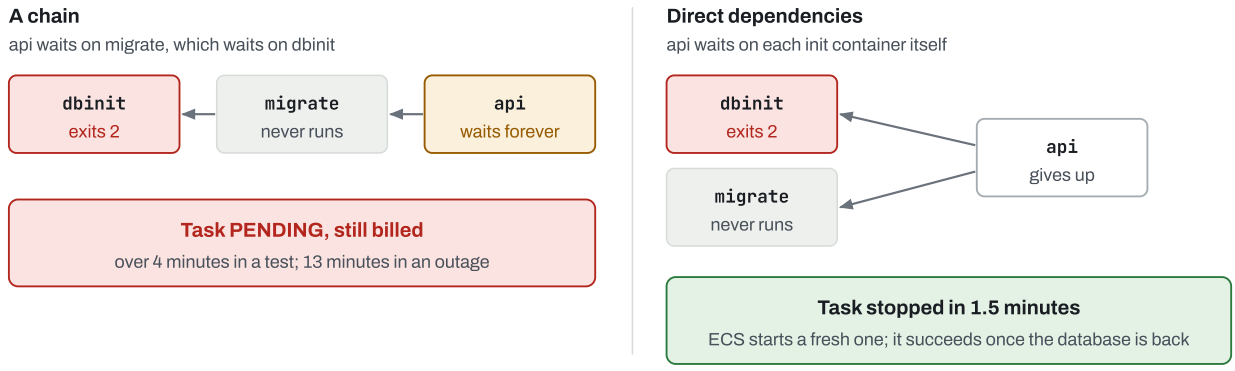


Figure 9.2 The same failure, two dependency shapes. A start timeout only covers a container that is still running, not one that already failed two links down a chain. The rule: the essential container depends directly on every init container, and each init container has a start timeout.

IN THE REPO `infra/aws/service.py` the api's start conditions

The fix, with the measurement that justified it in the comment. Each dependency names a condition: the init containers must exit with `SUCCESS`, the sidecar must be `HEALTHY`.

- All three are direct dependencies of the api, the essential container: when any fails, the api gives up and ECS stops the task.
- Each init container also has a 120-second start timeout, the most Fargate allows.
- The comment keeps the evidence: the chain stuck for more than four minutes; direct dependencies stopped the task in one and a half.

```
# Every init container is a direct dependency of the essential api: when one fails, the
# api gives up and ECS stops the task. Through a chain (api → migrate → dbinit) a dbinit
# that fails fast leaves migrate "stopped" without running, and the task PENDING forever
# (measured with one-off tasks: the chain stuck >4 min; direct: stopped in 1.5 min).
api.add_container_dependencies(
    ecs.ContainerDependency(
        container=dbinit, condition=ecs.ContainerDependencyCondition.SUCCESS
    ),
    ecs.ContainerDependency(
        container=migrate, condition=ecs.ContainerDependencyCondition.SUCCESS
    ),
    ecs.ContainerDependency(
        container=mock_llm, condition=ecs.ContainerDependencyCondition.HEALTHY
    ),
)
```

A release on ECS

1. CI copies the release images to ECR, authenticating with an OIDC role (Chapter 5).
2. The deploy registers a new task definition revision with the new image and updates the service.
3. ECS starts new tasks beside the old ones, waits for the load balancer's health check, shifts traffic, and stops the old tasks, which finish their in-flight requests first.
4. If the new tasks keep failing to start, the **deployment circuit breaker** rolls the service back to the previous revision, and the deploy fails.

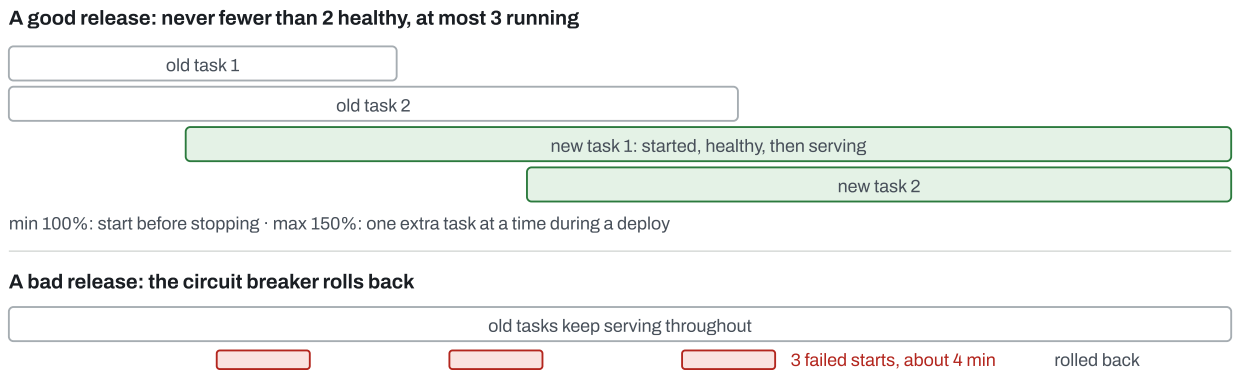


Figure 9.3 Rolling deploys on the reference service. In the deliberate bad-release drill, the new api refused to start on a production check; ECS retried three times, the circuit breaker rolled back, and CloudFormation removed the bad revision. Users kept being served by the old tasks.

Measured on the reference service, with a probe sending requests twice a second: a forward deploy took 3 minutes 17 seconds with 352 requests and none failed, and the deliberate bad release was rolled back in under 8 minutes while the old tasks kept serving. ECS Exec gives a shell into a running task, without SSH, for the rare case that needs one.

No readiness-only signal

The reference api follows the Kubernetes convention of two health endpoints. `/health` answers "is the process alive?" and checks nothing external. `/ready` answers "should traffic come here?" and requires

the database. On Kubernetes they feed two separate probes, and only the liveness probe restarts anything. **ECS has no readiness-only signal.** It acts on two checks, and both end in a replaced task: the container health check in the task definition (the reference api's calls `/health`), and the load balancer's target health check. So a deep check behaves like a liveness check wherever it sits, and a database outage turns into task churn. A drill on the load balancer's check showed exactly that:

DATABASE STOPPED FOR 7 MINUTES	LOAD BALANCER CHECKS `/READY` (DEEP)	LOAD BALANCER CHECKS `/HEALTH` (SHALLOW)
Target health	every api target unhealthy	2 of 2 healthy
What ECS did	replaced tasks, whose replacements could not start	nothing
What users saw	`/ready` 503; the page and `/health` fine	the same
Alarms	unhealthy targets and 5xx	5xx only

The rule on ECS: point the load balancer at a shallow check; rely on the init containers for readiness at start (migrations succeed only if the database answers); and detect outages with an alarm on 5xx responses. The api reconnected four seconds after the database returned, with no restart.

IN THE REPO `infra/aws/service.py` the service's deployment settings and the api's health check

The service definition and the api target group's health check, each with its reasoning beside it.

- `min_healthy_percent=100` and `max_healthy_percent=150`: start one new task before stopping an old one, one at a time.
- The circuit breaker is enabled with rollback.
- The health check path is `/health`, and the comment explains why `/ready` would turn an outage into churn. ECS has no readiness-only signal: the api's container health check, in the task definition, also calls `/health`, and ECS replaces the task when either one fails.
- The deregistration delay covers the longest answer. ECS takes a task out of the load balancer before it sends SIGTERM, and the balancer drops whatever is still in flight when the delay ends: at 30 seconds, long answers were cut during deploys.

```
468     return ecs.FargateService(
469         self,
470         f"{name.title()}Service",
471         service_name=name,
472         cluster=cluster,
473         task_definition=task,
474         desired_count=2,
475         # A public IP to pull images without a NAT gateway; the SG admits only the ALB.
476         # production: private subnets, a NAT gateway or VPC endpoints for ECR and logs.
477         assign_public_ip=True,
478         vpc_subnets=public,
479         security_groups=[sg],
480         min_healthy_percent=100,  # start a new task before stopping an old one
481         # 150% of 2 = one extra task at a time during a deploy. Tasks still start together
482         # (the first deploy, a scale-out): their migrate init containers then take turns on
483         # an advisory lock (apps/api/migrations/env.py).
484         max_healthy_percent=150,
485         circuit_breaker=ecs.DeploymentCircuitBreaker(enable=True, rollback=True),
486         health_check_grace_period=Duration.seconds(120),
487         # `aws ecs execute-command`: a shell without SSH. production: off, or limited by
488         # IAM to the on-call role, with session logging.
```

```
516     # Liveness, not readiness. ECS replaces any task its load balancer calls unhealthy,
517     # so a deep check (/ready: the database) turns a database outage into task churn:
518     # measured, every api task marked unhealthy and replacements stuck in dbinit. Kubernetes
519     # separates the two (readinessProbe /ready, livenessProbe /health); ECS has no
520     # readiness-only signal: this check and the container health check both replace tasks.
521     # Readiness at start is covered by the init containers: migrate succeeds only if the
522     # database answers. A database outage still shows: /api/ready 503 and the 5xx alarm.
523     health_check=elbv2.HealthCheck(
524         path="/health", interval=Duration.seconds(10), healthy_threshold_count=2
525     ),
526     # ECS takes a task out of the ALB first, waits this long, and only then sends it
527     # SIGTERM; at the end of the wait the ALB drops whatever is still in flight. So the
528     # wait must cover the longest chat stream (LLM_STREAM_TIMEOUT_S, 120 s): at 30 s,
529     # any answer longer than that was cut during a deploy. Deploys wait up to this long
530     # per task only while requests are in flight.
531     deregistration_delay=Duration.seconds(130),
```

Scaling out, and the state that must be shared

The api's login endpoint is rate-limited per client: 30 attempts a minute, counted in Valkey. While the api ran with a private Valkey container beside it in each task, scaling to two tasks quietly doubled the limit, because each task counted on its own.

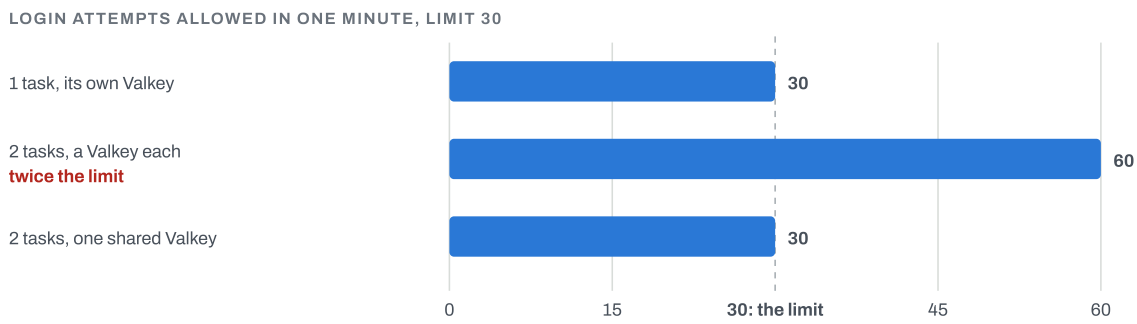


Figure 9.4 Per-task state breaks when you scale. Measured through the load balancer: with a cache container in each task, every task added raised the effective limit. One shared ElastiCache node, with TLS and reachable only from the api's security group, restored it.

The general rule: a sidecar suits state that belongs to one task. Anything that must be counted once across copies (rate limits, sessions, locks) needs a shared store. The fix shipped through a pull request together with two tasks per service, one per zone, and **autoscaling** from two to four api tasks on requests per target, capped so that four tasks times one worker times ten pooled connections stays well inside the small database's connection limit (two workers of five at first, until a review matched the task's half CPU to one worker).

Autoscaling takes minutes. Under a steady 80 requests a second, the scaling alarm fired five minutes after the load started and four tasks were running a minute later: about six minutes from load to capacity, made of three one-minute data points, the metric's delay and the task start. The k6 load test that drove it sent 33,601 requests with none failed and a 95th-percentile latency of 58 ms. Autoscaling follows trends, so the minimum count must carry a sudden spike on its own. Stopping one task by hand, as a host failure would, caused no failed requests: ECS started a replacement at once.

> HOW TO SAY IT
>
> *"The api runs as an ECS service on Fargate: two tasks across two zones, autoscaling to four, rolling deploys with the circuit breaker on, so a release that can't start rolls itself back."*
>
> *"ECS has no readiness-only signal: both its health checks replace tasks, so both check a shallow endpoint; readiness at start comes from the init containers, and outages show up in the 5xx alarm."*

Running a command in a task

ECS Exec opens a shell in a running container through Systems Manager, with no SSH and no open port. It is how an operator runs a one-off command on the service, such as the runbooks' "turn the assistant off" or "end this user's sessions". The shell gets the container's environment, not the environment of the process the start script launched. The api's start script builds `DATABASE_URL` from the separate secrets ECS injects, so in an exec shell the command-line tool found no database URL, and the runbooks had no path on AWS until the operator script built it the same way.

IN THE REPO `infra/aws/ops.sh` the operator's command line, in a task

`./ops.sh cli assistant --off --reason "..."` runs the same command as `make assistant` on a compose host, in one of the api's running tasks.

- It picks a running api task and opens ECS Exec in its api container.
- It builds `DATABASE_URL` from the injected parts, exactly as `container/api.sh` does.
- The arguments travel base64-encoded and NUL-separated, so a reason with spaces or quotes arrives as typed, with no quoting rules to get wrong.

```
cli)
  shift; (($#)) || { echo "usage: ops.sh cli <command> [args], e.g. cli assistant --off --reason '...'"
>&2; exit 2; }
  task=$(aws ecs list-tasks --cluster triage --service-name api --desired-status RUNNING --query
'taskArns[0]' --output text)
  [[ $task == arn:* ]] || { echo "no running api task" >&2; exit 1; }
  # container/api.sh composes DATABASE_URL before it starts gunicorn, so a shell opened with ECS
  # Exec has the parts but not the URL: compose it the same way. The arguments travel base64,
  # NUL-separated, so spaces and quotes in them (a reason) arrive as typed.
  args=$(printf '%s\0' "$@" | base64 -w0)
  aws ecs execute-command --cluster triage --task "$task" --container api --interactive --command \
    "sh -c 'export
DATABASE_URL=postgresql://\${DB_USER}:\${DB_PASSWORD}@\${DB_HOST}:\${DB_PORT}/\${DB_NAME}; echo $args |
base64 -d | xargs -0 python -m app.cli'" ;;
```

Traps

SYMPTOM	CAUSE	FIX
A command run through ECS Exec cannot find a setting the service uses	The start script built it for the process it started; the exec shell gets only the container's environment	Build it the same way in the operator's command.
A database outage makes ECS replace healthy api tasks	The load balancer checks a deep readiness endpoint, and ECS treats it as liveness	Check a shallow endpoint; detect outages with a 5xx alarm.
A task sits in PENDING for many minutes, billed	Init containers in a chain: one failed fast, the next never ran, the essential container waits	Make the essential container depend directly on every init container, each with a start timeout.
Rate limits are higher than configured after scaling out	Each task counts in its own sidecar store	A shared store for anything that must be counted once.

SYMPTOM	CAUSE	FIX
Two new tasks run migrations at the same moment	Every task migrates in an init container, and the deploy starts several at once	Make each run take a lock (a Postgres session advisory lock) so concurrent runs take turns, or run migrations as a one-off task before the deploy. Starting one new task at a time is not enough: the first deploy and a scale-out still start several.
The web container's proxy returns 502 for every api call on Fargate	It resolves the api through Docker's internal DNS, which Fargate does not have	Route `/api` at the load balancer. Service Connect alone does not help this nginx: it publishes names in `/etc/hosts`, which nginx's runtime resolver never reads.
Streams are cut during a deploy	The target group's deregistration delay (300 seconds by default; 30 here) is shorter than the longest answer: ECS sends SIGTERM only after it ends, and the load balancer drops what is still in flight	Set the deregistration delay above the longest answer, then the stop timeout (Fargate allows up to two minutes) above the server's graceful drain.
A short spike overwhelms the service before autoscaling reacts	Scaling takes several minutes from load to capacity	Size the minimum task count for spikes; scale on a leading metric.
An outage is reported that the load balancer never saw	The client's own network (DNS timeouts, lost packets) failed, not the service	Judge outages from server-side metrics, and run synthetic checks from inside the cloud.

Questions

1. What is the difference between a task definition and a service?
2. Why should the load balancer's health check on ECS be shallow?
3. What do `min_healthy_percent=100` and `max_healthy_percent=150` mean for a service of two tasks?
4. Why did scaling to two tasks double the login rate limit?
5. How long does autoscaling take to add capacity, and what does that imply?

Answers

1. A task definition is a versioned description of the containers to run. A service keeps a number of copies of one revision running, replaces failed ones, and rolls them to a new revision during a deploy.
2. ECS replaces any task the load balancer calls unhealthy. If the check depends on the database, a database outage makes every task "unhealthy", and ECS churns tasks that cannot help. A shallow check keeps the tasks; the outage is detected by the 5xx alarm instead. The same holds for the container health check in the task definition, which also ends in a replaced task.
3. Never fewer than two healthy tasks, and at most three running: one new task starts, becomes healthy, then one old task stops, and the cycle repeats.
4. Each task had its own Valkey container and counted attempts on its own, so each allowed the full limit. Shared limits need a shared store.
5. About six minutes in the reference build, from the start of the load to the new tasks running. The minimum task count must absorb a sudden spike without help.

10

PART V · RUNNING ON AWS

Kubernetes and EKS

IN ONE SENTENCE

Kubernetes is a portable orchestrator that keeps declared workloads running through a control loop, with a richer model than ECS (separate readiness and liveness, init containers, an ecosystem of controllers), at the price of a cluster someone must run and a standing cost.

AT A GLANCE

- You declare objects (a Deployment of three replicas of an image); controllers continuously reconcile reality toward them.
- The control plane (API server, etcd, scheduler, controllers) decides; the nodes run pods. Managed offerings such as EKS, GKE and AKS run the control plane for you.
- Readiness and liveness are separate probes: one controls traffic, the other restarts containers. On ECS both health checks replace the task; neither only withholds traffic.
- Every pod gets a routable IP and services get DNS names, so containers on different hosts find each other the way Compose services do on one host.
- Pick ECS for a few AWS-only services and a small team; pick Kubernetes for many services, a platform team, portability, or its ecosystem.

THEN AND NOW

THEN	NOW	WHY IT CHANGED
Run your own cluster manager and its database	→ **A managed control plane: EKS, GKE, AKS**	The control plane is the hardest part to operate well; providers run it.
Deploy with scripts that call the cluster	→ **GitOps: the cluster pulls its desired state from Git (Argo CD, Flux)**	Git becomes the audit log and the rollback button.
Fixed node groups sized by hand	→ **Node autoscaling that picks instance types per workload (Karpenter, EKS Auto Mode, GKE Autopilot)**	Capacity follows pods instead of guesses.
Ingress controllers configured per vendor	→ **The Gateway API, a standard routing model**	Portable routing across implementations.

The model: declare, then reconcile

Kubernetes inverts the deploy script. Instead of a sequence of commands, you submit objects that describe the end state: a **Deployment** that says "three replicas of this image", a **Service** that gives them a stable name, a **ConfigMap** and **Secret** for configuration. Controllers then run a loop forever: observe the cluster, compare it with the objects, act on the difference. A node dies, three replicas become two, and the loop starts a third elsewhere without anyone being paged.

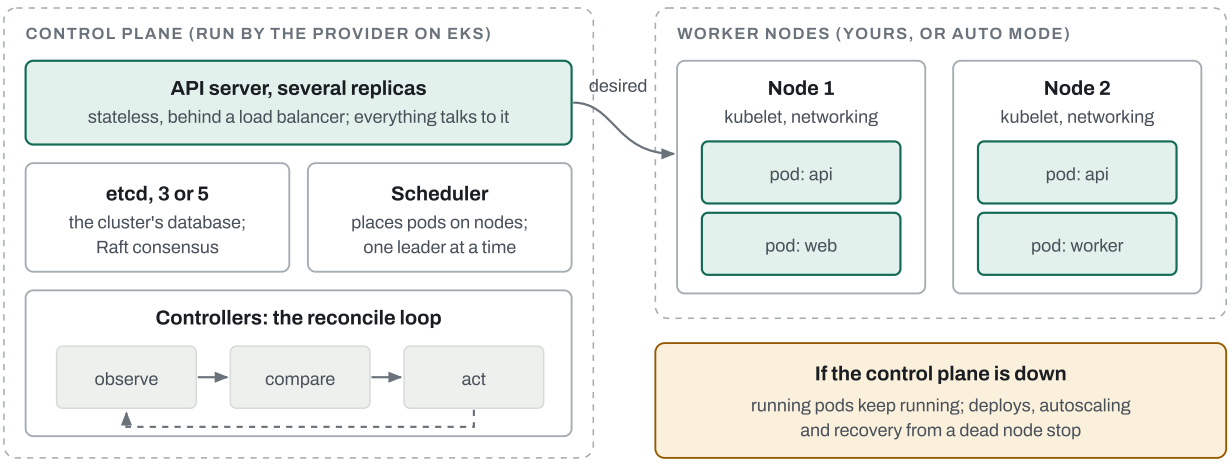


Figure 10.1 Kubernetes in one picture. etcd survives the loss of a minority of its members: one of three, or two of five. The API server is stateless and replicated; the scheduler and controllers elect a leader. On a managed service, all of the left half is the provider's job.

Two probes, two questions

The biggest practical difference from ECS for the reference service is health. Kubernetes asks two separate questions. A **readiness probe** decides whether a pod receives traffic: when it fails, the pod is removed from its Service's endpoints, but keeps running. A **liveness probe** decides whether the container is stuck: when it fails, the container is restarted. The reference api was written for exactly this split, with `/ready` and `/health`.

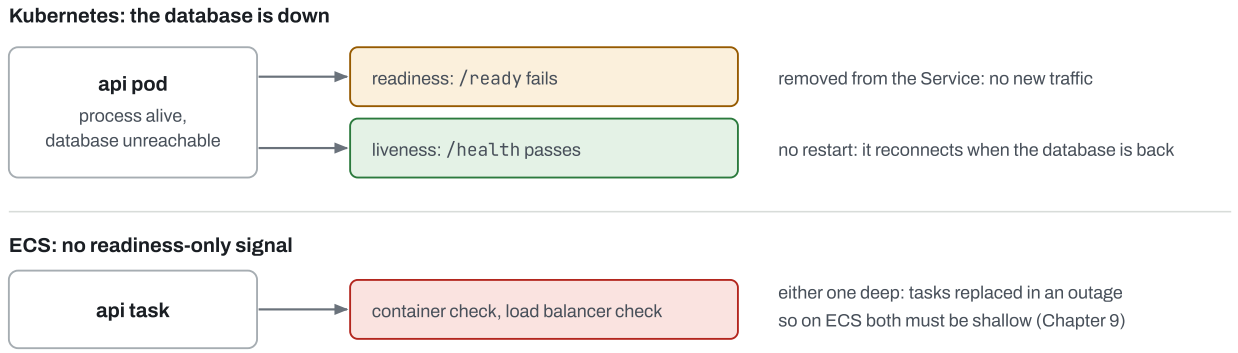


Figure 10.2 Readiness and liveness. Kubernetes lets an unready pod sit out of rotation without being restarted. ECS cannot express that: both its checks end in a replaced task, which is why the reference service's container check and load balancer check both use the shallow endpoint there.

IN THE REPO `apps/api/app/routes/health.py`

The two endpoints and the reasoning behind each. They fit Kubernetes probes unchanged.

- `/health` checks nothing external, so a database outage never restarts every replica at once.
- `/ready` fails only on hard dependencies. The cache and the identity provider are soft: they are reported as degraded but keep the instance ready, so a partial outage does not become a total one.
- Each check has its own short deadline, so the probe always answers on time.

```
"""Liveness vs readiness - two different questions, two endpoints.

/health (liveness): "is this process alive and its event loop turning?"
No dependency checks: if the database is down, restarting the api does
not help, and a liveness probe that checks it would restart every replica
at once. `async def` on purpose: it runs on the event loop, so it fails
exactly when the loop is blocked, and never waits for a worker thread.

/ready (readiness): "should traffic be sent here right now?" Checks hard
dependencies. Postgres is one; Redis is not - the rate limiter fails open,
so a Redis outage is reported as "degraded" but keeps the instance ready.
Nor is the identity provider: signed-in users carry on without it. Its
state comes from the background check (app/oidc.py), never a call per
probe. Marking every instance unready over a soft dependency turns a
partial outage into a total one.
"""
```

```python
@router.get("/health")
async def health() -> dict[str, str]:
    return {"status": "ok"}


@router.get("/ready")
async def ready(request: Request) -> JSONResponse:
    state = request.app.state
    database, redis = await asyncio.gather(
        probe(_ping_database(request), timeout_s=2.0),
        probe(state.redis.ping(), timeout_s=0.25),
    )
    idp = state.oidc.reachable
    body = {
        "status": "ready" if database else "not_ready",
        "checks": {
            "database": "ok" if database else "unavailable",
            "redis": "ok" if redis else "degraded",
            "identity_provider": "unknown" if idp is None else "ok" if idp else "degraded",
        },
    }
    return JSONResponse(body, status_code=200 if database else 503)
```

ECS and Kubernetes, side by side

CONCERN	ECS (BUILT IN THE REFERENCE)	KUBERNETES AND EKS
Unit	Task definition (containers sharing localhost)	Pod spec
Keep N running	Service	Deployment (with a ReplicaSet)
Init work	Non-essential containers with `dependsOn: SUCCESS` and a start timeout	`initContainers`, sequential, built in
Readiness and liveness	No readiness-only signal: the container check and the load balancer's check both replace the task	Separate readiness and liveness probes
Routing	Load balancer listener rules, URL rewrite	Ingress or the Gateway API; on AWS the Load Balancer Controller creates the ALB
Finding services	Load balancer, Service Connect, Cloud Map	Service and cluster DNS: `api:8010` resolves through the pod's resolver, as in Compose

CONCERN	ECS (BUILT IN THE REFERENCE)	KUBERNETES AND EKS
Configuration and secrets	Environment plus Secrets Manager injection	ConfigMap and Secret, plus External Secrets Operator or a CSI driver
AWS permissions	Task role	IRSA or EKS Pod Identity
Rollout safety	Min and max percent, circuit breaker with automatic rollback	`maxSurge`, `maxUnavailable`, a progress deadline; rollback by hand or with Argo Rollouts
Autoscaling	Application Auto Scaling on requests or CPU	Horizontal Pod Autoscaler for pods; Karpenter or Auto Mode for nodes
Deploy	`cdk deploy` from CI	`kubectl apply`, Helm, or Argo CD pulling from Git
Cost floor	None: pay per task	The control plane, about $0.10 an hour, before any node

Of the two difficulties the reference build met on ECS, Kubernetes handles one natively: the api's readiness and liveness split would have worked as designed. The other too, once its configuration stopped assuming Docker. The web image's nginx used to ask Docker's DNS address, `127.0.0.11`, which a pod does not have. It now writes its resolver at start from `/etc/resolv.conf`, and takes the api's address from a setting, `API_UPSTREAM`: on Kubernetes the service's full name, because nginx's resolver ignores search domains. The problem was in the image's configuration, not in ECS, and the same images could now run on EKS Auto Mode.

Why orchestrators exist at all

Container networking on one host is easy: Compose creates a private network and its service names resolve. Across hosts, that stops. You need IP connectivity between machines and an orchestration layer on top: Kubernetes gives every pod a routable IP through a CNI plugin, and gives services stable names. Placement, health, rescheduling after a node failure, and rolling updates across many hosts are the other jobs a single host never needed.

CHOOSE	WHEN
Compose on one host	One host is enough, and short maintenance windows are acceptable.
ECS, Cloud Run or Container Apps	A handful of services, a small team, one cloud. Less to operate and no control-plane bill.

CHOOSE	WHEN
Kubernetes (EKS, GKE, AKS)	Many services, a platform team to run it, portability across clouds, or a need for its ecosystem: operators, GitOps, a service mesh. Or the organization already runs it.

> **HOW TO SAY IT**
>
> *"For a handful of services ECS is less to run. Kubernetes earns its cost with many services and a platform team, or when the organization has standardized on it, and it would give us separate readiness and liveness for free."*

Traps

SYMPTOM	CAUSE	FIX
A database outage restarts every pod at once	The liveness probe checks the database	Liveness checks only the process; readiness checks hard dependencies.
A partial outage takes the whole service out of rotation	Readiness fails on a soft dependency, such as a cache	Report soft dependencies as degraded without failing readiness.
The team spends its time on the cluster, not the product	Kubernetes adopted before its benefits were needed	Use a managed container platform until many services or a platform team make the cluster pay.
A cluster costs money with nothing running	The managed control plane is billed per hour	Count it in the cost floor; share clusters across services.

Questions

1. What is the reconcile loop, and why does it make deploy scripts unnecessary?
2. What happens to running workloads if the control plane fails?
3. What is the difference between a readiness probe and a liveness probe?
4. When would you choose EKS over ECS?

Answers

1. Controllers repeatedly observe the cluster, compare it with the declared objects, and act on the difference. You change the declaration, and the controllers make it true, including after failures, so there is no script of steps to run.
2. Running pods keep running, because each node's kubelet carries on. Deploys, autoscaling and recovery from a dead node stop until the control plane is back.

3. Readiness decides whether a pod receives traffic; failing it removes the pod from rotation without restarting it. Liveness decides whether the container is stuck; failing it restarts the container.
4. When there are many services and a platform team, when portability across clouds matters, when the Kubernetes ecosystem (operators, GitOps, a mesh) is needed, or when the organization already runs Kubernetes.

PART V · RUNNING ON AWS

11 The same ideas on Azure and Google Cloud

IN ONE SENTENCE

Every major cloud offers the same building blocks under different names (a registry, a container platform, managed databases, a private network, workload identities and keyless CI), so the delivery path in this handbook carries over almost one to one, with a few differences worth knowing.

AT A GLANCE

- The concepts transfer: build once, promote, configuration from the environment, identities instead of keys, infrastructure as code.
- The names change, and so do a few defaults: Cloud Run and Container Apps give each service an HTTPS URL, immutable revisions, and scale to zero. Cloud Run splits traffic between revisions out of the box; Container Apps does once switched to multiple revision mode.
- Environments are separated by AWS accounts, Azure subscriptions and resource groups, or Google Cloud projects.
- Terraform works on all three with the same workflow, but resources are not portable: each cloud's resource is written for that cloud.

The map

The reference service was built on AWS. Everything in this table that is not AWS comes from the providers' documentation rather than from running it, but the mapping is close enough that a team moving clouds changes names, not designs.

CONCERN	AWS	AZURE	GOOGLE CLOUD
Image registry	ECR	Azure Container Registry	Artifact Registry
Containers, no servers	ECS on Fargate	Azure Container Apps	Cloud Run
Managed Kubernetes	EKS	AKS	GKE (Autopilot also runs the nodes)
Virtual machines	EC2	Virtual Machines	Compute Engine
Managed Postgres	RDS, Aurora	Azure Database for PostgreSQL, Flexible Server	Cloud SQL, AlloyDB

CONCERN	AWS	AZURE	GOOGLE CLOUD
Managed Redis or Valkey	ElastiCache	Azure Managed Redis	Memorystore
HTTP load balancer	Application Load Balancer	Application Gateway, Front Door; Container Apps has built-in ingress	Cloud Load Balancing; Cloud Run has a built-in HTTPS URL
Private network	VPC (regional), subnets, security groups	VNet (regional), subnets, network security groups	VPC (global), regional subnets, firewall rules
Secrets	Secrets Manager, Parameter Store	Key Vault	Secret Manager
A workload's identity	IAM role: task role, instance profile	Managed identity	Service account
CI without keys	IAM OIDC provider and a role	Entra ID federated credential	Workload Identity Federation
Environments apart	Accounts, in AWS Organizations	Subscriptions and resource groups	Projects
Native infrastructure as code	CloudFormation, with CDK on top	ARM templates, with Bicep on top	Infrastructure Manager, which runs Terraform
Logs, metrics, alarms	CloudWatch	Azure Monitor: Log Analytics, Application Insights	Cloud Logging, Cloud Monitoring
Budget alerts	AWS Budgets	Cost Management budgets	Billing budgets

Three differences worth knowing

Revisions and traffic splitting

On Cloud Run and Container Apps, every deploy of a new image or new container settings creates an immutable **revision**, and traffic can be split between revisions by percentage. On Container Apps that takes multiple revision mode: the default, single revision mode, moves all traffic to the new revision once it is ready and retires the old one. Container Apps also keeps secret values and ingress settings app-wide, outside revisions, so changing them creates no revision, and moving traffic cannot roll them back. With splitting on, a canary (5% to the new revision) or a rollback (100% back to the old one) is a

setting, not a redeploy. On ECS the equivalent is a rolling deployment with its circuit breaker, or the service's built-in blue/green, canary and linear strategies, which shift traffic between revisions behind a load balancer and roll back on an alarm. CodeDeploy is the older route to the same.

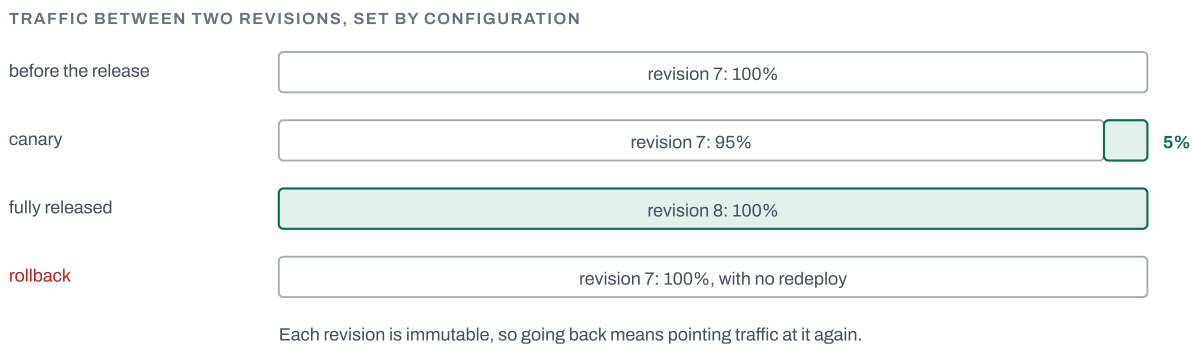


Figure 11.1 Revisions and traffic splitting on Cloud Run, and on Container Apps in multiple revision mode. A canary and a rollback are both changes of percentages, which makes them quick and cheap to rehearse.

HTTPS out of the box

Cloud Run and Container Apps give each service an HTTPS address with a managed certificate. On AWS, a load balancer needs a certificate from ACM and a domain you control. That is why the reference build's AWS stage ran over plain HTTP, with that exception named explicitly in the service's production checks.

Scale to zero

Cloud Run and Container Apps can run no instance at all while a service is idle, and start one on the next request, at the cost of a cold start. A Fargate service keeps its minimum number of tasks running, and billing, at all times. For an internal tool used a few times a day, scale to zero changes the cost picture; for an interactive service, the cold start usually rules it out.

Terraform across clouds

The language, the plan-and-apply workflow and the state file are the same on every cloud, through each cloud's provider. The resources are not: an ECS service has no Azure twin with the same fields, so it is rewritten as a Container App. Remote state with a lock lives in an S3 bucket, an Azure storage blob (with a lease) or a Google Cloud bucket, and CI authenticates to each through that cloud's OIDC federation. Chapter 7 shows the same registry declared on all three.

> **HOW TO SAY IT**
>
> *"The design is cloud-neutral: one image per version, configuration from the environment, workload identities, keyless CI, infrastructure as code. Moving clouds renames the pieces; Cloud Run and Container Apps would also give us revisions, HTTPS and scale to zero by default."*

Questions

1. How do Cloud Run and Container Apps make a rollback cheaper than a redeploy?

2. Why did the reference AWS build run over plain HTTP, when the same service on Cloud Run would not have?

3. What carries over between clouds in Terraform, and what does not?

Answers

1. Every deploy of a new image or container settings is an immutable revision, and traffic is split between revisions by configuration (on Container Apps, in multiple revision mode). A rollback sets 100% of traffic back on the previous revision, without building or deploying anything. Settings kept outside revisions, such as Container Apps' secrets and ingress, are not rolled back this way.
2. An AWS load balancer needs a certificate for a domain you control, and the build had no domain. Cloud Run and Container Apps give each service a managed HTTPS address by default.
3. The language, the plan-and-apply workflow and remote state carry over. The resource definitions do not: each cloud's resources are written for that cloud.

PART VI · THE REFERENCE BUILD

12 The reference build, end to end

IN ONE SENTENCE

How it all comes together: one change from issue to production, the infrastructure code that runs it on AWS, and what ten measured stages of building and breaking it taught.

AT A GLANCE

- A change travels one path, issue to production, and every gate on it is a file in the repository or a setting that cannot be skipped.
- Five CDK stacks and two workflows hold the whole deployment. Read them in the order a release travels, and they explain how a version tag becomes a running service.
- Every stage ended in evidence: a digest compared, requests counted during a deploy, a metric read from the load balancer.
- Most failures were configuration, and several were invisible until something was measured.
- A client-side probe reported an outage that never happened: judge availability from server-side metrics.
- The finished loop: one version tag, and the release is live on ECS 8 minutes 37 seconds later, with no failed requests.
- A last stage held the code to its own written rules and found eight gaps on the path to AWS; each was fixed and shown before and after. Testing the running system end to end then found four more.
- The first full build, VM and ECS stacks included, cost about sixty cents of cloud time, and each later session cents more, because everything billed by the hour was torn down after each session.

The stages

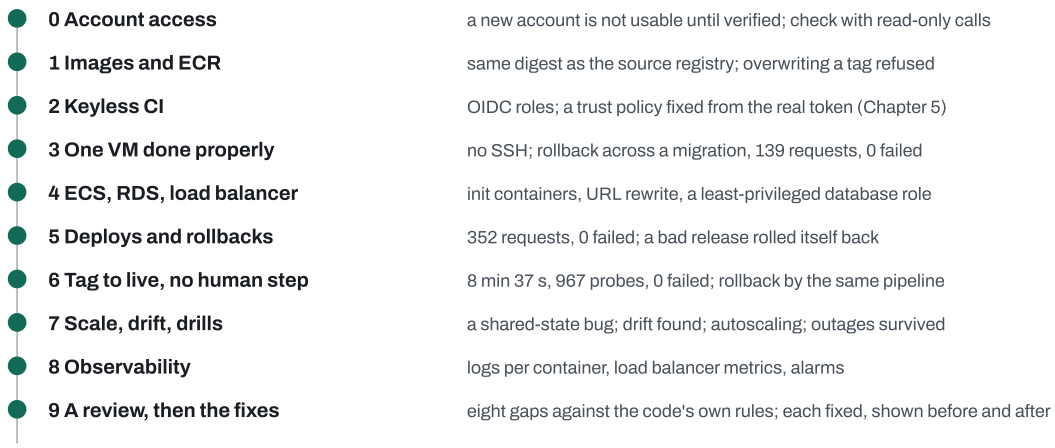


Figure 12.1 The reference build, stage by stage. Each stage built on the last and ended with a measurement. After the path and the code, the sections below tell the stories behind the stages that taught the most.

One change, from issue to production

Every change to the reference service takes the same path, and each step is held by a file in the repository or a setting on GitHub, not by anyone's memory. Chapters 2 to 5 explain the pieces one at a time. Here is the whole path in one place, with the file to read at each step. The timings are from a real release: the fixes in the version this handbook quotes took exactly this path.

STEP	WHAT HAPPENS	HELD BY
1 · Issue	The problem, and how to tell it is fixed. Every change starts from one.	.github/ISSUE_TEMPLATE: a bug form and a feature form
2 · Branch and commits	A branch named `type/<issue>-<short-desc>`, such as `feat/12-post-alerts`; commits in Conventional Commits form (`fix:`, `docs:`).	CONTRIBUTING.md
3 · Push	A pre-push hook runs lint, types and unit tests in containers, about 10 seconds, before anything leaves the machine.	scripts/git-hooks/pre-push, installed by `make hooks`
4 · Pull request	The template asks for `Closes #N`, what changed and why, how it was verified, and a checklist. The code owners file requests the owner's review.	pull_request_template.md, CODEOWNERS

STEP	WHAT HAPPENS	HELD BY
5 · CI	Six jobs: lint, the api tests on a real stack, the production image, the security scans, the web build and the browser tests. Five start at once; the image waits for lint. All six are required, on a branch up to date with main. About three minutes.	ci.yml; branch protection on main
6 · Merge	One approval from a code owner, then a squash merge: one commit per change on a linear main. `Closes #N` closes the issue. GitHub never lets authors approve their own pull requests, so a solo owner merges with the administrator bypass, deliberately (Chapter 2).	branch protection; the repository allows squash merges only
7 · Tag	`git tag -a vX.Y.Z` on main, then push the tag. Only that pattern starts a release; any other tag starts nothing. Only admins may create one, because a tag runs the workflow of the commit it points at.	the release workflow's trigger; a tag ruleset
8 · Release	Checks that the tag is on main, scans again, builds each image natively for two processor types, publishes them to GitHub's registry, smoke-tests and scans what it published, for each platform, then copies it to ECR through OIDC, and writes the release notes: the merged pull requests, after upgrade notes that list the settings added or removed since the last release. About four minutes.	release.yml
9 · Deploy	Starts when the release succeeds, waits for the reviewer the `aws-demo` environment requires, assumes the deploy role through OIDC, and runs `cdk deploy` with the new version: a rolling update with the circuit breaker armed, then a smoke test through the load balancer. About nine and a half minutes when the deploy created the stack from nothing; about four for an update.	deploy.yml, infra/aws
10 · Rollback	The same deploy, started by hand with the previous version. The infrastructure stays; the image changes.	the deploy workflow's manual trigger

Two properties make the path trustworthy. No step depends on someone remembering it: each gate is a file reviewed like code, or a setting only an administrator can override. And every step leaves evidence another person can read later: the issue, the pull request with its checks, the release with its notes, the deploy with its approval.

The infrastructure code, end to end

Everything AWS runs comes from one folder, `infra/aws`: a single CDK application of five stacks (Chapter 7). Three cost nothing or almost nothing and stay up between sessions: the image registry, the identity that lets CI reach AWS, and the network. Two are billed by the hour and are built for a session and destroyed after it: the single VM of Chapter 6, and the application stack, which holds everything a request touches. Read in the order a release travels, these files are a complete account of how a version tag becomes a running service.

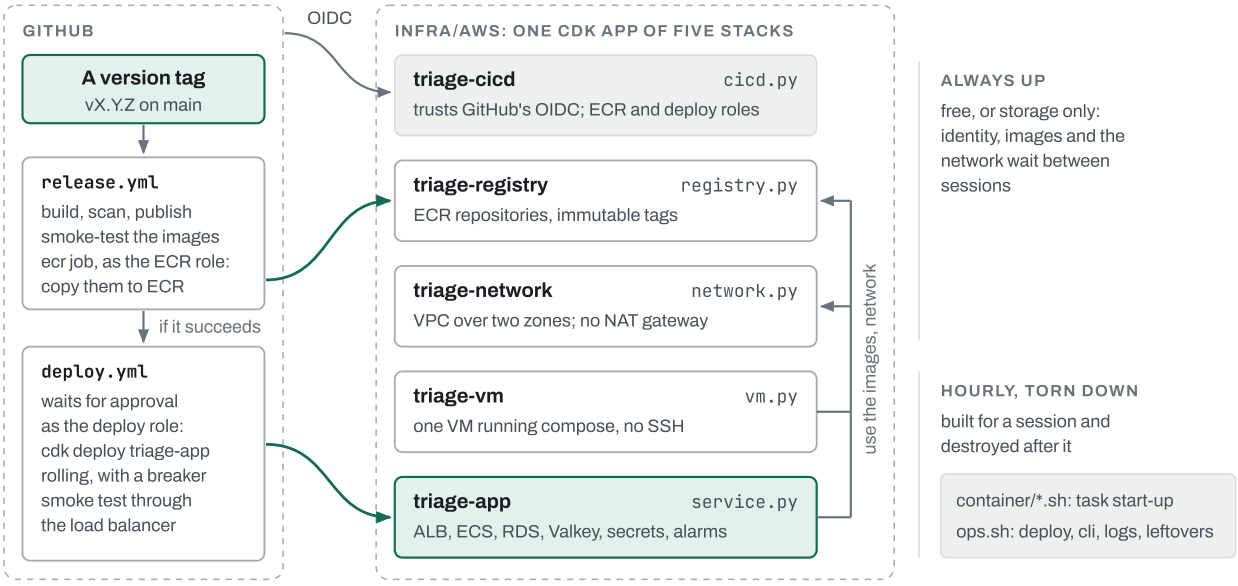


Figure 12.2 The deployment as code. Both workflows reach AWS with GitHub's OIDC tokens, as roles the identity stack holds: the release copies images into the registry, and the deploy updates the application stack alone. The VM and the application stack use the registry's images and the network, and the identity stack limits the ECR role to the registry's repositories.

IN THE REPO `infra/aws/README.md` and the registry stack, `infra/aws/registry.py`

The folder's README draws the path of a request in ten lines, from the load balancer to the containers of one task. The registry stack is the first thing a release touches.

- The load balancer routes `/api`, which nginx does in compose. nginx finds the api through Docker's DNS server, which Fargate does not have, so on ECS it only serves files.
- Image tags are immutable: once pushed, a tag always names the same bytes. The first stage tested it by trying to overwrite one.
- Lifecycle rules keep storage flat: untagged images expire after 7 days, and each repository keeps at most 20 images.

```
internet → ALB :80 -+- /api/metrics, /api/alerts/alertmanager → 404
                    +- /api/*  → rewritten to /* → api task (target health: /health)
                    +- else    → web task, nginx serving the built app (target health: /healthz)

api task (containers share localhost):
  dbinit, migrate   init containers: the app's least-privileged DB role, then migrations as owner
  mock-llm          sidecar
  api               starts after both init containers succeed and the sidecar is healthy
Valkey: one ElastiCache node shared by every api task (rate limits must be counted once).
2 tasks per service, one per AZ; the api scales to 4 on requests per target.
```

```python
for name in IMAGES:
    self.repos[name] = ecr.Repository(
        self,
        name,
        repository_name=f"triage-assistant-{name}",
        image_scan_on_push=True,
        # Tags are immutable: a tag, once pushed, always means the same bytes.
        image_tag_mutability=ecr.TagMutability.IMMUTABLE,
        # Throwaway account: `cdk destroy` must leave nothing behind.
        removal_policy=RemovalPolicy.DESTROY,
        empty_on_delete=True,
        lifecycle_rules=[
            ecr.LifecycleRule(
                description="expire untagged images after 7 days",
                tag_status=ecr.TagStatus.UNTAGGED,
                max_image_age=Duration.days(7),
            ),
            ecr.LifecycleRule(description="keep the last 20 images", max_image_count=20),
        ],
    )
```

IN THE REPO `infra/aws/service.py` who may talk to whom, and the database

The application stack starts with its boundaries. Each security group admits traffic from another group, never from an address range, so the rules follow the services wherever their tasks land.

- Only the load balancer's group is open to the internet, on port 80. The api and web tasks accept traffic from the load balancer alone. The database and the cache accept it from the api alone, and may not open connections of their own.
- The database lives in subnets with no route to the internet, encrypted, with credentials that Secrets Manager generated (Chapter 5).
- Each setting a production database would change carries a `production:` comment: Multi-AZ, deletion protection, longer backups, a final snapshot.

```
# --- Who may talk to whom: security groups reference each other, not IP ranges ---------
alb_sg = ec2.SecurityGroup(self, "AlbSg", vpc=vpc, description="internet to ALB port 80")
alb_sg.add_ingress_rule(ec2.Peer.any_ipv4(), ec2.Port.tcp(80), "HTTP")
api_sg = ec2.SecurityGroup(self, "ApiSg", vpc=vpc, description="ALB to api tasks port 8010")
api_sg.add_ingress_rule(alb_sg, ec2.Port.tcp(8010), "from the ALB only")
web_sg = ec2.SecurityGroup(self, "WebSg", vpc=vpc, description="ALB to web tasks port 8080")
web_sg.add_ingress_rule(alb_sg, ec2.Port.tcp(8080), "from the ALB only")
db_sg = ec2.SecurityGroup(
    self,
    "DbSg",
    vpc=vpc,
    description="api tasks to Postgres port 5432",
    allow_all_outbound=False,
)
db_sg.add_ingress_rule(api_sg, ec2.Port.tcp(5432), "from api tasks only")
cache_sg = ec2.SecurityGroup(
    self,
    "CacheSg",
    vpc=vpc,
    description="api tasks to Valkey port 6379",
    allow_all_outbound=False,
)
cache_sg.add_ingress_rule(api_sg, ec2.Port.tcp(6379), "from api tasks only")
```

```python
# --- Database: managed Postgres in subnets with no internet route ---------------------
db = rds.DatabaseInstance(
    self,
    "Db",
    instance_identifier="triage-db",
    engine=rds.DatabaseInstanceEngine.postgres(version=rds.PostgresEngineVersion.VER_17),
    instance_type=ec2.InstanceType.of(
        ec2.InstanceClass.BURSTABLE4_GRAVITON, ec2.InstanceSize.MICRO
    ),
    vpc=vpc,
    vpc_subnets=ec2.SubnetSelection(subnets=db_subnets),
    availability_zone="ap-south-1c",  # where RDS had capacity (network.py)
    security_groups=[db_sg],
    credentials=rds.Credentials.from_secret(owner),
    database_name=DB_NAME,
    allocated_storage=20,
    storage_type=rds.StorageType.GP2,
    storage_encrypted=True,
    multi_az=False,  # production: True (a standby in the other AZ, ~2x the price)
    backup_retention=Duration.days(1),  # production: 7-35 days, and a restore rehearsed
    delete_automated_backups=True,  # production: False
    deletion_protection=False,  # production: True
    removal_policy=RemovalPolicy.DESTROY,  # production: SNAPSHOT
)
```

IN THE REPO `infra/aws/service.py` the api container's settings, and its start script, `container/api.sh`

The api container's environment is where the load balancer and the application meet. Five of these values changed when a review held the code to its own rules (below).

- `ROOT_PATH` is `/api` because the load balancer rewrites `/api/ready` to `/ready` before the api sees it, as nginx does in compose.
- `FORWARDED_ALLOW_IPS` is the VPC's range. The load balancer appends the client's address to `X-Forwarded-For`, so only the addresses it added can be believed.
- One worker per half-CPU task, with a pool of ten connections: four tasks then hold at most forty database connections.
- The keep-alive outlasts the load balancer's 130-second idle timeout, and the graceful shutdown ends inside the 120-second stop timeout (Figure 12.3).
- ECS injects each secret as its own variable; `api.sh` builds the database URL from them before it starts gunicorn.

```
environment={
    **db_env,
    "APP_ENV": "prod",
    "APP_VERSION": tag,
    "ROOT_PATH": "/api",  # the ALB strips /api, as nginx does in compose
    # Whose X-Forwarded-For to believe: the ALB's addresses, i.e. the VPC. Not "*":
    # an ALB appends the client to what the client sent, so with "*" the left-most
    # entry, which the client wrote, became its address (the sign-in rate limit's key).
    "FORWARDED_ALLOW_IPS": VPC_CIDR,
    # One worker per usable CPU, rounded up (gunicorn.conf.py): a 0.5 vCPU task runs
    # one, and capacity comes from tasks. The pool keeps the connection budget.
    "WEB_CONCURRENCY": "1",
    "DB_POOL_SIZE": "10",  # x 1 worker x 4 tasks: inside a db.t4g.micro's ~80
    # Idle connections: the ALB keeps them 130 s (idle_timeout), so the api must keep
    # them longer, or the ALB reuses one the api has just closed and answers 502.
    "GUNICORN_KEEPALIVE": "140",
    # In-flight requests get 110 s after SIGTERM, inside the 120 s stop timeout.
    "GUNICORN_GRACEFUL_TIMEOUT": "110",
    "REDIS_URL": redis_url,
```

```
# The api reads one DATABASE_URL; ECS injects the secret as separate values, so compose
# it here. The generated passwords have no punctuation, so they need no URL-encoding.
export DATABASE_URL="postgresql://${DB_USER}:${DB_PASSWORD}@${DB_HOST}:${DB_PORT}/${DB_NAME}"
exec gunicorn app.asgi:app -c gunicorn.conf.py
```

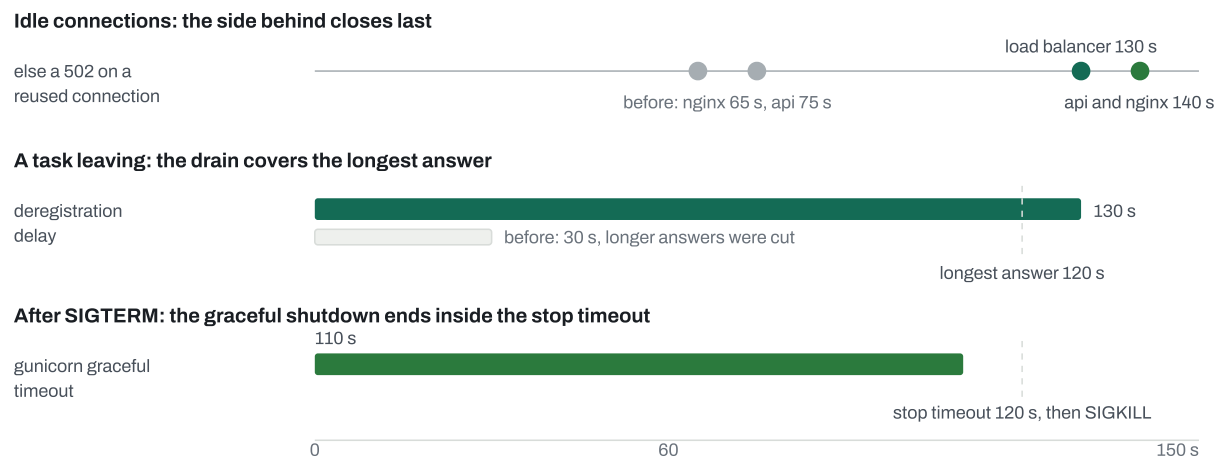


Figure 12.3 Three orderings the AWS stack keeps, drawn to scale. Break the first and the load balancer answers 502 on a connection the api has just closed. Break the second and deploys cut long answers: with a 30-second delay, three of four ninety-second answers ended mid-stream when their task stopped; with 130 seconds, none did. Break the third and ECS kills the process before its requests finish.

IN THE REPO `infra/aws/service.py` routing at the load balancer

Two listener rules and a default carry the routing the edge does in compose, checked in priority order.

- Priority 5 answers 404 for the metrics and alert-webhook paths, so the internet never reaches them. Where Prometheus and Alertmanager run, in the compose stacks, they call the api from inside the network.
- Priority 10 sends `/api/*` to the api's target group, and a URL rewrite strips the prefix.
- Everything else falls through to the web service.
- The rewrite is newer than the CDK construct for listener rules, so it is set on the CloudFormation resource directly: an escape hatch, used when a construct lags the service.

```
elbv2.ApplicationListenerRule(
    self,
    "Internal",
    listener=listener,
    priority=5,
    conditions=[
        elbv2.ListenerCondition.path_patterns(["/api/metrics", "/api/alerts/alertmanager"])
    ],
    action=elbv2.ListenerAction.fixed_response(
        404, content_type="application/json", message_body='{"error":"not_found"}'
    ),
)
api_rule = elbv2.ApplicationListenerRule(
    self,
    "Api",
    listener=listener,
    priority=10,
    conditions=[elbv2.ListenerCondition.path_patterns(["/api/*"])],
    action=elbv2.ListenerAction.forward([api_tg]),
)
# URL rewrite (ALB, 2025): /api/ready → /ready. Not in the CDK L2 yet, so set it on the
# CloudFormation resource directly (an "escape hatch").
api_rule.node.default_child.add_property_override(
    "Transforms",
    [
        {
            "Type": "url-rewrite",
            "UrlRewriteConfig": {"Rewrites": [{"Regex": "^/api/(.*)$", "Replace": "/$1"}]},
        }
    ],
)
```

IN THE REPO `infra/aws/service.py` scaling and alarms

The service grows with its load and reports when it is unwell.

- Autoscaling runs two to four api tasks, adding one while each serves over 1,000 requests a minute. The ceiling comes from the database's connection limit, not from the load.
- Scale-out waits one minute between steps and scale-in five, so the count does not flap.
- The two alarms watch what users feel: unhealthy targets and 5xx responses. Neither notifies anyone yet; in production they page the on-call through SNS.

```
# --- autoscaling: more api tasks while each serves over 1,000 requests a minute --------
# Bounded by the database: 4 tasks x 1 worker x DB_POOL_SIZE 10 = 40 connections, inside
# a db.t4g.micro's ~80. Scale-in waits longer than scale-out, so it does not flap.
scaling = api_svc.auto_scale_task_count(min_capacity=2, max_capacity=4)
scaling.scale_on_request_count(
    "Requests",
    requests_per_target=1000,
    target_group=api_tg,
    scale_out_cooldown=Duration.seconds(60),
    scale_in_cooldown=Duration.seconds(300),
)

# --- alarms (the first 10 are free); production: an SNS topic to the on-call ------------
cw.Alarm(
    self,
    "ApiUnhealthy",
    alarm_name="triage-api-unhealthy-targets",
    metric=api_tg.metrics.unhealthy_host_count(period=Duration.minutes(1)),
    threshold=0,
    comparison_operator=cw.ComparisonOperator.GREATER_THAN_THRESHOLD,
    evaluation_periods=2,
    treat_missing_data=cw.TreatMissingData.NOT_BREACHING,
)
cw.Alarm(
    self,
    "Api5xx",
    alarm_name="triage-api-5xx",
    metric=api_tg.metrics.http_code_target(
        elbv2.HttpCodeTarget.TARGET_5XX_COUNT, period=Duration.minutes(1)
    ),
    threshold=5,
    evaluation_periods=1,
    treat_missing_data=cw.TreatMissingData.NOT_BREACHING,
)
```

IN THE REPO `.github/workflows/deploy.yml` the last mile: a release onto the stack

The deploy workflow (Chapter 5 shows how it gets its credentials) ends in one command: `cdk deploy triage-app` with the release's version.

- `--exclusively` deploys the application stack alone. The other stacks are deployed once, from a workstation, and rarely change.
- CloudFormation waits until ECS reports the service stable. A release whose tasks keep failing is rolled back by the circuit breaker, and the step fails.
- The smoke test calls the readiness endpoint through the load balancer, the path a user's request takes.
- The infrastructure code comes from main and the image from the release, so rolling the application back never rolls the infrastructure back.

```
78  # Rolling deploy; CloudFormation waits until ECS is stable. A release whose tasks keep
79  # failing is rolled back by the ECS deployment circuit breaker, and this step fails.
80  - name: cdk deploy triage-app
81    working-directory: infra/aws
82    run: |
83      CDK_DEFAULT_ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
84      export CDK_DEFAULT_ACCOUNT
85      uv sync --frozen
86      npx --yes aws-cdk@2.1143.0 deploy triage-app --exclusively --require-approval never \
87        -c tag="$TAG" --outputs-file outputs.json
88  - id: smoke
89    name: Smoke test through the load balancer
90    working-directory: infra/aws
91    run: |
92      url=$(jq -r '."triage-app".Url' outputs.json)
93      echo "url=$url" >> "$GITHUB_OUTPUT"
94      curl -fsS --retry 5 --retry-delay 5 --retry-all-errors "$url/api/ready"
```

From demonstration to production

The stack is sized for a throwaway account. Every place production differs is marked in the code with a `production:` comment or listed in the folder's README, so this list is the code's own:

AREA	IN THE REFERENCE BUILD	IN PRODUCTION
Entry	HTTP on port 80, no certificate	HTTPS with an ACM certificate, port 80 redirecting to it, and AWS WAF in front
Sign-in and model	no identity provider, reported as degraded; the mock model and insecure cookies, both waived by name in the production checks	the organization's OIDC provider, a model provider and HTTPS, with nothing waived

AREA	IN THE REFERENCE BUILD	IN PRODUCTION
Database	one zone, one day of backups, deleted with the stack	Multi-AZ, deletion protection, 7 to 35 days of backups and a rehearsed restore, a final snapshot
Cache	one node, no AUTH user	a replica in the other zone with automatic failover, and an AUTH user
Network	tasks in public subnets with public addresses, no NAT gateway	private subnets, with a NAT gateway or VPC endpoints for ECR and logs
Containers	a writable root filesystem; ECS Exec on	a read-only root with a writable `/tmp`; ECS Exec off, or limited to the on-call role and logged
Operations	logs kept 3 days; alarms notify nobody; secrets deleted with the stack	log retention set by what incidents need; alarms page the on-call; secrets retained

Images: the same bytes in a second place

The release images already existed in GitHub's registry, built for two processor types. They were copied into ECR as whole multi-architecture indexes, and the index digest in ECR matched the original exactly. Then a deliberate test: pushing a different image over an existing version tag. ECR refused, because the repositories have immutable tags, and the digest behind the tag was unchanged. A tag in that registry always names the same bytes.

One surprise: the registry's scan-on-push never ran on the copied images. It fires for ordinary pushes, and the copy took another path. The lesson is general: scanning belongs in the pipeline, before publishing, not only in the registry (Chapter 4).

The first ECS deploy: two failures, both configuration

- `No export named … found.` The new application stack referenced the image repositories, which needed new exports from the registry stack, and the deploy had been limited to the new stack alone. Cross-stack references are ordered dependencies: deploy the producer first (Chapter 7).
- **"No subnets exist in Availability Zones with sufficient capacity."** The database instance type was listed as available in all three zones, yet two had no capacity at that moment. Capacity is per zone and temporary. An extra isolated subnet in the third zone fixed it without touching anything running (Chapter 8).

After those, the containers started in the designed order (setup, migrations, sidecars, then the api), the readiness endpoint reported the database and cache healthy and the identity provider degraded (there was none in this stage), and internal paths returned 404 at the load balancer as intended.

An outage that never happened

During the bad-release drill, a probe on a developer's machine sent a request twice a second and reported 15 failures. The timeline did not fit: failures began after the bad task was already gone and continued after the rollback. Splitting curl's timings showed why.

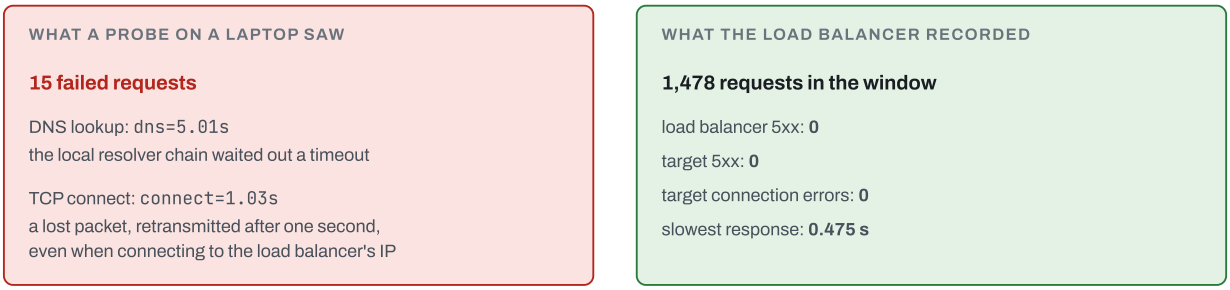


Figure 12.4 The same window, from two sides. The failures were in the client's own network path, not the service. Availability is judged from server-side metrics; real teams add synthetic checks that run from inside the cloud.

The whole loop, with no human step

With the deploy workflow in place, the only human action was pushing a version tag (the environment has since gained a required reviewer, so a release now also waits for one approval). Everything after it ran on its own: the release built, scanned and published the images, then smoke-tested them while copying them to ECR; the deploy workflow then ran in the gated environment, applied the infrastructure code with the new version, waited for ECS to be stable, and smoke-tested through the load balancer.

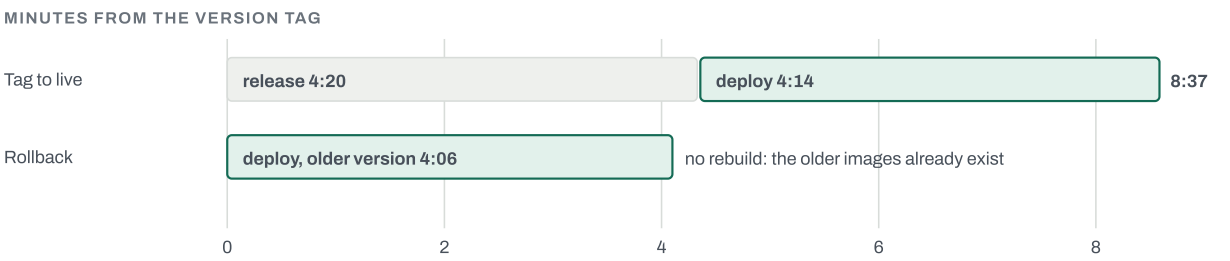


Figure 12.5 Tag to live in 8 minutes 37 seconds, with 967 probe requests during the deploy and none failed. A rollback is only the deploy half, with an older version: 4 minutes 6 seconds. Over both, the load balancer counted 1,311 requests with no errors.

Three design choices made this loop safe to leave unattended. The infrastructure code always comes from main while the image comes from the release, so rolling the application back never rolls the infrastructure back. Deploys share a concurrency group without cancellation, so two deploys queue

rather than race. And workflow inputs reach shell commands only through environment variables, never interpolated into the script text, which closes a script-injection route.

Outage drills

DRILL	WHAT USERS SAW	WHAT THE PLATFORM DID
Stop one api task, as a lost host would	182 probes, none failed	ECS started a replacement at once; back to full strength in under a minute
Reboot the database (single zone)	Readiness returned 503 for about 8 seconds; the pages and the liveness endpoint stayed up	Nothing replaced: shorter than the load balancer's two failed checks
Stop the database for about 7 minutes	Readiness 503 throughout; zero non-200 answers from the pages and the liveness endpoint	With a shallow health check, nothing; the api reconnected 4 seconds after the database returned
Deploy during the outage	Nothing: the old tasks kept serving	The new task's setup container hit its start timeout; ECS reported "task failed to start" and stopped it

The first database-stop drill, run with the deep health check, is the one that changed the design: ECS replaced api tasks whose replacements could not start, and one replacement sat in PENDING for 13 minutes. Chapter 9 explains the health-check rule and the dependency fix that came out of it.

What the on-call looks at

Every container's output went to CloudWatch Logs, one stream per container, and CloudWatch Logs Insights queries them with a SQL-like language. Over the last four hours of the session:

```
filter @logStream like /^api\// | parse @message "\"level\": \"*\"" as level
| stats count(*) as n by level | sort n desc
```

LEVEL	LINES	WHAT THEY WERE
info	47,723	ordinary request logs
warning	1,501	1,453 of them "identity provider unavailable": expected, because there was none
error	15	all from the deliberate bad-release drill

Those 1,453 expected warnings are how alert fatigue starts. Alert on symptoms users feel (5xx responses, latency, unhealthy targets), not on log warnings. The load balancer is the source of truth for

users, and its totals for the session were: 42,434 requests, no load balancer errors, no connection errors to targets, 281 server errors from the api, every one of them during the two deliberate database outages (honest 503s from readiness), and 107 client errors, which were the rate-limit tests' 429s.

Not built, but worth knowing: Container Insights for per-task CPU and memory, tracing through X-Ray or any OpenTelemetry backend (the api already exports OTLP), and CloudWatch Synthetics, which runs probes from inside AWS and would have avoided the false outage above.

A review against its own rules

The last stage read the whole service against what its own comments and chapters say. Eight gaps were on the path to AWS. Each was reproduced first, then fixed, then shown: on AWS where it could be, otherwise locally.

THE RULE THE CODE BROKE	BEFORE	AFTER
Trust only the proxies in front	The api trusted every proxy (`*`), and the load balancer appends to `X-Forwarded-For`: a client could choose its own address, and so its own sign-in rate-limit bucket	36 sign-ins, each with a forged address: 30 allowed, 6 refused, and the real address logged for all 36
The drain covers the longest request	A 30-second deregistration delay: three of four ninety-second answers cut when their task stopped	130 seconds: four of four completed
The side behind closes idle connections last	The api closed them at 75 seconds and nginx at 65, behind a load balancer that keeps them 130	140 seconds for both
Migrations never collide	Two tasks starting together ran the same migrations: one of the two failed, three times in three	A database lock: on the first deploy, one task ran all nine migrations, the other none, and both succeeded
One worker per CPU	Two workers in a half-CPU task, against the server's own configuration file	One worker per task; capacity comes from tasks
Public logs name no account	The role ARNs were variables, printed with every step's inputs: the account ID in 51 log lines per release and 3 to 5 per deploy	Secrets, masked: none in either
A person approves every deploy	The deploy workflow's comment said so; the environment had no reviewer, and deploys ran unattended	The deploy waited for approval

THE RULE THE CODE BROKE	BEFORE	AFTER
The security scan blocks merges	The CI file's comment said so; the scan was not a required check	Required: six checks in all

None of these failed a test or raised an alarm. Each was a rule written down, in a comment or a chapter, that the code did not keep. A team can run the same review: read the comments and documents as claims, and test them like code.

A second pass: the running system

Reading finds what the code says; running it finds what it does. The release was then tested end to end, locally and on a fresh stack: sign-in under attack (a replayed callback, a forged state, another provider, a cross-site sign-out, a reused cookie), team isolation, the rate limit with forged addresses, the load balancer's path rules against encoded and dotted paths, and every failure drill. All of that held. Four things did not:

AREA	WHAT WAS WRONG	FIXED BY
Operator commands on ECS	The runbooks' commands (turn the assistant off, end a user's sessions) failed on ECS. The start script builds the database URL for the server it starts, so a shell opened with ECS Exec had the parts but not the URL.	`ops.sh cli` runs them in a running task and builds the URL the same way (Chapter 9).
Release tags	Unprotected. A tag on any branch runs that branch's release workflow, and the cloud registry's role trusts every `v*` tag.	A tag ruleset: only admins create, move or delete release tags (Chapter 5).
Upgrades	An upgrade stopped at start-up. An earlier release had added production checks that an older host's settings failed, and the release notes listed only pull request titles.	Release notes open with the settings added or removed since the last release (Chapter 13).
Deploy output	Every deploy printed a warning that was false, which teaches people to skip warnings.	Acknowledged in the code, with the reason.

Two anomalies appeared once each, while faults overlapped, and not again in a dozen targeted runs. They were recorded as an open issue, not "fixed" by a guess, and one was then traced. The database pool had stayed full after a network cut, with the database healthy. The cause was the api's address: Docker usually gives a container another address when it rejoins a network, and every pooled connection stayed bound to the old one. Data sent on them was never acknowledged, so each would hang for about 15 minutes, holding its slot. Reproduced on demand, it was fixed with a socket option, `TCP_USER_TIMEOUT`, that drops a connection whose data goes unacknowledged for 10 seconds; a new drill fills the pool and changes the address (Handbook 2, Chapter 15). The other, an HTTP 500 during a

worker kill, never came back; the drills now keep both containers' logs, so it can be traced if it does. An anomaly that will not reproduce usually means a variable nobody has changed yet.

> **HOW TO SAY IT**
>
> *"Every change takes one path, from issue to production, and every gate on it is code or a setting nobody can skip. We proved each property with traffic flowing: zero failed requests across deploys, an automatic rollback of a bad release, a lost task and a database outage absorbed. And every way this demo differs from production is marked in the code."*

Traps

SYMPTOM	CAUSE	FIX
Most services answer "not signed up" on a brand-new cloud account	Activation (payment and phone verification) has not finished propagating	Finish verification, then check access with read-only calls before building.
A monitoring probe reports failures the service never had	The probe's own DNS or network path	Split the client's timings; compare with load balancer metrics; probe from inside the cloud.
A probe killed mid-run prints no summary	It was stopped with a signal it did not handle	Trap the termination signal and print the counts.
A client chose its own rate-limit bucket	The api trusted every proxy, and the load balancer appends to the forwarded header	Trust only the load balancer's addresses: the VPC range.
Long answers cut during every deploy	The deregistration delay was shorter than the longest request	Set it to the longest request; ECS sends SIGTERM only after it ends.
An alert channel full of expected warnings	Alerting on log levels rather than user-facing symptoms	Alert on 5xx, latency and unhealthy targets.

Questions

1. Trace a bug fix from issue to production. Which file or setting holds each gate?
2. On ECS, why does the load balancer route `/api` rather than nginx?
3. The load balancer keeps idle connections 130 seconds, and the longest answer takes 120. What must the api's keep-alive, the deregistration delay and gunicorn's graceful timeout be, and why?

4. A synthetic check from your laptop reports an outage. What do you look at before declaring an incident?

5. Why does a rollback through the pipeline take about half as long as a release?

6. Why does the deploy workflow take its infrastructure code from main rather than from the release being deployed?

Answers

1. The issue form; `CONTRIBUTING.md` for the branch and commit names; the pre-push hook for lint, types and unit tests; the pull request template and the code owners file; `ci.yml`, with branch protection requiring all six jobs on an up-to-date branch; the squash-only merge setting; the tag pattern that starts `release.yml`, and the ruleset that lets only admins create such a tag; `release.yml` for scanning, building, smoke-testing and copying to ECR; `deploy.yml`, gated by the `aws-demo` environment's reviewer, for `cdk deploy` and the smoke test. A rollback is `deploy.yml` run by hand with the previous version.
2. In compose, nginx finds the api by its service name; this ECS stack gives the api no such name (no service discovery). So on ECS nginx only serves files, and the load balancer does the edge's routing: a rule sends `/api/*` to the api's target group and rewrites the path, and another answers 404 for the internal paths.
3. Keep-alive above 130 seconds (140), so the load balancer never reuses a connection the api has just closed. A deregistration delay of at least 120 seconds (130), because the load balancer drops whatever is in flight when the delay ends, before ECS sends SIGTERM. A graceful timeout below the 120-second stop timeout (110), or ECS kills the process before its requests finish.
4. The server-side metrics for the same window, starting with the load balancer's request count and error counts, and the client's own timings: a slow DNS lookup or connect points at the client's network, not the service.
5. It runs only the deploy half. The older images were built, scanned and published when that version was released, so nothing is rebuilt.
6. So that deploying an older application version never rolls the infrastructure back. The application is versioned by its images; the infrastructure moves forward through its own pull requests.

PART VII · OPERATING IT

13 Environments, releases and rollback

IN ONE SENTENCE

Releasing safely means choosing how traffic moves to a new version, deciding in advance how each kind of change is undone, and reaching real users in stages that each end in something you can measure.

AT A GLANCE

- Four ways to move traffic: rolling, canary, blue/green, and feature flags. They protect against different failures.
- Every kind of change has its own rollback, chosen before shipping: code, schema, configuration, a feature, infrastructure.
- Rollbacks roll back code, never the schema. Migrations are written expand-then-contract so the previous release runs on the newer schema.
- A release that contracts what an earlier one expanded cannot be skipped over; release notes name the minimum version they can follow.
- Reaching users is a staged road (a real host, real sign-in, a real model, a pilot team, then everyone), and each stage has a "done when".

THEN AND NOW

THEN	NOW	WHY IT CHANGED
A release night, the whole system switched at once	→ **Rolling or canary releases during working hours**	Problems surface on a fraction of traffic, while people are awake to see them.
Rollback means restoring last night's backup	→ **Rollback means deploying the previous version; data stays**	Restoring loses every write since the backup.
Schema changes need downtime	→ **Expand, migrate, contract, across several releases**	The old and new code run side by side during every rolling deploy.
A feature ships when its code ships	→ **Code ships dark; a flag turns the feature on per tenant or percentage**	Turning a feature off is faster than any redeploy.
Going live is one big step	→ **Staged exposure, each stage with an exit criterion**	Each stage tests one new kind of risk.

Four ways to move traffic

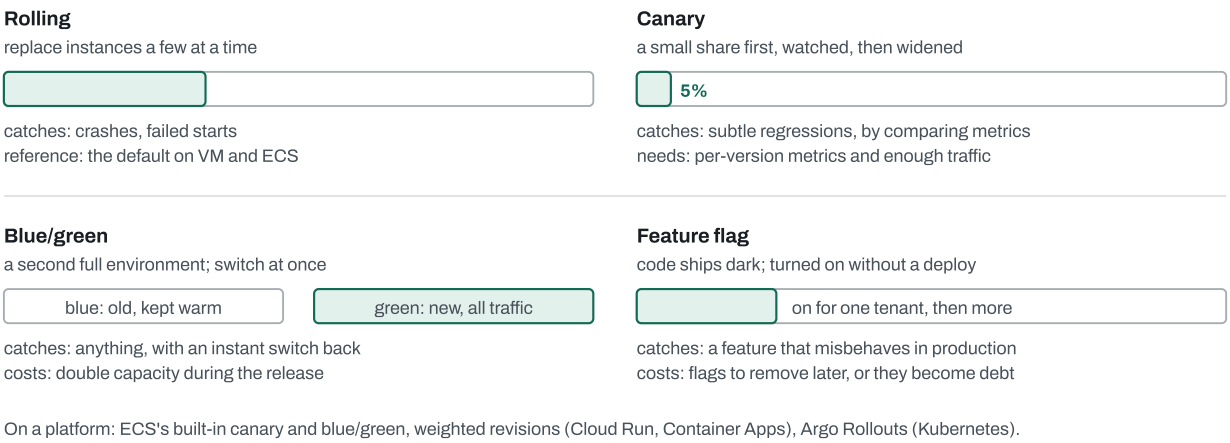


Figure 13.1 Release strategies, and what each protects against. A rolling deploy is safe against crashes because readiness gates it, but it moves all traffic within seconds, so it cannot catch a regression that only shows in the numbers. That is the canary's job.

The reference service uses rolling deploys everywhere, and for subtle regressions it currently relies on "deploy, watch the dashboard for half an hour, roll back if needed", which works because rollbacks are cheap and cross migrations. The next steps, in order of effort, are a canary on one host (a second api service and a weighted upstream in nginx), then the platform's own traffic splitting (on ECS, its built-in canary strategy) with automatic analysis against the service's objectives.

Rollbacks roll back code, never the schema

During a rolling deploy, the old and new versions run at the same time against the same database. So a schema change cannot break either of them, and it is done in steps that each leave both versions working. This is the **expand and contract** pattern.

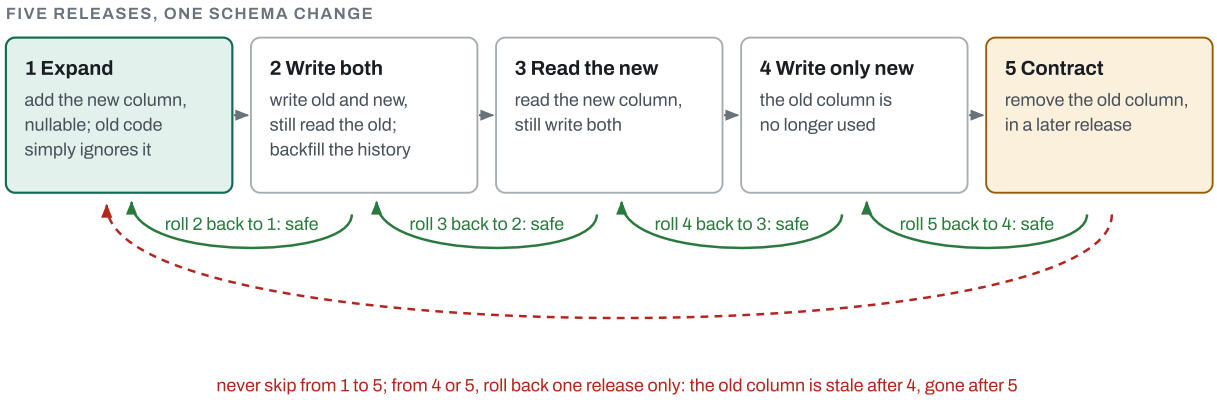


Figure 13.2 Expand and contract. Each release can run on the next release's schema, because it will, on the day it is rolled back to. Writes to the old column stop only after reads have moved to the new one, so a rollback to the release before never reads a stale column. Never rename in place: a rename is an expand, a backfill and a contract.

The rule produced one of the reference service's decision records, after a rollback failed. The procedure had been "deploy the previous tag", and the previous tag's migration tool had never heard of the revision the newer release applied. The deploy stopped before touching anything, so it failed safely, but rollback was impossible across any migration: exactly when a rollback is most likely needed.

ADR-0015 ACCEPTED

Rollbacks roll back the code, never the schema

PROBLEM	Rolling a release back to one with fewer migrations failed: the older migration tool could not find the database's current revision.
DECISION	A deploy never reverses a migration. When the database is ahead of the image, it skips migrating, says so, and rolls out the older code on the newer schema. Reversing a migration is a separate, deliberate step, decided by a person, run with the newer image.
REJECTED	Automatic downgrades: the least-tested code in any project, able to lose data, and run under pressure.
SEE IT IN	infra/aws/container/migrate.sh on ECS, and scripts/deploy.sh on a VM.

IN THE REPO `infra/aws/container/migrate.sh` the decision, carried out on ECS

The migration init container on ECS, in twelve lines. It asks the migration tool for the database's current revision first.

- If the tool cannot locate that revision, the database is ahead of this release: it prints so and exits successfully, and the older code starts on the newer schema.
- Any other failure, such as a wrong password or an unreachable host, still fails the task.
- Otherwise it migrates forward to the newest revision this release knows.

Measured on ECS and on the VM: rollbacks across a migration, under steady traffic, with no failed requests.

```
# Schema migrations as the owner, before the api starts (an init container).
# On a rollback the database can be ahead of this release: alembic then cannot find the
# current revision, and the right move is a code-only rollback, not a failed deploy.
# Any other error (wrong password, unreachable host) still fails the task.
export
MIGRATIONS_DATABASE_URL="postgresql://${DB_OWNER_USER}:${DB_OWNER_PASSWORD}@${DB_HOST}:${DB_PORT}/${DB_
NAME}"
if ! out=$(alembic current 2>&1); then
  if printf '%s' "$out" | grep -q "Can't locate revision"; then
    echo "the database is ahead of this release: code-only rollback, no migration"; exit 0
  fi
  printf '%s\n' "$out" >&2; exit 1
fi
exec alembic upgrade head
```

One rollback per kind of change

Decide, before shipping, how each kind of change will be undone, and practice it once.

KIND OF CHANGE	ROLLBACK
Application code	Deploy the previous version through the same pipeline.
Database schema	Do not. The previous version runs on the new schema; fix forward. A downgrade is a separate, deliberate decision.
Configuration	Restore the previous value, through the same reviewed path that changed it.
A new feature	Turn its flag off.
Infrastructure	Revert the pull request and apply; or fix forward. Rolling back the application never rolls back the infrastructure.

KIND OF CHANGE	ROLLBACK
A prompt or model (AI features)	Switch the version back in configuration (Handbook 1, Chapter 12).

Two more rules keep rollbacks honest. Releases are never skipped when one contracts what another expanded: the reference service's row-level-security release hides every row from a release two versions older, which never says who is asking, so a host must pass through the release in between. And a breaking API change is announced in the release notes, never discovered by a client. The same holds for settings. One release added production checks that stopped an older host's api until its settings changed, and its notes, a list of pull request titles, did not say so. The reference release notes now open with the settings added to or removed from `.env.example` since the previous release.

Handing a service to an infrastructure team

A service that can be released safely also has to be operable by someone else. These are the questions an infrastructure team asks, with the reference service's answers. Every answer points to something already in the repository.

THEY ASK	THE REFERENCE SERVICE'S ANSWER
What runs?	Three images (api, web, an HTTPS edge), one migration command, Postgres with pgvector, Valkey, a model provider, an OIDC identity provider.
Health?	A liveness endpoint that checks nothing external, a readiness endpoint that checks the database with a real query, and a proxy health check.
Resources?	For the api, 2 CPUs and 1 GiB carry about 1,000 signed-in reads a second or 500 concurrent streams; it idles at about 170 MiB. Every container has CPU and memory limits and no swap.
Configuration?	A documented settings table, secrets marked; production refuses unsafe values at startup.
Logs, metrics, traces?	JSON lines on standard output with a request id; Prometheus metrics; optional OpenTelemetry traces with ids and counts, never content.
Deploy order and stopping?	Migrations, then the api (rolling, readiness-gated), then web. On SIGTERM, up to 120 seconds of drain for streams.
Outbound traffic?	The model provider and the identity provider over HTTPS; certificates; optionally a trace collector. Nothing else.

THEY ASK	THE REFERENCE SERVICE'S ANSWER
Scale and its limits?	The api is stateless and scales out; the connection budget allows about 12 replicas before the pooler needs raising; the model provider's quota binds first in real use.
Backups?	Postgres only, copied off the host; the cache is disposable. A backup counts once it has been restored.

The road to real users

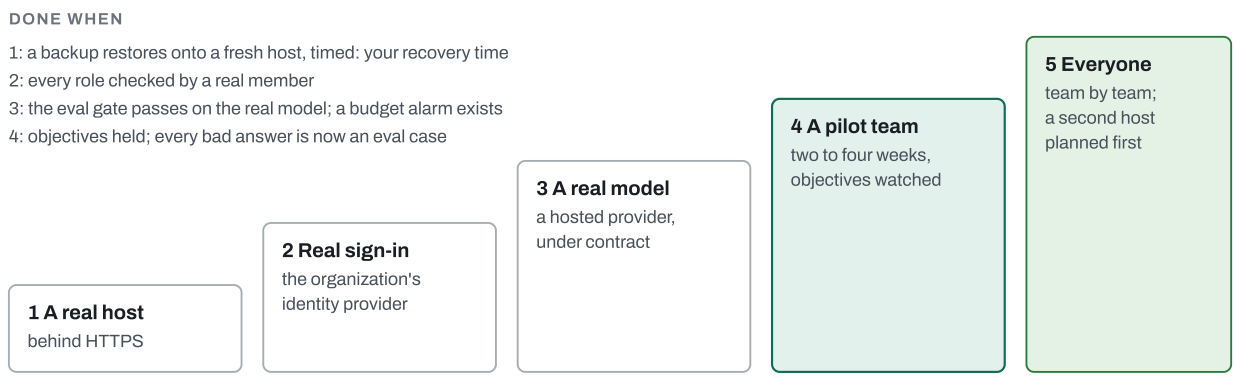


Figure 13.3 Staged exposure. Each stage adds one new kind of risk (real infrastructure, real identity, a real model, real users) and ends in a measurable "done when", not a feeling.

> HOW TO SAY IT
>
> *"Every kind of change has a rehearsed rollback: code by redeploying the previous version, features by flag, and the schema never, because migrations are expand-then-contract."*
>
> *"We roll out in stages, and each stage ends in a measurement: a timed restore, real sign-ins per role, the eval gate on the production model, a pilot that held its objectives."*

Traps

SYMPTOM	CAUSE	FIX
A rollback fails with "Can't locate revision"	The older release's migration tool does not know the newer revision	Detect a database ahead of the image and roll back the code only.
After an upgrade, every row seems to have vanished	A release that contracts was reached by skipping the one that expanded	Never skip such releases; state the minimum previous version in the release notes.

SYMPTOM	CAUSE	FIX
A downgrade under pressure loses a column's data	Automatic schema reversal	Never reverse migrations automatically; fix forward.
A regression reaches every user before anyone notices	Rolling deploys move all traffic in seconds	A canary compared on metrics, or a flag, for changes whose risk is in the numbers.
Clients break after a release	A breaking API change was not announced	Breaking changes go in the release notes, with the migration path.
Old feature flags nobody dares remove	Flags without an owner or an end date	Remove a flag once its feature is fully on; track flags like debt.

Questions

1. What does a canary catch that a rolling deploy does not?
2. Why is a column rename done across several releases?
3. What should a deploy do when the database is ahead of the image it is deploying?
4. Why does each stage on the road to real users need a "done when"?

Answers

1. Regressions that do not crash anything but show in the numbers: higher error rates, slower answers, worse quality. A rolling deploy only waits for health checks, then moves all traffic within seconds.
2. Because during a rolling deploy the old code still runs against the database. Adding the new column, writing both, backfilling, switching reads while still writing both, then stopping writes to the old column, and only later dropping it keeps every running version working, and every rollback to the release before correct, at every step.
3. Treat it as a rollback: skip migrations, say so, and run the older code on the newer schema. Reversing a migration is a separate decision made by a person.
4. Each stage introduces one new kind of risk. A measurable exit (a timed restore, real sign-ins per role, the eval gate on the production model) proves that risk is handled before the next one is added.

14 Cost, teardown and lessons learned

PART VII · OPERATING IT

IN ONE SENTENCE

Cloud cost is a design property you can see per hour and per component, teardown is a procedure worth writing down and testing, and the small operational mistakes of a real build are the lessons most worth passing on.

AT A GLANCE

- Split stacks by what bills by the hour, so the expensive parts can be destroyed and recreated in minutes.
- Know the hourly price of every component before it runs; the reference ECS stack cost about $0.17 an hour.
- Teardown has an order: first stop anything that would recreate the stack, then destroy, then check for leftovers.
- A failed teardown is fixed in the code too, so the next one works.
- Most lost time in a real build comes from small habits: reading the failing step, chaining commands safely, never killing by pattern, never sharing a checkout.

What it cost, per hour

The reference ECS stack's cost is written into its own code, component by component. Roughly half is compute, and the rest is spread over the managed services that replaced containers.

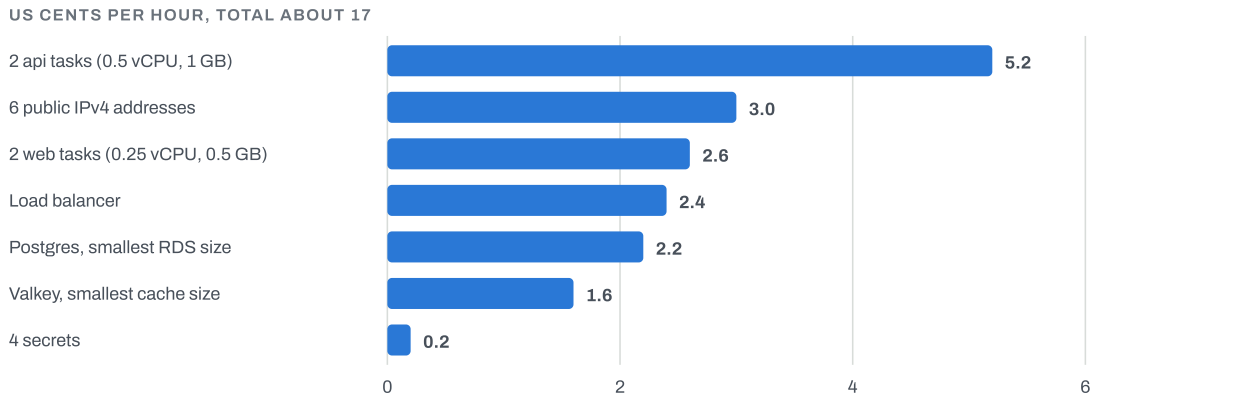


Figure 14.1 Where the money went, per hour, in the region used. Public IPv4 addresses, which many people assume are free, were the second-largest item: one reason production networks keep tasks on private addresses.

Two numbers put this in perspective. The first full build of the reference deployment, a VM for about 45 minutes and the ECS stack for about three hours, came to roughly sixty cents; each later session on the ECS stack added cents more, about thirteen for the end-to-end test. And the permanent parts (image

repositories holding about 1.3 GB, the CI roles, the network) cost cents a month, so they stay, and the pipeline keeps copying each release into the registry.

Teardown, in order

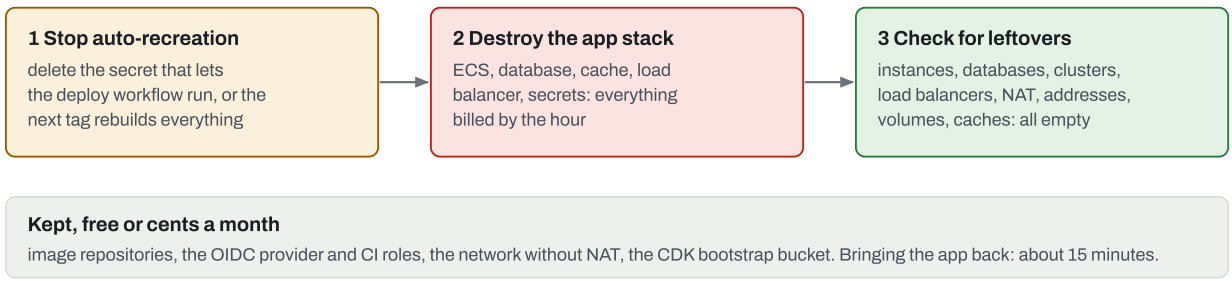


Figure 14.2 Teardown has an order. The first step is the one people forget: with the deploy workflow wired up, any later release tag would start recreating the whole stack, one approval away.

IN THE REPO `infra/aws/ops.sh` the leftovers check

A small operator script holds the commands people need under time pressure. Its `leftovers` command lists what could still be billing after a teardown, one resource type per line: instances, databases, caches, clusters, load balancers, NAT gateways, addresses, volumes, secrets and stacks. It does not cover everything an account can hold, such as database snapshots, so an empty list means this stack left nothing behind, not that the account is empty.

- It checks stopped instances too: a stopped VM still pays for its disk.
- It lists stacks in every state but deleted. A stack in `DELETE_FAILED`, the failure described below, still holds what it could not delete. The first version listed only completed stacks and missed the cache; a review of the script found both.

```
leftovers)
  echo "EC2:";  aws ec2 describe-instances --filters Name=instance-state-
name,Values=pending,running,stopping,stopped --query 'Reservations[].Instances[].
[InstanceId,InstanceType,State.Name]' --output text
  echo "RDS:";  aws rds describe-db-instances --query 'DBInstances[].
[DBInstanceIdentifier,DBInstanceStatus]' --output text
  echo "ElastiCache:"; aws elasticache describe-replication-groups --query 'ReplicationGroups[].
[ReplicationGroupId,Status]' --output text
  echo "ECS:";  aws ecs list-clusters --query clusterArns --output text
  echo "ALB:";  aws elbv2 describe-load-balancers --query 'LoadBalancers[].LoadBalancerName' --output
text
  echo "NAT:";  aws ec2 describe-nat-gateways --filter Name=state,Values=available,pending --query
'NatGateways[].NatGatewayId' --output text
  echo "EIP:";  aws ec2 describe-addresses --query 'Addresses[].PublicIp' --output text
  echo "EBS:";  aws ec2 describe-volumes --query 'Volumes[].[VolumeId,Size,State]' --output text
  echo "Secrets:"; aws secretsmanager list-secrets --query 'SecretList[].Name' --output text
  # Every state but deleted: a stack stuck in DELETE_FAILED still holds what it could not delete.
  echo "Stacks:"; aws cloudformation list-stacks --query 'StackSummaries[?
StackStatus!=`DELETE_COMPLETE`].[StackName,StackStatus]' --output text ;;
```

The teardown failed once. CloudFormation deleted the database and its password secret within two seconds of each other, because the template referred to the secret only inside a dynamic reference, which does not order deletes. The database was already gone, but its delete handler could not read the password, so the stack ended in `DELETE_FAILED`. It was finished by deleting the stack again while retaining the resources that no longer existed, and fixed in code with an explicit dependency (Chapter 7). A fuller check afterwards, beyond what the script covers, found nothing left: no instances, databases, clusters, load balancers, addresses, volumes, caches, alarms, secrets or snapshots.

Lessons from a real build

None of these is dramatic. Together they cost more time than any of the cloud's own surprises.

SYMPTOM	CAUSE	FIX
A security check failed on a pull request that changed nothing related	A new advisory against the base image; main already had the fix	Read the failing step before touching anything; update the branch.
A rate-limit test failed once on main and passed on its pull request	The test depends on where a one-minute window falls	Record it as a flake and make the timing controllable; never retry it silently.
`pkill -f pattern` killed the operator's own shell	The pattern also matched the command line running the kill	Kill by process ID, never by a pattern the killer itself contains.
Code that had just failed lint was committed	Commands chained with `;`	Chain with `&&`.

SYMPTOM	CAUSE	FIX
A branch and its files vanished mid-task; commits appeared that nobody on the task made	Another person or automated coding agent was working in the same checkout and switching branches	One worktree per parallel session; check the current branch before each commit; stage explicit paths; read `git log` on main before pushing.
Build output containing account details was swept into a commit	A broad `git add` swept in build output	Ignore generated directories from the first commit; history keeps what was pushed.
An infrastructure pull request could not be merged by its only author	Required reviews, and no second reviewer yet	An administrator override, used deliberately and flagged in the pull request.

The shared-checkout incident deserves one more sentence, because coding agents make it common. The safe response was the same as for any surprise: stop, read `git status`, `git log` and the open pull requests before changing anything; follow the decision already merged rather than fighting it; recover files from Git's own history; and rebuild the change on a fresh branch from the current main.

> HOW TO SAY IT
>
> *"We know the hourly cost of every component, and everything billed by the hour lives in one stack we can destroy and recreate in about fifteen minutes. Teardown is scripted, including the leftovers check."*

Questions

1. Why is the first teardown step disabling the deploy workflow, not destroying the stack?
2. Which item of the reference stack's hourly cost surprises people most, and what does it suggest?
3. What should happen after a teardown fails halfway?

Answers

1. With the deploy workflow wired up, the next release tag would start a deploy that recreates the whole stack once someone approves it. Deleting the secret that lets the deploy run makes the teardown stick.
2. Public IPv4 addresses, the second-largest item. Production networks keep tasks on private addresses behind a load balancer, which also removes them from the internet.
3. Finish it (for example by retaining resources that are already gone), check for leftovers, and fix the cause in code so the next teardown works.

APPENDIX

A Glossary

Terms used in this handbook, each with the chapter that explains it. Application terms are in Handbook 2's glossary; AI terms in Handbook 1's.

Application Load Balancer (ALB) AWS's HTTP load balancer: terminates TLS, routes by path and host, and checks target health. Chapter 9

Artifact A versioned output of the pipeline, such as an image, built once and promoted unchanged between environments. Chapter 3

Blue/green deployment Running a second full environment on the new version and switching traffic at once, keeping the old one for a switch back. Chapter 13

Branch protection Repository rules that require checks and reviews before anything reaches the main branch. Chapter 2

Canary Sending a small share of traffic to a new version, watching it, and widening in steps. Chapter 13

CDK The AWS Cloud Development Kit: infrastructure written in a programming language and synthesized to CloudFormation. Chapter 7

cloud-init A tool that configures a new virtual machine on first boot from user data. Chapter 6

CloudFormation AWS's native infrastructure-as-code service, which creates and updates stacks of resources. Chapter 7

Continuous delivery Every change that passes the pipeline is releasable, and releasing is a routine, automated step. Chapter 3

Dependabot GitHub's bot that opens pull requests to update dependencies and pinned images. Chapter 4

Drift Differences between declared infrastructure and what actually runs, usually from manual changes. Chapter 7

ECR Amazon Elastic Container Registry: private image storage in an AWS account. Chapter 5

ECS Amazon Elastic Container Service: runs containers as tasks and services, on EC2 or on Fargate. Chapter 9

EKS Amazon Elastic Kubernetes Service: managed Kubernetes control planes. Chapter 10

Environment A separate copy of the system (development, staging, production) with its own configuration and data. Chapter 13

Fargate Serverless compute for containers on AWS: no instances to manage. Chapter 9

Feature flag A switch that turns code on or off per tenant or percentage without a deploy. Chapter 13

GitHub Actions GitHub's CI/CD: workflows of jobs triggered by repository events. Chapter 3

gitleaks A scanner for secrets committed to Git history. Chapter 4

IAM role	An AWS identity with permissions that a service, a task or a federated identity assumes temporarily. Chapter 5
Immutable infrastructure	Replacing servers or containers with new ones built from an image, rather than changing them in place. Chapter 6
Infrastructure as code (IaC)	Declaring infrastructure in reviewed, versioned code that a tool applies. Chapter 7
Kubernetes	A container orchestrator: pods, deployments and services scheduled across a cluster of nodes. Chapter 10
Least privilege	Granting each identity only the permissions its job needs. Chapter 5
NAT gateway	Lets resources in private subnets reach the internet without being reachable from it; priced per hour and per gigabyte. Chapter 8
OIDC federation	A CI job proves its identity to a cloud with a short-lived token and assumes a role: no stored keys. Chapter 5
Pinning	Referring to an action, image or dependency by an immutable identifier (a commit SHA, a digest) rather than a movable tag. Chapter 4
Pull request	A proposed change with its review, checks and discussion, merged into the main branch when approved. Chapter 2
Rollback	Returning to the previous known-good version; for code, redeploying the previous tag. Chapter 13
Rolling deployment	Replacing instances a few at a time, new ones healthy before old ones drain. Chapter 13
SBOM	A software bill of materials: the list of components inside an artifact. Chapter 3
Secrets Manager	AWS's store for secrets, which can generate, rotate and inject them at runtime. Chapter 5
Security group	A stateful AWS firewall attached to resources, allowing traffic by port and source. Chapter 8
Semantic versioning	MAJOR.MINOR.PATCH version numbers that signal breaking changes, features and fixes. Chapter 3
Subnet	A range of addresses in a VPC within one availability zone; public if routed to an internet gateway, private otherwise. Chapter 8
Supply chain	Everything that goes into an artifact (dependencies, base images, actions, the build itself) and can be attacked. Chapter 4
Tag (release)	A Git tag such as v1.2.3 that marks a release and, in the reference pipeline, triggers image publishing. Chapter 3
Task definition	ECS's description of a task's containers: images, CPU, memory, environment, secrets and roles. Chapter 9
Teardown	Deleting everything a stack created, in the order its dependencies require, and verifying nothing billable remains. Chapter 14
Trivy	A scanner for vulnerabilities in images, dependencies and configuration. Chapter 4
Trunk-based development	Short-lived branches merged often into one main branch, which stays

releasable. Chapter 2

VPC — A virtual private cloud: an isolated network in an AWS account, divided into subnets. Chapter 8

VPC endpoint — A private connection from a VPC to an AWS service, avoiding the NAT gateway and the internet. Chapter 8

APPENDIX

B Decision index

RECORD	DECISION	DISCUSSED IN
`ADR-0011`	Releases are version tags that publish images; one host deploys by rolling	1
`ADR-0015`	Rollbacks roll back the code, never the schema	13
`ADR-0022`	Scan images and commits; pin base images and scanners by digest	4
`ADR-0026`		4

APPENDIX

C Repository index

FILE	QUOTED IN CHAPTER
`.trivyignore.yaml`	4
`.github/CODEOWNERS`	2
`.github/dependabot.yml`	4
`.github/pull_request_template.md`	2
`.github/workflows/ci.yml`	3
`.github/workflows/deploy.yml`	5, 12
`.github/workflows/release.yml`	1, 3
`apps/api/Dockerfile`	4
`apps/api/app/routes/health.py`	10
`infra/aws/README.md`	12
`infra/aws/app.py`	7
`infra/aws/cicd.py`	5
`infra/aws/container/api.sh`	12
`infra/aws/container/migrate.sh`	13
`infra/aws/network.py`	8
`infra/aws/ops.sh`	9, 14
`infra/aws/registry.py`	12
`infra/aws/service.py`	5, 7, 9, 12
`infra/aws/vm.py`	5, 6
`scripts/check_trivyignore.py`	4

FILE	QUOTED IN CHAPTER
`scripts/deploy.sh`	6

APPENDIX

D Trap index

SYMPTOM	FIX	CHAPTER
The dev server never reloads; tests run on stale files	Move it to the distribution's home directory.	1 · From laptop to production
Everything is slow: installs, searches, Git status	Linux file system; install Node and Python inside the distribution; trim the Windows PATH from the Linux one if needed.	1 · From laptop to production
A script fails in the container with "no such file", and the file exists	The `.gitattributes` rule; renormalize once; `core.autocrlf` off.	1 · From laptop to production
Builds die mid-way; the machine freezes	Limits in `.wslconfig`; the agent's tool memory cap; restart the distribution after changing them.	1 · From laptop to production
Docker Desktop refuses to start, or "cannot connect to the daemon"	One engine per distribution; update WSL with the web download when the store is blocked.	1 · From laptop to production
Package downloads fail with certificate errors in a build	Install it in all three places; pass it to the build.	1 · From laptop to production
DNS stops after connecting to the VPN	Disable generation in `wsl.conf`; write the resolver yourself; or mirrored networking mode.	1 · From laptop to production
The agent's sandbox is unavailable	WSL2 with the two packages; a profile for the sandbox on distributions that need it; otherwise a container for unattended runs.	1 · From laptop to production
A published port is unreachable from the browser	Keep the default NAT mode unless a tool needs otherwise; check the ignored-ports setting in mirrored mode.	1 · From laptop to production
A colleague's laptop behaves differently from yours	Ship the dev container definition in the repository; the toolchain, the agent and its allowlist come with it.	1 · From laptop to production

SYMPTOM	FIX	CHAPTER
A rollback has nothing to go back to	Deploy exact version tags, or digests, from a registry with immutable tags.	1 · From laptop to production
Two servers on the same version behave differently	Build once in CI; hosts only pull.	1 · From laptop to production
Every release replaces a component that did not change	Compare builds by a hash of their inputs, and replace a component only when its inputs changed.	1 · From laptop to production
An image tag now points at different code	Make the release workflow the only pusher, use immutable tags, verify signed build provenance, or deploy by digest.	1 · From laptop to production
Images from a public repository can be pulled by anyone	Expect it: never put a secret in an image.	1 · From laptop to production
An undo of one experiment also lost other uncommitted work in the same file	Commit first, or undo the experiment with a reverse patch.	2 · Git and the pull request
Teammates' branches no longer line up with yours after a rebase	Rebase only your own unpushed work; use `--force-with-lease` on your own branch, never on a shared one.	2 · Git and the pull request
A service restarted mid-merge runs conflict markers as code	Production is never a developer checkout; deploy built releases.	2 · Git and the pull request
A solo owner cannot merge their own pull request	Until a second reviewer exists, merge with the admin bypass, deliberately and noted in the description.	2 · Git and the pull request
Branch protection is missing on a private repository	Budget for it, or keep the repository public.	2 · Git and the pull request
Shell scripts fail with odd errors after a checkout on Windows	A `.gitattributes` file that forces LF.	2 · Git and the pull request

SYMPTOM	FIX	CHAPTER
Your branch vanishes and files disappear while you work	One `git worktree` per parallel session; confirm the current branch before every commit; stage explicit paths, never everything.	2 · Git and the pull request
"No checks" right after opening a pull request	Look at the workflow runs for the branch rather than trusting an empty answer.	2 · Git and the pull request
A commit went in even though lint had just failed	Chain with `&&`, so a failing check stops the chain.	2 · Git and the pull request
Nobody can tell which commits were generated	An `Assisted-by:` trailer, enforced by a commit-message hook; the committer still owns the change.	2 · Git and the pull request
The review bot approved; a real bug merged	Comment-only mode with repo-owned rules; required checks and a human approval remain the gate.	2 · Git and the pull request
A copied release runs on one processor type and fails on the other	Copy the whole index with `docker buildx imagetools create` or `skopeo copy --all`.	3 · CI/CD and artifacts
`exec format error` **when a container starts**	Build in CI for both, or pass `--platform linux/amd64` when building locally.	3 · CI/CD and artifacts
The registry's scan never ran on a copied image	Scan in the pipeline, before publishing, and do not rely on the registry alone.	3 · CI/CD and artifacts
An old vulnerability blocks an unrelated pull request	Read the failing step before changing anything; update the branch from main.	3 · CI/CD and artifacts
A release has to be cut again just to retry one failed job	Re-run only the failed jobs of the same run.	3 · CI/CD and artifacts
A timing-sensitive test fails once on main and passes on retry	Treat it as a real flake: make time controllable in the test, or assert on behavior that does not depend on the clock.	3 · CI/CD and artifacts
A self-hosted runner ran an outsider's code	Use hosted runners for public repositories; isolate self-hosted ones.	3 · CI/CD and artifacts

SYMPTOM	FIX	CHAPTER
A trusted CI action starts exfiltrating secrets	Pin every third-party action to a full commit SHA.	4 · The software supply chain
The scanner itself is the attack	Run scanners from images pinned by digest, with no Docker socket.	4 · The software supply chain
Builds differ depending on the day they ran	Pin the base by digest and take fixes as a reviewed digest bump.	4 · The software supply chain
Everyone ignores the red security check	Block only findings with a fix; accept a fix you cannot take yet, with an expiry; report third-party images.	4 · The software supply chain
A pull request goes red for a vulnerability it did not introduce	Take the bot's digest bump, or accept the risk with an expiry.	4 · The software supply chain
The secret-scanning action fails for an organization	Run its command-line tool instead.	4 · The software supply chain
A leaked key was deleted from the repository, but it is still abused	Rotate first, then remove.	4 · The software supply chain
An agent added a package that does not exist, or a look-alike	Installs only from a lockfile the pipeline checks; a registry allowlist where one exists.	4 · The software supply chain
A pinned scanner action ran attacker code	Pin every action to a full commit SHA and let the update bot move the SHA.	4 · The software supply chain
The job fails with `Not authorized to perform sts:AssumeRoleWithWebIdentity`, and the policy looks right	Read the real subject from the repository's OIDC setting, or print the token's claims in a test job, and match on that.	5 · Identity without keys
No token is issued at all	Grant it on the job that needs it, not the whole workflow.	5 · Identity without keys
A deploy job that worked on tags is refused after you add an environment	Trust the environment in the deploy role. It is the better gate anyway.	5 · Identity without keys

SYMPTOM	FIX	CHAPTER
Anyone who can push can publish images to the cloud registry	A tag ruleset: only admins create, move or delete release tags.	5 · Identity without keys
A deliberate break of the policy fails, so the policy seems proven	Run the passing case first, then the failing one.	5 · Identity without keys
You cannot scope the registry login permission to one repository	Grant it on `*`, scope push and pull to the repositories, and say why in a comment.	5 · Identity without keys
The CDK deploy role can do anything in the account	Narrow the execution policy before production use.	5 · Identity without keys
A workflow input runs as shell code	Pass inputs through `env:` and quote them, as `deploy.yml` does.	5 · Identity without keys
Every deploy causes a burst of 502 errors	Start the new version beside the old, wait for health, then drain the old.	6 · The deployment ladder, and one VM done properly
A script that worked yesterday cannot find the api container	Never hardcode a container name; ask Compose for the service's container.	6 · The deployment ladder, and one VM done properly
A routine `compose up` quietly rolled the service back	The deploy records the version in the env file, so every later command uses it.	6 · The deployment ladder, and one VM done properly
The VM boots without Docker or the deploy user, and no error appears	Validate it with `cloud-init schema` before deploying, in CI.	6 · The deployment ladder, and one VM done properly
A first-boot loop ran as two broken commands	Parse the YAML and run the resulting command once in a test.	6 · The deployment ladder, and one VM done properly
The cloud refuses the instance type	List the eligible types first and pick one that meets the stack's minimum.	6 · The deployment ladder, and one VM done properly

SYMPTOM	FIX	CHAPTER
The backup was lost with the server	Copy dumps off the host, and rehearse a restore on a clean one.	6 · The deployment ladder, and one VM done properly
A certificate expired without warning	Monitor certificate expiry yourself.	6 · The deployment ladder, and one VM done properly
`No export named … found` when deploying one stack	Deploy the producing stack first; cross-stack references are ordered dependencies.	7 · Infrastructure as code
A change made in the console disappeared	Run drift detection, and make every change in code.	7 · Infrastructure as code
A stack delete ends in `DELETE_FAILED`	Add an explicit dependency; finish a stuck delete by retaining the resources that are already gone.	7 · Infrastructure as code
A dependency fix creates a cycle	Depend on the specific resource; synthesize and lint the template in CI.	7 · Infrastructure as code
The account ID ended up in a public repository	Ignore generated output from the first commit; read the account from the environment.	7 · Infrastructure as code
The tool cannot work out the account from a sign-in profile	Export the account explicitly in the environment before running it.	7 · Infrastructure as code
Secrets readable in the Terraform state	Remote, encrypted state with restricted access and a lock.	7 · Infrastructure as code
A small edit to the first-boot script replaced the server	Expect it; keep data off the instance and the boot script small.	7 · Infrastructure as code
The database will not create in a zone the catalog says it supports	Add a subnet in another zone; plan for at least three possible zones for data.	8 · Networking on AWS
Removing a security group rule did not cut the application off	Know that revoking access does not end live sessions; stop the resource for a real outage test.	8 · Networking on AWS

SYMPTOM	FIX	CHAPTER
The bill shows large data processing charges	VPC endpoints for ECR and S3.	8 · Networking on AWS
Every model call fails after a network hardening change	Allow the provider's hostname explicitly; configure the corporate proxy where there is one.	8 · Networking on AWS
Public IP addresses appear on the bill	Private subnets in production; count public addresses in a cheap design.	8 · Networking on AWS
The site breaks when the load balancer's IP changes	Use an alias record that points at the load balancer's name.	8 · Networking on AWS
A command run through ECS Exec cannot find a setting the service uses	Build it the same way in the operator's command.	9 · Containers on ECS Fargate
A database outage makes ECS replace healthy api tasks	Check a shallow endpoint; detect outages with a 5xx alarm.	9 · Containers on ECS Fargate
A task sits in PENDING for many minutes, billed	Make the essential container depend directly on every init container, each with a start timeout.	9 · Containers on ECS Fargate
Rate limits are higher than configured after scaling out	A shared store for anything that must be counted once.	9 · Containers on ECS Fargate
Two new tasks run migrations at the same moment	Make each run take a lock (a Postgres session advisory lock) so concurrent runs take turns, or run migrations as a one-off task before the deploy. Starting one new task at a time is not enough: the first deploy and a scale-out still start several.	9 · Containers on ECS Fargate
The web container's proxy returns 502 for every api call on Fargate	Route `/api` at the load balancer. Service Connect alone does not help this nginx: it publishes names in `/etc/hosts`, which nginx's runtime resolver never reads.	9 · Containers on ECS Fargate
Streams are cut during a deploy	Set the deregistration delay above the longest answer, then the stop timeout (Fargate allows up to two minutes) above the server's graceful drain.	9 · Containers on ECS Fargate

SYMPTOM	FIX	CHAPTER
A short spike overwhelms the service before autoscaling reacts	Size the minimum task count for spikes; scale on a leading metric.	9 · Containers on ECS Fargate
An outage is reported that the load balancer never saw	Judge outages from server-side metrics, and run synthetic checks from inside the cloud.	9 · Containers on ECS Fargate
A database outage restarts every pod at once	Liveness checks only the process; readiness checks hard dependencies.	10 · Kubernetes and EKS
A partial outage takes the whole service out of rotation	Report soft dependencies as degraded without failing readiness.	10 · Kubernetes and EKS
The team spends its time on the cluster, not the product	Use a managed container platform until many services or a platform team make the cluster pay.	10 · Kubernetes and EKS
A cluster costs money with nothing running	Count it in the cost floor; share clusters across services.	10 · Kubernetes and EKS
Most services answer "not signed up" on a brand-new cloud account	Finish verification, then check access with read-only calls before building.	12 · The reference build, end to end
A monitoring probe reports failures the service never had	Split the client's timings; compare with load balancer metrics; probe from inside the cloud.	12 · The reference build, end to end
A probe killed mid-run prints no summary	Trap the termination signal and print the counts.	12 · The reference build, end to end
A client chose its own rate-limit bucket	Trust only the load balancer's addresses: the VPC range.	12 · The reference build, end to end
Long answers cut during every deploy	Set it to the longest request; ECS sends SIGTERM only after it ends.	12 · The reference build, end to end
An alert channel full of expected warnings	Alert on 5xx, latency and unhealthy targets.	12 · The reference build, end to end

SYMPTOM	FIX	CHAPTER
A rollback fails with "Can't locate revision"	Detect a database ahead of the image and roll back the code only.	13 · Environments, releases and rollback
After an upgrade, every row seems to have vanished	Never skip such releases; state the minimum previous version in the release notes.	13 · Environments, releases and rollback
A downgrade under pressure loses a column's data	Never reverse migrations automatically; fix forward.	13 · Environments, releases and rollback
A regression reaches every user before anyone notices	A canary compared on metrics, or a flag, for changes whose risk is in the numbers.	13 · Environments, releases and rollback
Clients break after a release	Breaking changes go in the release notes, with the migration path.	13 · Environments, releases and rollback
Old feature flags nobody dares remove	Remove a flag once its feature is fully on; track flags like debt.	13 · Environments, releases and rollback
A security check failed on a pull request that changed nothing related	Read the failing step before touching anything; update the branch.	14 · Cost, teardown and lessons learned
A rate-limit test failed once on main and passed on its pull request	Record it as a flake and make the timing controllable; never retry it silently.	14 · Cost, teardown and lessons learned
`pkill -f pattern` killed the operator's own shell	Kill by process ID, never by a pattern the killer itself contains.	14 · Cost, teardown and lessons learned
Code that had just failed lint was committed	Chain with `&&`.	14 · Cost, teardown and lessons learned
A branch and its files vanished mid-task; commits appeared that nobody on the task made	One worktree per parallel session; check the current branch before each commit; stage explicit paths; read `git log` on main before pushing.	14 · Cost, teardown and lessons learned

SYMPTOM	FIX	CHAPTER
Build output containing account details was swept into a commit	Ignore generated directories from the first commit; history keeps what was pushed.	14 · Cost, teardown and lessons learned
An infrastructure pull request could not be merged by its only author	An administrator override, used deliberately and flagged in the pull request.	14 · Cost, teardown and lessons learned

Engineering Handbooks · Handbook 3 of 3 · Cloud Delivery & DevOps · Edition 1

Reference implementation: triage-assistant at `handbook-1 (unknown)`.